

1. VHDL/Verilog의 필요성.

이번 Chapter에서 다룰 내용은 다분히 Hardware적인 색깔이 짙은 내용이다.

일반적으로 어떠한 system을 제작하기 위해서는 당연히 해당 system을 구축하기 위한 hardware를 구상해야 된다.
그런데, 이 hardware를 개발하는 데 있어서 가장 중요한 기술은 다음과 같이 3가지라고 보면 된다.

① CPU 및 주변 Devices에 대한 시험 program의 개발 능력.
▶ C/C++ Coding 능력을 의미. 그러나, 주로 C언어를 사용한다.
② CPU 및 주변 Devices간의 원활한 신호처리를 위한 Controller와 임의의 알고리즘을 수행해 주는 Chip 설계 능력.
▶ VHDL/Verilog Coding 능력을 의미.
③ Analog 회로 개발 관련 능력.

실질적으로, 어떠한 board를 개발해 달라는 의뢰가 왔을 때, 대부분의 개발자들은 이전에 개발했던, 회로를 일단 검토하여 의뢰자가 원하는 사양에 맞도록 회로를 개발해 나간다.
그러나, 대부분의 경우, 위에서 언급한 ①~③번은 그대로 사용할 수 없고, 수정을 해주어야 한다. 그래서, 실재로 업무가 주어졌을 때, 일단, 원활히, board를 개발할 수 있으려면, 기존에 얼마나 많은 회로 설계 경험이 있었는 지, 그리고, 얼마나 다양한 안정된 회로도를 가지고 있는지가 중요한 능력이 된다. 그리고 나서, ①~③번을 수행할 수 있는 능력이 그 기술자의 연봉을 결정한다고 보면 될 것이다.

위에서 언급한 ①번을 수행하기 위해서는 C/C++(거의 C언어로 작성한다.) Coding능력뿐만 아니라, 사용하는 devices에 대한 정확한 register들의 분석과 timing도를 이해해야 개발 할 수 있으므로, 일반적으로 Hardware 독립적으로 개발하는 Windows Program을 개발하는 것과는 차원이 다르다.

사실, RTOS를 기반으로 한 device driver의 개발은 일반적으로 hardware 개발자의 몫은 아니다. 보통, hardware 개발자는 OS없이 하나의 task로 주어진 board의 동작을 시험할 수 있는 시험 program만 개발 할 수 있으면 된다.
실질적으로 hardware 개발자의 능력은 ②번과 ③번에 좌우된다고 보면 된다.

VHDL/Verilog는 입·출력의 timing이 주어졌을 때, 그 기능을 수행하도록 일종의 만능 보드와 같은 Chip에 기능을 부여하는 것이다.
이 말이 잘 이해가 가지 않는 분은 계속해서 이번 단원을 읽어보면 이해가 될 것이다.

③번에서 언급한 analog 회로 개발이란, 주로 Power와 Clock관련 회로 및 device에 첨가되는

주변 RLC회로를 말한다. 그러나, 이 분야는 사실, 상당히, 어렵고 경험을 많이 요구하므로 주로 외국에서 관련 기술을 도입하거나, 해당 device의 datasheet에서 제공하는 reference 회로를 참고하는 것이 현실이다.

처음부터 모든 것을 다 얻을 수는 없고, 이번 Chapter에서는 주로 Matlab의 Simulink를 사용하여 VHDL Code 개발에 주력을 하도록 하자.

ASIC뿐만 아니라, FPGA, CPLD를 포함한 대부분의 비메모리 소자의 개발자들은 주로 전통적인 Text 기반의 VHDL code를 개발해 왔고, 개발하고 있는 실정이다.

약어 정리
ASIC : Application Specific Integrated Circuit.
FPGA : Field Programmable Gate Array.
CPLD : Complex Programmable Logic Device
VHDL : VHSIC Hardware Description Language
VHSIC : Very High Speed Integrated Circuit
Verilog : Cadence Design Systems Inc.의 trademark

그러나, Text 기반의 VHDL code 개발은 많은 시간과 경험을 요구하며, 무엇보다 유지·보수가 힘들다. 그래서, 외국에서는 Text 기반의 VHDL Code보다는 주로 Visual Tool인 mentor의 Renoir를 사용하고 있다.
현재, 발표되고 있는 FPGA는 1000만 gate급 이상이다. 그러므로, 개발한 VHDL Code가 차지하는 용량문제보다는 주로 알고리즘의 원활한 개발, 즉, 기능 구현에 중점을 두고 있다.
마치, 과거에 PC에서 사용하는 Hard Disk의 용량이 적어서 Application의 크기를 줄이는 것에 중점을 두었다가, 수십 Giga Bytes의 대용량 Hard Disk가 나오면서 Application의 크기보다는 기능에 중점을 둔 것과 같다.

이번 Chapter에서 소개할 내용은 Matlab의 Simulink와 Xilinx의 System Generator를 사용하여 통신관련 **IP(Intellectual Property)**를 사용하는 방법을 소개하게 될 것이다.

우리가 보통, Computer를 조립할 때, 모니터, RAM, CPU, 마더보드 등등의 제품들을 각각 구입하여 하나의 완성된 Computer로 조립한다.
VHDL Code도 마찬가지로 하나의 완성된 Computer를 만드는 것이 아니라, 모니터, RAM 등과 같이 일정부분의 기능만 수행하도록 만든 Code를 보통, **IP**라고 부른다.
예를 들면, Computer에서 주로 사용하는 System Bus인 PCI Bus의 기능만 수행하는 PCI IP도 있고, Digital Filter의 기능만 수행하는 IP도 있다.
Hand Phone에서는 대부분 ARM CPU를 사용하는 데, 이 ARM Core도 IP 형태로 판매되고 있다.
Windows Program에서 본다면, COM(Component Object Model)에 해당한다.

사실, 우수한 기능을 수행하는 IP만 만들어서 사업을 하는 회사도 많이 있다. 대부분의 IP는 수천만원에서 수백억을 하는 경우가 비일비재하고, IP를 사용하는 Chip당 로얄티를 지불하는 경우도 있다.
그러나, IP로 판매를 한다는 것은 그 IP에 대한 모든 기능상의 책임을 진다는 것과 같다. 즉, IP를 구입한 회사에서 그 IP를 사용하는 중에 문제가 발생하여 손해를 보게되면, 그에 대한 모든 배상을 책임져야 한다. 결국, 잘못된 IP를 만든다면, 쫄딱망할 수도 있다.

어쨌든, Xilinx System Generator에서 제공하는 IP는 결국, 그 성능이 믿을 만 하다는 것을 알 수 있다.

이번 Chapter는 Chapter 10.에서 배운 2진 연산에 대한 이해와 Chapter 15.에서 배운 Simulink에 대한 사전 지식이 필요하다.

이제, 2진 연산과 Simulink에 대한 이해를 바탕으로 우선, VHDL에 대해서 간단히 살펴보고, Xilinx System Generator의 사용 방법을 살펴보게 될 것이다.
결국, 이번 Chapter에서는 Simulink와 VHDL 관련 tool인 Xilinx Foundation Series4.1i, Modelsim, Leonardo Spectrum, Active-HDL과 같이 다양한 Tool들에 대한 사용방법을 배우고, 처음 듣는(?) VHDL 전문 용어들을 배우게 될 것이다.
그리고, 앞으로 Xilinx Foundation Series 대신에 사용될 Xilinx ISE version에 대해서는 [부록]에서 간단히 설명할 것이다. 그러나, Xilinx ISE version은 합성과 simulation 환경만 바뀌었고, 실질적인 Chip의 P&R에 대해서는 Xilinx Foundation Series와 동일하다.
P&R에 대해서는 뒤에서 설명할 것이다.

만일, Hardware를 한번 자기 손으로 꾸미고 싶다면, 이번 Chapter는 상당히 유익한 내용이 될 것이다. 그러므로, 쉽게 포기하지 말기 바란다.
또한, 이 책에서는 시중에 있는 VHDL 관련 서적과 같이 자세한 VHDL Syntax에 대한 설명보다는 실질적으로 합성 및 구현하는 방법에 초점을 두고 설명할 것이다.
앞으로 빠른 시일내에 강력한 최신 tool로 무장한 VHDL 관련 서적을 출간할 계획이다. 그때 부족한 부분을 배우기 바란다.

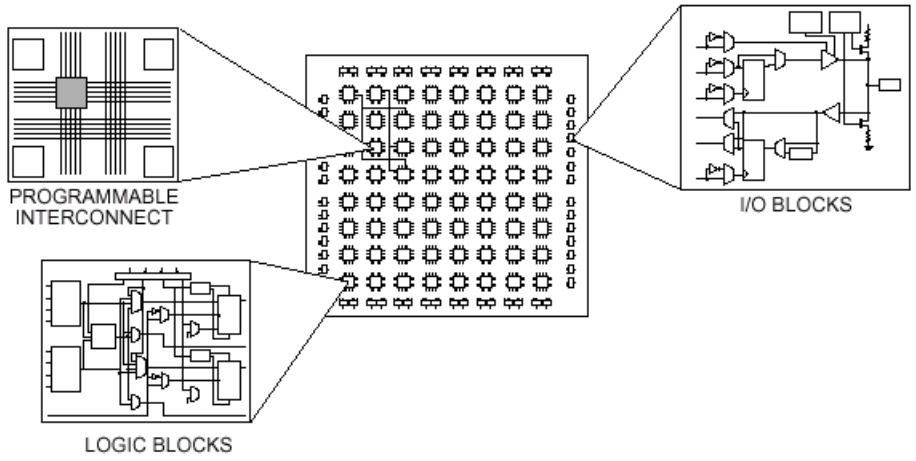


어떠한 기능을 수행하는 board를 설계하기 위해서는 여러 가지 종류의 부품(device)들을 사용해야 한다. 그러나, 이들 부품들은 서로 다른 회사에서 만들다 보니, clock에 따른 동작 특성이 저마다 다른 것이 보통이다.
이와 같이 서로 다른 동작 특성을 갖는 부품들 간에 정보 교환을 원활하게 하기 위해서 일종의 중계 역활을 해주는 Chip이 필요하게 된다. 그러나, 사용자가 어떤 부품을

사용할지는 아무도 모르기 때문에 이처럼 중계 역활을 해주는 Controller Chip은 대부분 구입할 수가 없고 자신이 직접 Chip을 설계 해 주어야 한다.
또는, 빠른 속도를 요구하는 알고리즘을 구현하고 싶을 때는 보통, 고속의 CPU에 C/C++ code로 알고리즘을 구현하기보다는 능력만 되면, Chip으로 구현하는 경우가 많다.

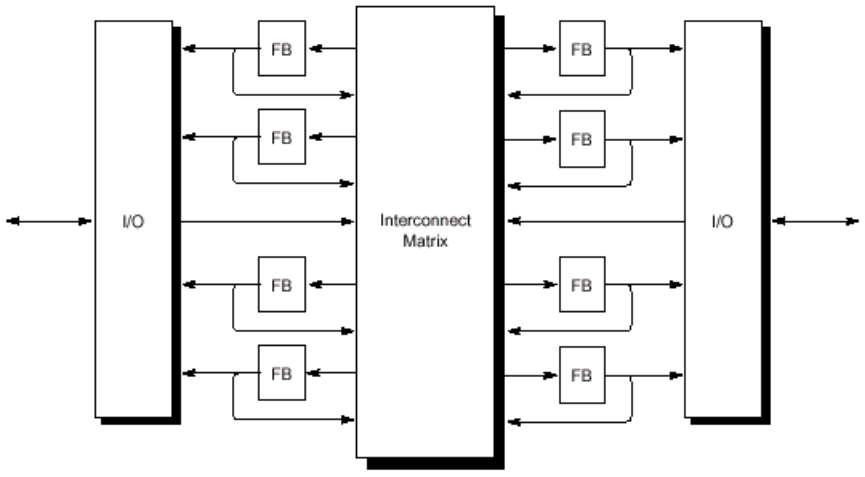
우리가 시중에서 사는 대부분의 IC 소자들은 일정한 기능을 갖고 있다. 즉, CPU 기능, IO 기능, Memory 기능, 제어 기능 등등 , 그러나, FPGA와 CPLD라고 불리는 소자들은 내부에 어떠한 기능이 부여되어 있지 않은 것이 보통이다. 즉, Chip의 내부가 백지와 같은 상태이므로, 사용자가 VHDL/Verilog와 같은 HDL(Hardware Description Language)을 사용하여 기능을 부여해 주어야 한다.

PLD는 보통, FPGA와 CPLD로 나뉘는 데, [그림 19-1]는 FPGA의 내부 구조를 보여 주고 있다.



[그림 19-1] FPGA의 내부 구조.

그리고, [그림 19-2]는 CPLD의 내부 구조를 보여주고 있다.



[그림 19-2] CPLD의 내부 구조

FPGA는 작은 CLB(Configurable Logic Block)라고 하는 block들이 수많이 모여있고, 이들은 Interconnection path를 통하여 서로 간에 연결되고, IOB(Input Output Block)를 통하여 외부와 연결된다. 그러나, CPLD는 FPGA의 CLB에 비교해서 상당히 커다란 Matrix(구체적으로 Switch Matrix) 또는 Macro Cell로 비교적 간단하게 구성되어 있다.

이처럼 FPGA는 수많은 CLB로 구성되어 있고, 무엇보다 반도체 기술의 공정이 발달할수록 더 많은 CLB를 내장 할 수 있는 가능성이 있다. 그래서, FPGA는 요사이 1000만 gate급 이상이 출시되고 있지만, CPLD는 보통 커봐야 10만 gate정도이다.
그러나, FPGA로 Chip을 설계하려는 경우, 이들 수많은 CLB가 서로 간에 연결이 되어야 하므로 결국, CLB간의 line 사이에서 신호 전달로 인한 delay가 발생하는 문제가 있다. 이와 같은 delay를 routing delay라고 한다. 반면에, CPLD는 하나의 커다란 Switch Matrix 또는 macro cell로 그 내부 구조에서 발생하는 delay는 일정한 특징을 갖기 때문에 routing delay를 예측하기도 쉽고, 실질적으로 delay가 거의 발생하지도 않는다.

이밖에도 FPGA를 사용하기 위해서는 갖가지 delay들을 고려해 주어야 한다.

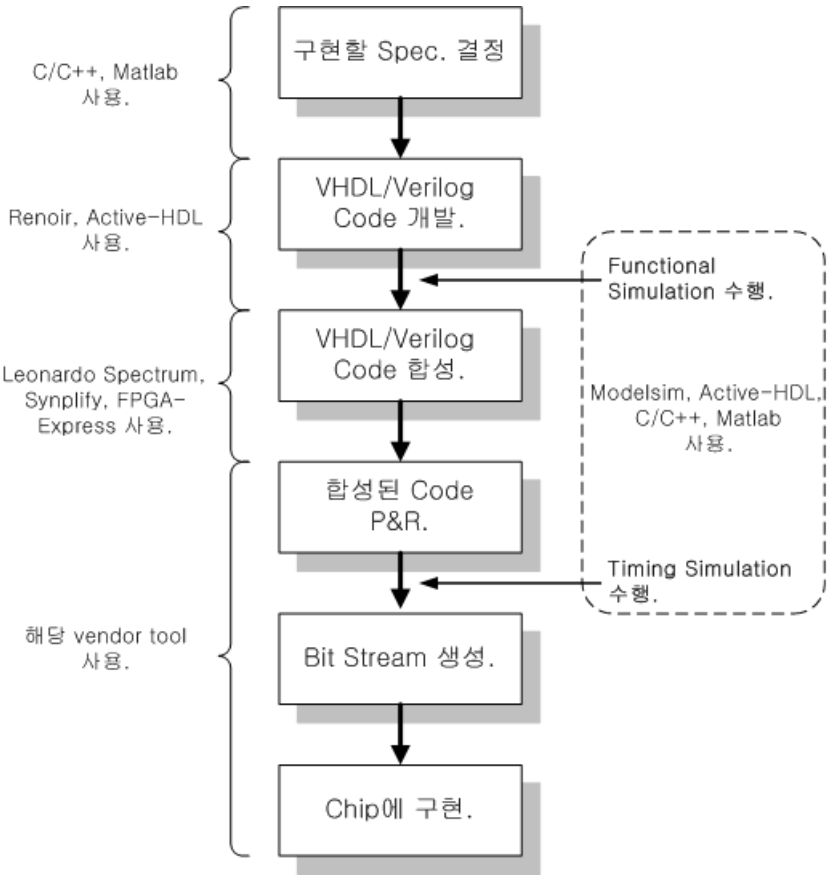
어쨌든, VHDL/Verilog를 사용하여 FPGA/CPLD에 기능을 부여해 주기 위해서는 [그림 19-3]과 같은 과정을 통하여 bit stream code를 만들어 주어야 한다. 바로 이 bit stream code가 실질적으로 FPGA/CPLD에 저장되어 새로운 기능을 부여해 준다.

대부분의 Chip 개발자들은 UltraEditor와 같은 editor만으로 VHDL code를 개발하는 것이 상례이다. 그러나, 필자의 경우에는 Renoir와 같은 Visual Tool과 Active-HDL과 같은 막강한 tool을 사용하여 Code를 개발하고, Active-HDL 또는 Modelsim을 사용하여 Simulation을 수행한다.

합성(synthesize)이라는 것은 VHDL/Verilog와 같은 text 기반의 문서를 바탕으로 사용할 device(FPGA, CPLD)에 logic을 구현하는 작업을 말하며, P&R(Place And Routing)이라는 것은 실질적으로 합성에 의해서 만들어진 logic을 사용할 device에 배치 및 선(line)들을 연결 시켜주는 작업을 말한다.

일반적으로 합성 작업은 Synopsys의 FPGA-Express, 또는 Mentor의 Leonardo Spectrum, 그리고 Synplicity의 Synplify를 사용한다. 이두저도 없으면 vendor에서 제공하는 tool을 사용하면 된다.
그러나, 전문 합성 tool을 사용하는 것이 보다 효율이 좋은 것은 당연한 사실이다.
Synopsys의 Synopsys design compiler가 가장 우수한 합성능력을 가지고 있지만, 이것은 너무 고가이고, 대부분 ASIC 개발용으로 사용한다. FPGA 계열에서는 Synplify가 가장 우수한 합성 능력을 가지고 있지만, latch가 발생하는 HDL Code를 합성하는 경우에는 문제가 있다고 한다. 그러나, latch가 발생하는 HDL Code를 만드는 것은 처음부터 좋지 않은 방법이다.

요즘, 각 Device vendor들은 앞에서 언급한 전문 합성 tool 회사로부터 OEM 방식으로 자신의 Software에 해당 합성 tool을 끼워서 공급해 주는 추세이다. 그러나, 이들 전문 합성 tool들은 보통 수천만원을 할 정도로 고가이므로 Device vendor에서 제공하는 OEM software는 극히 기능이 제한되어 있다.



[그림 19-3] VHDL Code 개발 과정.

VHDL/Verilog Code를 개발하는 과정에서 재대로 code를 개발하였는지, 중간에 Simulation을 수행하게 되는데, VHDL/Verilog Code에**만** 기반을 두고 Simulation을 하는 것을 **Functional Simulation**이라고 한다. 그리고, 합성이 끝난 뒤에, 실질적으로 사용할 device의 특성, 즉, delay를 고려하여 simulation을 하는 것을 **Timing Simulation**이라고 한다.
이때 VHDL의 경우 주로 사용되는 것이 **Modelsim, Active-HDL**이고, Verilog의 경우에는 **Verilog-XL**이 사용된다.

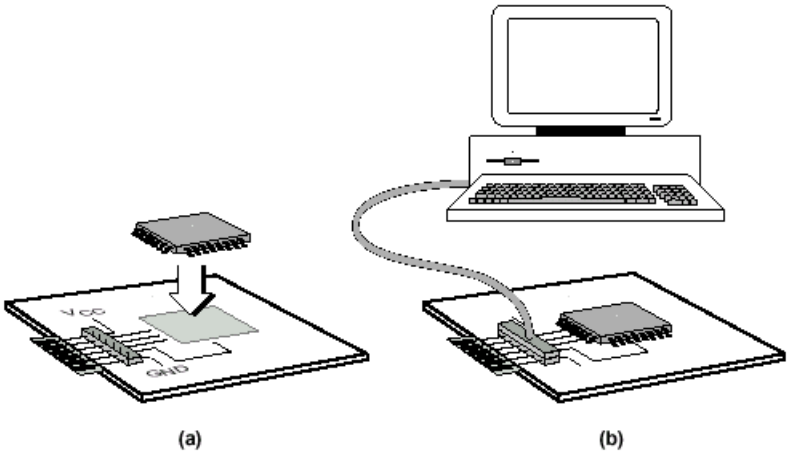
여러분이 주의 할 것은 CPLD의 경우 delay 성분이 적으므로 functional simulation으로 충분하지만, FPGA의 경우, 충분한 경험이 없으면, delay 성분을 고려하지 않은 Functional

Simulation과 delay 성분을 고려한 Timing Simulation의 결과는 다르게 나오는 것이 보통이다. 그러나, 실제로 device가 동작하는 timing flow는 timing simulation과 비슷하므로, Code 수정이 불가피해 진다. 물론, 많은 시간을 들여서 이 문제를 해결하면서 경험을 쌓는 것이 좋겠지만, 필자의 경우, Renoir를 사용하여 이 문제를 해결한다.

경험으로 비추어 볼 때, 45[MHz]까지는 Modelsim의 Timing simulation 결과가 바로 Device의 실질적인 동작 특성과 같다. 그런데, Renoir에서 만들어주는 VHDL/Verilog Code는 이와 같은 Timing Simulation을 만족해 주며, 실질적으로 직접 Text Editor를 사용한 Code보다 Chip 용량을 그다지 많이 차지하지는 않는다. 그러나, Renoir는 OEM 방식으로 vendor를 통하여 얻을 수 없고, 직접 구입해야 하는 데, 역시, 수천만원하는 고가이므로 접하기 쉽지 않은 tool이다.

또 하나 주의 할 것은 P&R과 bit stream code는 사용하려는 device의 vendor가 제공하는 tool을 사용해야 된다. 왜냐하면, P&R과 bit stream code생성은 어디까지나 device 의존적이기 때문이다.

최종적으로 bit stream code를 만들었으면, [그림 19-4]와 같이 PC를 이용하여 download cable로 device 또는 PROM에 bit stream을 저장하면 된다.



[그림 19-4] bit stream downloading.

일반적으로 CPLD는 ROM 구조를 가지므로 일단, CPLD에 bit stream이 저장되면, 전원이 나가도 계속해서 데이터를 가지고 있지만, FPGA는 RAM 구조이므로, 전원이 나가면, FPGA에 저장되어 있던 데이터는 모두 없어진다. 그래서, FPGA의 경우에는 PROM에 데이터를 저장하고, 전원이 들어오면 자동으로 PROM에 저장한 데이터가 FPGA에 들어가도록 설계를 한다.

[그림 19-4]에서 보여준 downloading cable은 각 vendor에서 제공을 해 주며, 만일, 얻기가

힘들면 해당 vendor의 홈페이지에 가보면 제작하는 방법이 설명되어 있다. 사실, downloading cable 제작은 간단하므로 쉽게 제작할 수 있을 것이다.

현재, 전 세계 PLD 시장은 CPLD급으로는 Lattice가, 그리고 FPGA급으로는 Altera와 Xilinx가 양분하고 있다.
아마도, 대부분의 engineer들은 CPLD급의 Lattice를 주로 사용할 것이고, 특별히, 전문적인 Chip 개발자들만 FPGA를 사용하여 알고리즘을 구현할 것이다.

여기서 주로 다룰 target device는 Xilinx Device이다. 왜냐하면, Simulink와 연결되는 software를 개발한 업체가 Xilinx이기 때문이다. Altera에서 공급되는 Software는 대부분 Altera 회사 자체에서 개발하는 데, Xilinx는 대부분의 Software를 전문 업체에게 의뢰하여 개발한다. 그러다 보니, software 자체가 Altera보다 안정적이고, 정교한게 사실이다.
또한, Xilinx에서는 XFS(Xilinx Foundation Series) 한 개의 software로 모든 Xilinx Device를 사용할 수 있지만, Altera에서 공급하는 software는 중·저용량을 위한 Maxplus-II, 대용량의 Quartus-II, 이렇게 2종류의 software를 따로 사용해야 하는 것이 흠이다.
그리고, Xilinx의 XFS는 software의 기능에 따라서 각각 program을 독립적으로 분리해서 사용할 수 있으므로 메모리 부담이 작지만, Altera에서 공급하는 software는 분리가 안되고, 항상 커다란 한 개의 program을 실행해야 하므로 메모리 부담이 커서 잘못하면 PC가 쭝나는 수가 있다.
물론, 구현하려고 하는 device에 따라서 다르기는 하지만, 일반적으로 Xilinx의 XFS를 사용하려는 경우에는 PC의 메모리 용량이 256[MBytes] 정도면 되지만, Altera의 Quartus-II를 사용하려면, 기본적으로 512[MBytes] 이상이 준비되어 있어야 한다.

그러나, 현재, 국내 대부분 업체에서는 Altera device를 많이 사용하고 있고, 회사 자체에서도 Altera를 선호한다. 그것은 아마도, 적극적인 marketing과 다양한 무료 교육, 그리고, software에 대한 무상 공급(?)으로 인한 것 같다.

자! 이제 본격적으로 VHDL Coding 기법을 간단히 설명하고, Simulink에서 Xilinx의 program을 사용하여 구현하기 힘든 VHDL Code를 어떻게 개발하는 지 살펴보도록 하겠다.



여기서 다룰 VHDL Coding 기법은 다분히 개괄적이다. 즉, VHDL에 대한 모든 syntax를 논하지 않고, 아주 간단한 내용만 다룰 것이다.
이번 단원을 통하여 여러분은 VHDL Code가 어떤 구조이고, 어떻게 Simulation을 하고, 어떻게 합성 및 P&R을 하고, 또한, bit stream code를 어떻게 만드는 지, 전반적인 흐름을 맛

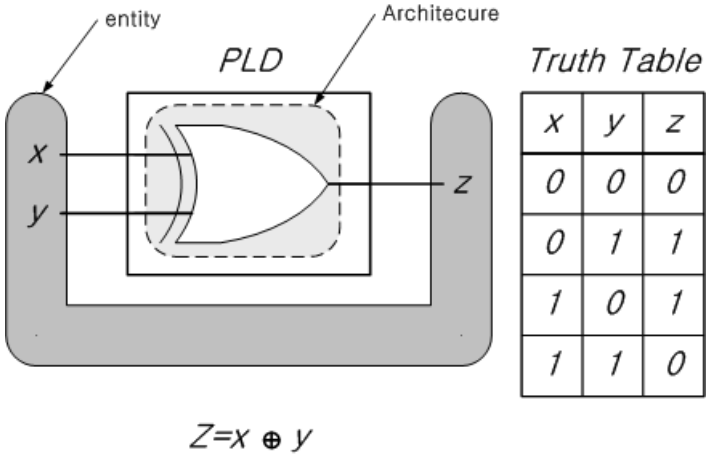
보게 될 것이다. 부족한 syntax 문제는 현재, 시중에서 판매되고 있는 전문 서적을 참고하거나, 차후 필자가 출시할 VHDL 책자를 기대해 보기 바란다.

아마도, VHDL을 처음 접하는 분들에게는 매우 인상적인 단원이 되리라고 본다.

이번 단원에서는 주로 **Functional Simulation**을 수행하는 방법에 대해서 주로 설명 하고, 다음 단원에서 Timing Simulationp 대해서 설명할 것이다.

이번 단원에서 구현할 logic은 [그림 19-5]와 같이 하나의 Exclusive-OR gate이다.
기존에 VHDL을 학습한 분들에게는 너무나도 간단한 예제라고 생각할지도 모른다.
그러나, 이번 단원의 목표는 [그림 19-3]에서 보여준 VHDL flow의 전 과정을 설명하는 것이다. 또한, 특정 vendor의 device에 상관없이 꼭 알아두어야 할 공통 사항들에 대해서 설명할 것이다. 그리고, VHDL을 처음 경험하는 분들을 위해서 배려한 부분이므로, 자세히 설명할 것이다.

일단, [그림 19-5]에서 보여준 PLD는 입력이 x, y 2개이고, 출력이 z 하나로 구성되어 있다.



x	y	z
0	0	0
0	1	1
1	0	1
1	1	0

[그림 19-5] Exclusive-OR gate.

일반적으로, FPGA는 수백개의 I/O pin을 가지고 있는데, 이중에서 2개는 입력 pin으로 1개는 출력 pin으로 각각의 pin을 설정해 주면 된다. 이처럼 pin을 설정해 주는 작업을 **pin locking**이라고 하며, **constraint file**에서 지정해 준다. xilinx의 경우, ***.ucf** file이 constraint file이다.

그리고, PLD 소자에 신호가 입·출력되는 x, y, z를 **port**라고 부르며, 이들 port의 종류로는 입력 port(**in**), 출력 port(**out**), 입·출력 port(**inout**), buffer type의 port(**buffer**), 이렇게 4종류가 있다.

또한, [그림 19-5]에서 보여준 것과 같이, PLD 내부의 기능과는 무관하게 단지 PLD 외부의 입・출력 port를 정의하는 것을 entity 선언문이라고 하며, PLD 내부의 기능을 구현하는 것을 architecture 선언문이라고 한다.

① entity 선언문의 형식 :

```
entity entity_name is
    [ port( 해당 port 선언 list);]
end [entity] [entity_name];
```

여기서, 각괄호(즉, **[,]**)는 생략해도 되는 부분이다. 상식적으로 port가 없는 entity가 무슨 의미가 있겠는가? 생각할지도 모르지만, 뒤에서 배울 **testbench file**을 만들 때, 그 의문의 해답을 얻을 수 있을 것이다.
또한, 주의 할 것은 모든 VHDL문은 C/C++ Code와 같이 **semicolon**(즉, **;**)으로 문장의 끝을 알려 주어야 한다. 그러나, [그림 19-5]에서 보여준 것과 같이 마지막 port에는 semicolon을 사용하지 않는다. 또한, **is** 지정어 뒤에도 semicolon을 사용하지 않는다. 왜냐하면, **is**로 문장이 끝난다면 이상하지 않겠는가?

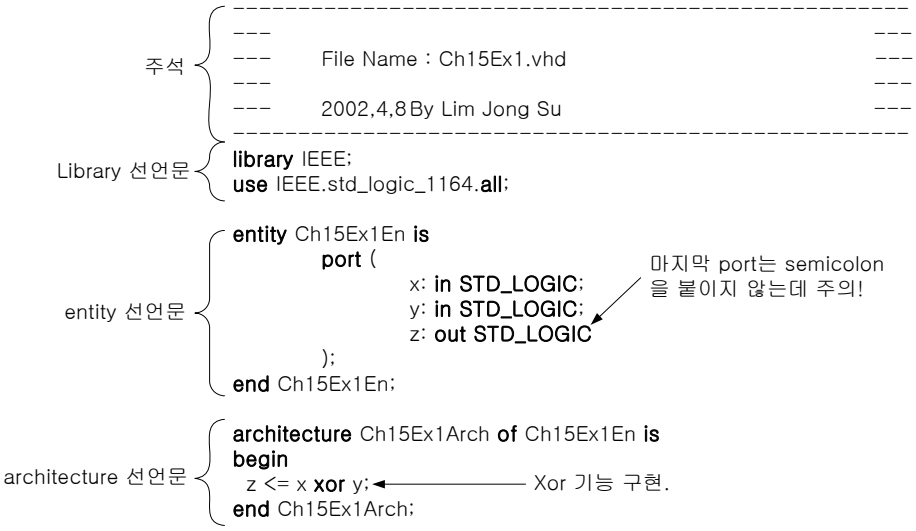
② architecture 선언문 :

```
architecture architecture_name of  entity_name is
    [ declarations ]
begin
    architecture body
end [architecture] [architecure_name];
```

여기서, 주의할 것은 하나의 entity에 서로 다른 여러 개의 architecture가 사용될 수 있다는 것이다. 왜냐하면, entity문은 단순히 port들을 정의한 문(statement)이므로 여러 가지 서로 다른 architecture로 entity가 선언한 port에 기능을 부여 할 수 있도록 VHDL Coding을 할 수 있기 때문이다.

[그림 19-5]와 같은 logic을 VHDL 문으로 작성해 보면 다음과 같다.

주의 :
Ch19Ex1.vhd file과 같이 파일이름을 정해야 하는데, 편집과정에서 잘못된 것을 확인하였다. 그러나, 수정할 내용이 너무 많아서 어쩔 수 없이 Ch15Ex1.vhd file의 파일이름을 그대로 사용할 것이므로, 혼돈하지 말기 바란다.



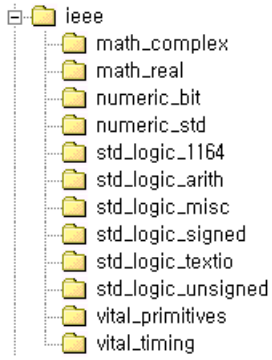
[그림 19-6] VHDL의 기본 구조.

[그림 19-6]에서 진한 글씨체로 표현한 **library, use, all, entity, is, port, in, std_logic, end, architecture, of, begin, xor**등등은 모두 **지정어**들이다.

VHDL은 대·소문자를 구분하지 않는다. 그러므로, library와 LiBrarY는 같은 문자열로 인식한다. 또한, 입력 port **x**와 **X**도 같은 문자로 인식한다.

[그림 19-6]에서 사용한 **library** IEEE;에서 IEEE는 library의 이름이고, 이 IEEE library의 대(大) 제목 하에서 data type, 기능 등에 의해서 분류해 놓은 것을 **package**라고 하며, 현재, [그림 19-6]에서는 std_logic_1164가 package에 해당한다.

std_logic_1164는 IEEE library에 속하는 package이므로, C언어의 구조체에서 field를 구분하는 것과 유사하게, IEEE.std_logic_1164와 같이 **dot(·)**를 사용한다.
IEEE library에는 [그림 19-7]과 같이 많은 package들이 속해 있다.



[그림 19-7] IEEE library에 속한 package들.

OEM 방식으로 전문 simulator인 modelsim을 vendor에서 제공하기 전까지는 [그림 19-7]에서 보여준 모든 package가 제공되지는 않았었다.

특히, VHDL에서 외부 파일을 access할 수 있는 기능을 부여해 주는 **std_logic_textio** package가 지원되지 않았었다. 현재, vendor에서 제공해 주는 package 범위가 어디까지인지는 OEM simulator version인 modelsim을 사용해 본적이 없으므로 설명하기가 힘들다.
그러나, 주위에서 OEM simulator version인 modelsim을 사용하는 분들의 얘기로는 거의 모든 package를 제공한다고 한다.

어쨌든, **use** IEEE.std_logic_1164.**all**은 library IEEE에 속해 있는 std_logic_1164 package를 **모두(all) 사용(use)**하라는 뜻이다.

기존에 C언어에서는 해당 library를 사용할 때, 단순히, library의 경로를 포함한 include만 사용하였지만, VHDL에서는 **use**와 같은 문을 사용하여야 하는데, 이와 같은 작업을 특별히, **가시화**라고 한다.

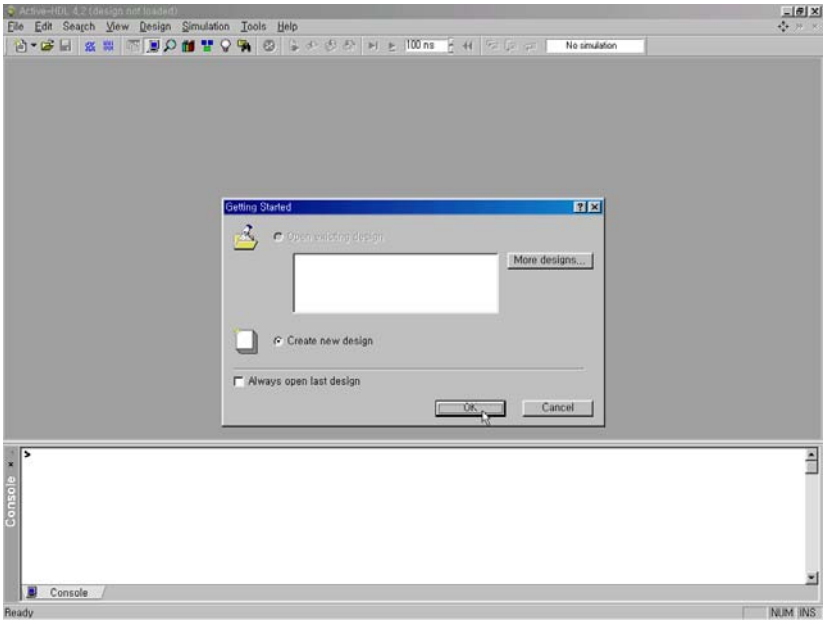
std_logic_1164에는 각각의 port에 대한 신호 레벨들과 여러 가지 함수(**function**)들이 정의되어 있다. 신호 레벨이란, 일반적으로 **high(1)**, **low(0)**, -(**Don't care**)를 의미한다. 물론, 이밖에도 여러 가지 신호 레벨이 더 정의되어 있지만, 솔직히, 알아둘 필요가 없다고 본다.

또한, VHDL에는 function과 procedure라는 C언어의 함수와 같은 기능을 수행하도록 선언해 주는 문들이 있다. 이들 function과 procedure, 그리고 새로운 data type이 모여서 하나의 package를 이루는 것이다.

지금까지 설명한 내용을 다시 한번 잘 숙지해 보면, [그림 19-6]에 있는 Ch15Ex1.vhd file에 대해서 이해를 할 수 있을 것이다.

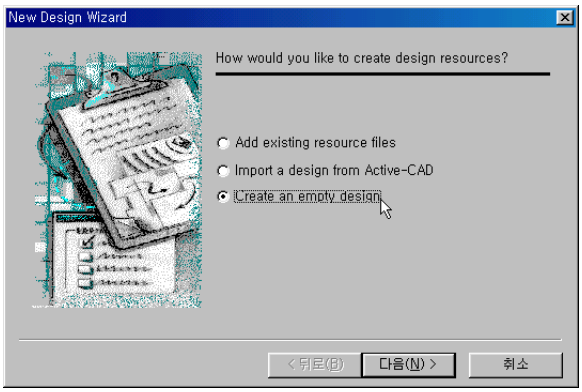
자! 그렇다면, 이제, Ch15Ex1.vhd file에 문법상(syntax)의 error는 없는지, 그리고, 우리가 원하는 exclusive-OR 동작을 제대로 하는지 **Aldec Inc.**의 **Active-HDL**로 검증을 해 보겠다. 사실, Aldec회사의 Active-HDL은 Xilinx의 Foundation series software의 상당 부분을 개발해준 미국의 venture 기업이다. modelsim에 비해서는 그 기능이 다소 떨어지기는 하지만, 솔직히, 대단한 기술력을 갖고 있는 기업이라고 생각한다. 상당히 적은 인력으로 놀라운 기능을 자랑하는 Active-HDL을 개발한 것을 보면, 우리가 배울 것이 많다고 생각한다.
이 software는 Aldec의 홈페이지에 가보면 20일 동안 사용할 수 있는 evaluation version을 구할 수 있다. 현재, 상당히 많은 대학과 연구소에서 사용하고 있다.

Active-HDL을 처음 실행하면, log 화면이 뜨고, [그림 19-8]과 같은 window가 나타난다.



[그림 19-8] Active-HDL 사용 방법(1).

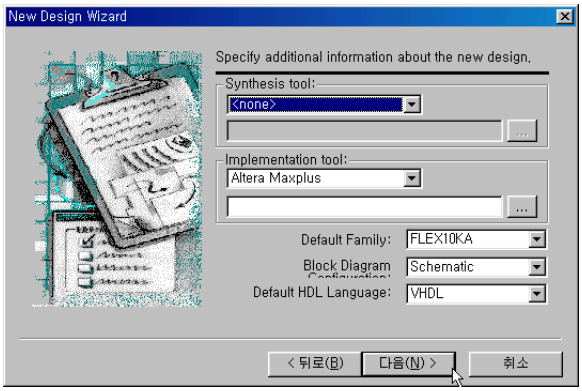
이제, **Getting Started** window에서 **Create new design**을 선택한 뒤에, **OK**버튼을 click 해 준다. 그러면, [그림 19-9]와 같이 **New Design Wizard** window가 나타날 것이다.



[그림 19-9] Active-HDL 사용 방법(2).

우리는 [그림 19-6]에서 보여준 **Ch15Ex1.vhd** file을 새롭게 만들 것이므로, **Create an empty design**을 선택한 뒤에, **다음**버튼을 click해 준다.

[그림 19-9]에서 **다음**버튼을 click해 주면, [그림 19-10]과 같이 두 번째 **New Design Wizard** window가 나타난다.



[그림 19-10] Active-HDL 사용 방법(3).

[그림 19-10]은 Active-HDL을 합성(Synthesis) tool과 사용할 device의 vendor program인 구현(Implementation) tool을 연결시켜주는 option window이다.
만일, 여러분이 [그림 19-10]에 사용할 tool을 적당히 지정해 주면, Active-HDL Main Window에 해당 **menu**가 새롭게 생겨난다.

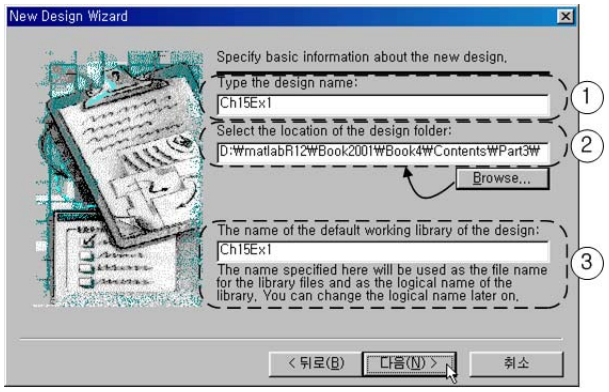
사실, Active-HDL은 전문 Simulation tool로서 합성기능은 없으므로 다른 전문 합성 tool을 사용해야 한다. 그러나, Active-HDL 상에서 합성을 한다는 것은 결국, Active-HDL에서 Timing Simulation을 하겠다는 뜻과 같다. 문제는 Active-HDL로 고성능의 FPGA에 대한

Timing Simulation을 수행하기에는 뭔가 부족한 느낌을 준다. 그래서, 필자의 경우에는 Active-HDL로 Functional Simulation만 하고, Timing Simulation은 전적으로 Modelsim을 사용하기 때문에 [그림 19-10]에서는 특별히, 지정하지 않고, **다음**버튼을 click한다.

이처럼, Functional Simulation은 Active-HDL을 사용하고, Timing Simulation은 Modelsim을 사용하는 이유는, 일단, Active-HDL은 Visual Studio와 같은 project 생성과정을 가지고 있으므로 사용하기가 쉽고, 무엇보다 앞으로 여러분이 배우게 될 강력한 편집기의 기능과 **자동으로 testbench file을 생성**해 주는 기능 때문에 HDL Code의 Simulation 첫 단계로 자주 사용한다.
그리고, 여기서 만들어진 testbench file을 사용하여 modelsim에서 timing simulation을 수행하는 방법을 주로 사용한다. testbench file에 대해서는 뒤에서 설명할 것이다.

만일, 여러분이 Lattice에서 제공하는 CPLD를 주로 사용할 것이라면, 대부분 timing simulation을 할 필요가 없으므로 Active-HDL로 충분할 것이다.

어쨌든, [그림 19-10]에서 **다음** 버튼을 click해 주면, [그림 19-11]과 같이 세 번째 **New Design Wizard** window가 나타날 것이다.



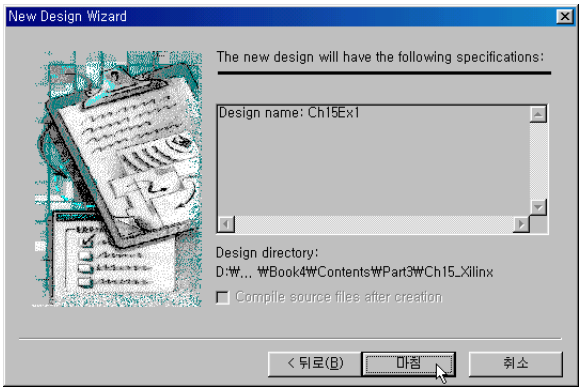
[그림 19-11] Active-HDL 사용 방법(4).

[그림 19-11]에서 ①번은 project의 이름을 의미하며, ②번은 project가 위치할 directory를 지정해 주는 곳이다. 일반적으로 HDL Tool에서는 project라는 단어와 design이라는 단어를 똑같은 의미로 사용한다.

> **주의 :**
> ②번에서 project의 위치를 지정할 때, **한글로 된 directory**를 사용되면 안 된다.
> 반드시, ②번에서는 영문만 사용해야 한다는 데 주의하자!

③번은 project의 working library를 지정하는 곳인데, working library, resource library, logical library의 개념은 뒤에서 배울 **"4. Timing Simulation"**에서 설명할 것이다.

이제, [그림 19-11]을 적당히 지정한 뒤에, **다음** 버튼을 click해 준다. 그러면, [그림 19-12]와 같이 지금까지 생성한 project에 대한 내용이 나타날 것이다.

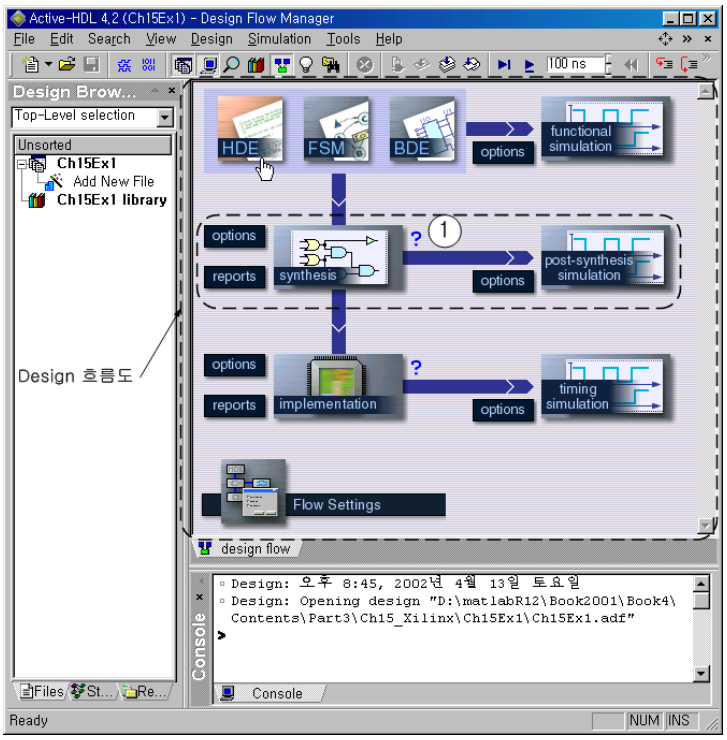


[그림 19-12] Active-HDL 사용 방법(5).

이제, **마침** 버튼을 click해 주면, [그림 19-13]과 같이 Active-HDL의 main window가 나타날 것이다.
[그림 19-13]에서 표시한 **Design 흐름도**는 실질적으로 HDL Code를 사용하여 최종적으로 Chip을 설계하는 순서를 보여주고 있다. 또한, Design 흐름도에 있는 각각의 icon들은 모두 고유한 의미를 가지고 있으며, mouse로 click해 주면 해당하는 작업을 수행해 준다.

> **주의 :**
> 만일, [그림 19-13]과 같은 Design 흐름도가 나타나지 않으면, toolbar에서  icon을 click해 주면 된다.

예를 들어서, HDL Code를 입력하기 원하면, [그림 19-13]과 같이 **HDE icon**을 click해 주면, HDL Code를 생성하는 데 도움을 주는 wizard가 나타나게 된다.
또한, **FSM icon**과 **BDE icon**은 각각 state diagram과 block diagram을 사용하여 simulink와 같이 구현하고자 하는 알고리즘을 states와 blocks의 조합으로 design해 주면, 해당 HDL Code를 자동으로 생성해주는 역할을 수행한다. 무슨 뜻인지 잘 모르겠으면 일단 한번, FSM icon과 BDE icon을 click 해보면 알 수 있을 것이다.

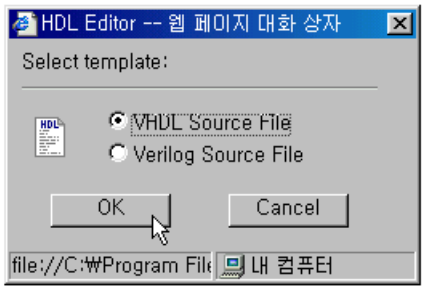


[그림 19-13] Active-HDL 사용 방법(6).

그러나, 만일, HDL Code를 사용하지 않고, vendor에서 제공해 주는 schematic 방법으로 design을 하면 실질적으로 합성할 필요가 없다는 데 주의하자.
각각의 vendor에서 제공해주는 software에는 **자신의 device에서만 사용**할 수 있는 schematic block들을 가지고 있는데, 이들 block들은 해당 vendor의 device에 최적으로 합성된 파일을 각각 가지고 있으므로 재차 합성을 할 필요가 없으며, 실질적으로 HDL Code를 사용하여 Chip을 구현하는 것보다 schematic 방법으로 Chip을 구현하는 것이 더 좋은 것은 당연한 사실이다.
그러나, schematic 방법으로 원하는 알고리즘을 구현한 다는 것은 거의 인간의 머리로는 조금 힘들지 않을 까? 생각한다. 그래서, 고안해 낸 것이 HDL Code인 것이다.

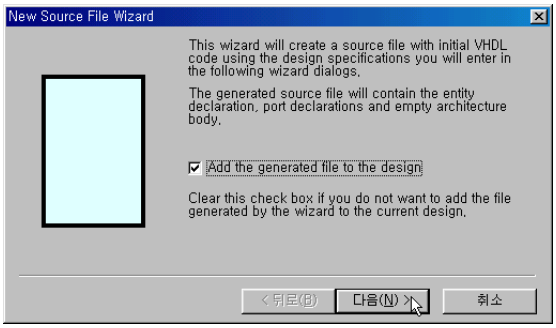
결국, vendor에서 제공해 주는 schematic 방법을 사용한다면, [그림 19-13]에서 ①번에 해당하는 과정은 거치지 않아도 된다. 그러나, 원래, Active-HDL을 개발한 Aldec이 Xilinx의 software를 실질적으로 개발하였으므로, Active-HDL에서 사용할 수 있는 schematic은 필자의 생각으로는 Xilinx device 뿐으로 알고 있다.

어쨌든, HDL Code를 입력하기 위해서 **HDE icon**을 click해 준다. 그러면, [그림 19-14]와 같은 Window가 나타날 것이다.



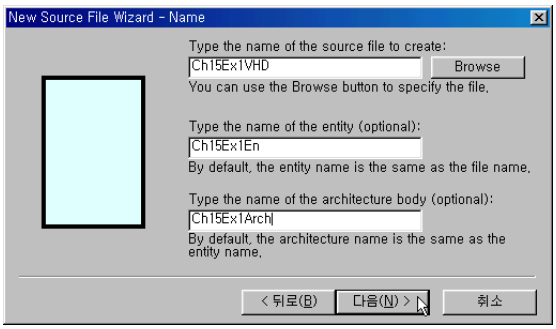
[그림 19-14] Active-HDL 사용 방법(7).

VHDL Source File을 선택하고, **OK**버튼을 click해 준다. 그러면, [그림 19-15]와 같은 window가 나타날 것이다. 생성된 VHDL file은 현재 project에 포함되어야 하므로 [그림 19-15]와 같이 check 상태를 유지하고, **다음** 버튼을 click해 준다.



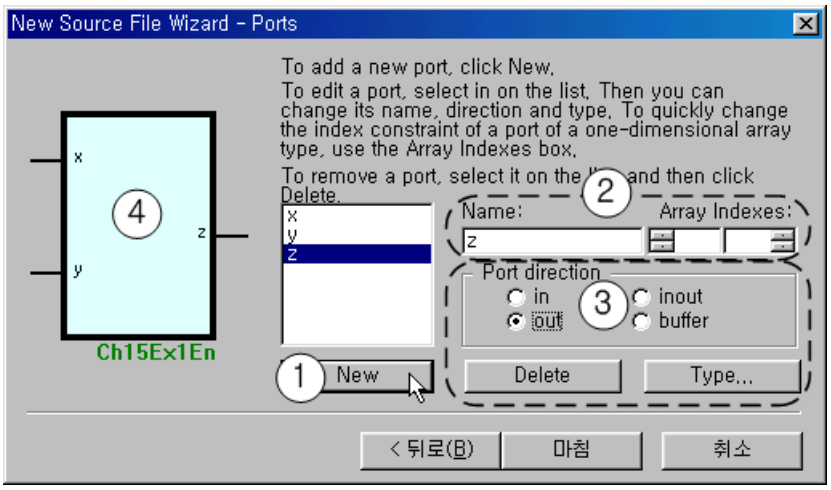
[그림 19-15] Active-HDL 사용 방법(8).

이제, [그림 19-16]과 같이 생성될 VHDL source file의 이름(즉, Ch15Ex1VHD.vhd)과 entity의 이름(즉, Ch15Ex1En), Architecture의 이름(즉, Ch15Ex1Arch)을 각각 지정한 뒤에 **다음** 버튼을 click 해 준다.



[그림 19-16] Active-HDL 사용 방법(9).

[그림 19-17]은 사용할 ports의 개수와 방향 및 type을 지정할 때 사용하는 window이다.



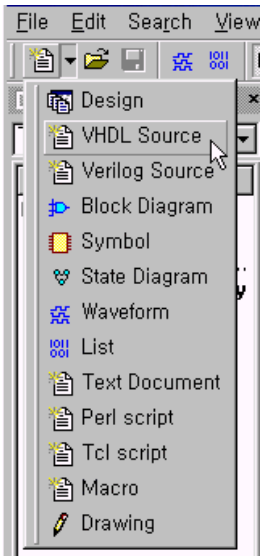
[그림 19-17] Active-HDL 사용 방법(10).

일단, ①번과 같이 **New** 버튼을 click해 준다, 그리고 ②번의 **Name:** 아래에 있는 editbox에 port의 이름을 기입한다. 그리고 나서, ③번과 같이 port의 방향을 결정한다. 이와 같은 방법으로 각각의 port를 추가해 주면 된다.

모든 ports의 추가가 완료되었으면, **마침** 버튼을 click 해 준다.
그러면, Active-HDL이 [그림 19-14]~[그림 19-17]에서 지정한 사항들에 근거하여 해당 Ch15Ex1VHD.vhd file을 자동으로 만들어 준다.

주의 :

만일, [그림 19-14]~[그림 19-17]과 같이 Active-HDL이 자동으로 생성해 주는 file이 필요 없다면, [그림 19-13]에서 아래 그림과 같이 **VHDL Source** menu를 선택해 주면 된다. 그러나, 이와 같이 만든 file은 반드시, **Design** menu에서 **Add Files to Design...** menu를 사용하여 현재 project에 첨가해 주어야 한다는 것에 주의하자.



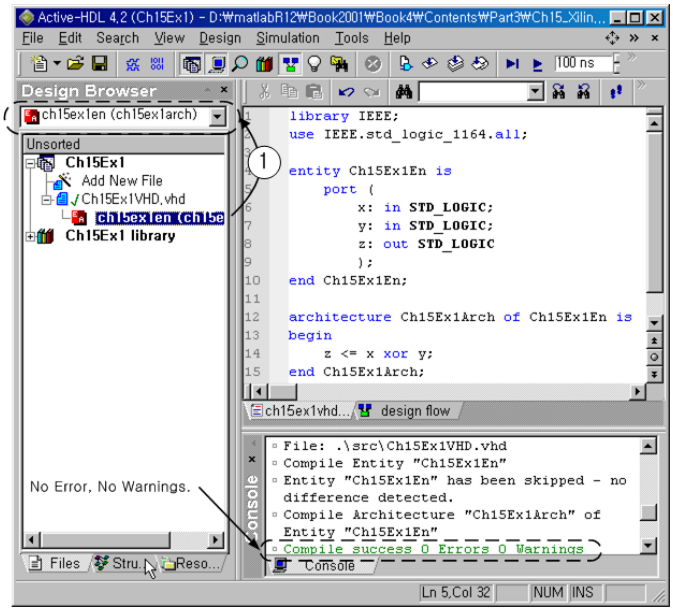
이제, Ch15Ex1VHD.vhd file의 내용을 [그림 19-6]과 같이 편집해 준다.

Active-HDL Main window의 toolbar에 있는 icon을 각각 click 해 보면 Active-HDL이 얼마나 강력한 편집기능을 제공하는 지 느낄 수 있을 것이다. 단, icon의 역기능은 인데 주의하자!

어쨌든, 편집이 끝났으면, 저장을 하고, compilation을 하기 위해서 icon을 click해 준다.

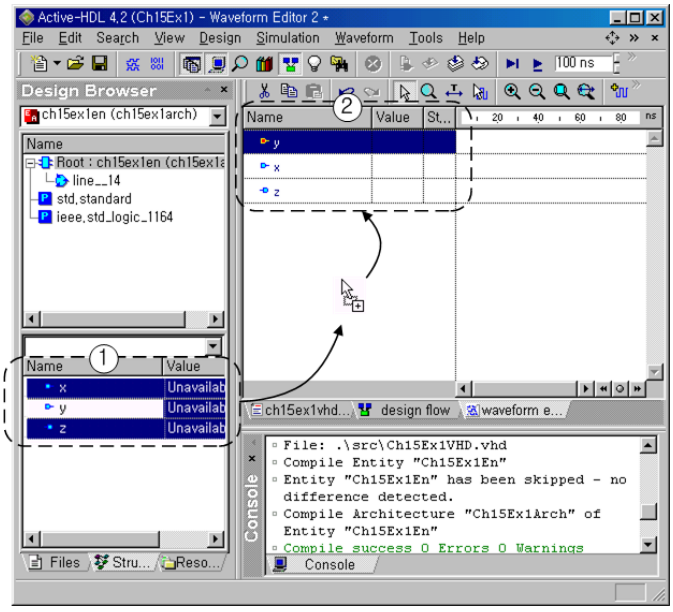
simulator에서 compiling을 한다는 것은 어디까지나, 해당 HDL code를 simulation하기 위해서 수행하는 과정을 말하고, 합성(synthesis)과는 전혀 무관하다는 데 주의하자!

일반적으로 하나의 Chip을 구현하기 위해서는 여러 개의 entity들을 사용할 수 있다. HDL은 계층적 구조를 형성하므로, Top level에서 simulation을 하기 위해서는 [그림 19-18]의 ①번 화살표가 가리키는 곳에 top level에 해당하는 entity/architecture pairs가 지정되어야 한다.



[그림 19-18] Active-HDL 사용 방법(11).

이제, [그림 19-18]과 같이 **Structure tab**을 click해 준 뒤에, wave window를 부르기 위해서, icon을 click해 준다. 그러면, [그림 19-19]와 같이 Active-HDL의 main window가 바뀔 것이다.

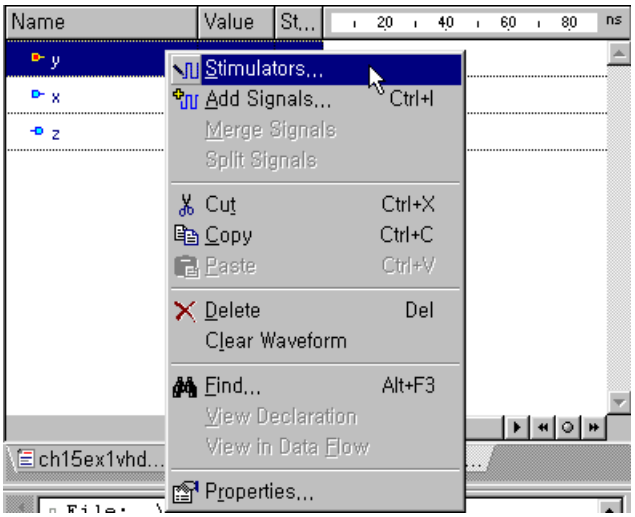


[그림 19-19] Active-HDL 사용 방법(12).

[그림 19-19]는 simulation 할 ports를 지정하는 방법을 보여주고 있다.
즉, ①번과 같이 **shift key**와 mouse를 사용하여 simulation할 ports를 지정하고, mouse를 ②번 영역으로 drag한 뒤에 drop해 주면, ②번과 같이 simulation할 port가 지정된다.

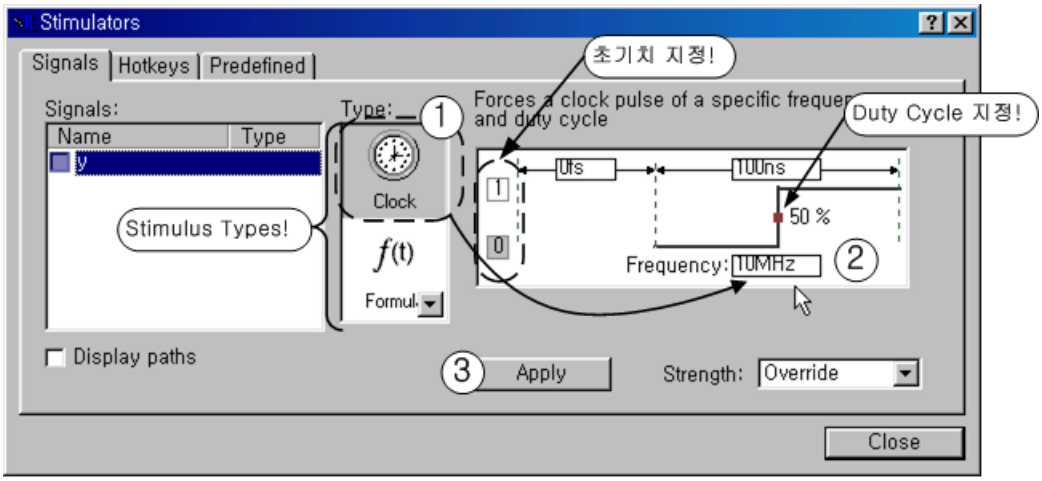
현재, simulation할 Ch15Ex1VHD.vhd file은 입력 port로 x, y를 그리고, 출력 port로 z를 가지고 있다. 일단, simulation을 하기 위해서는 입력 port에 test할 데이터를 입력해 주어야 한다. 일반적으로, HDL에서는 Test Data라고 하지 않고, **Test Vector**라는 용어를 사용한다.

즉, 입력 port y에 test vector를 입력하기 위해서는 ②번 영역에 있는 입력 port y, 즉, y 을 mouse로 선택한 뒤에, 오른쪽 mouse 버튼을 click 해 준다. 그러면, [그림 19-20]과 같은 contextmenu가 나타날 것이다.



[그림 19-20] Active-HDL 사용 방법(13).

[그림 19-20]과 같이 **Stimulators...** menu를 click해 주면, [그림 19-21]과 같은 **Stimulators** window가 나타난다.



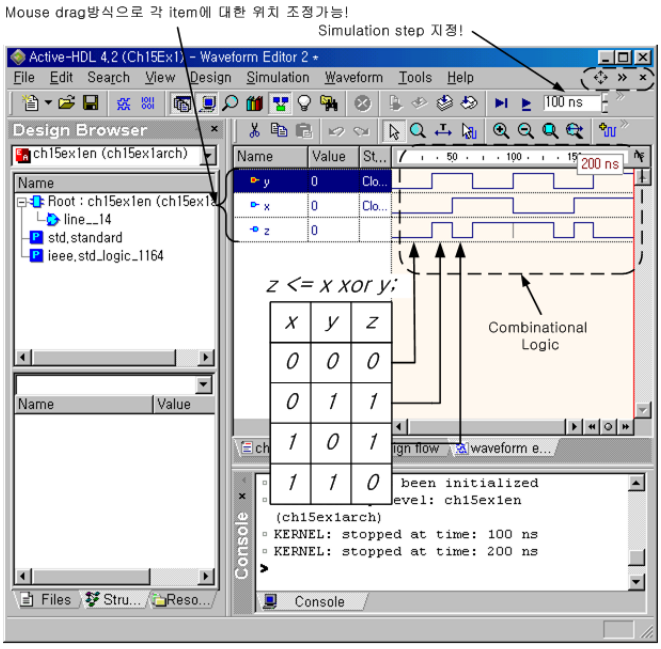
[그림 19-21] Active-HDL 사용 방법(14).

입력 port y에 주파수가 10[MHz]인 test vector를 할당하기 위해서는 우선, ①번과 같이 **clock icon**을 click 해 준다. 그러면, ②번과 같은 window가 만들어 질 것이다. 이제, mouse로 **10MHz**문자열을 click해 준 뒤에, **15MHz**로 편집해 준다. 사실, ②번에 있는 모든 문자열은 mouse로 선택하여 조정할 수 있는 것들인데 주의하자!
15MHz로 편집한 뒤에는 ③번의 **Apply** 버튼을 click해 준다. 그러면, 

과 같이 입력 port y옆에 있는 checkbox가 check 상태가 된다. 같은 방법으로, 입력 port x의 test vector로는 10[MHz]를 할당해 주자.

[그림 19-21]에는 **Clock**뿐만 아니라, 여러 가지 type의 **Stimulus**가 존재하므로, 각각의 기능을 확인해 보기 바란다.

이제, 입력 ports x, y에 test vector를 각각 지정하였으면, **Close** 버튼을 click 해 준다.
그리고 나서, Active-HDL의 main window에 있는 toolbar에서  icon을 click해 주면, [그림 19-22]와 같이 **z<= x xor y**에 대한 simulation이 나타나게 된다.
단, [그림 19-22]의 simulation 결과는  icon을 click 해 준 결과이다.



[그림 19-22] Active-HDL 사용 방법(15).

그런데, 주의할 것은 [그림 19-22]에서 보여주는 simulation 결과는 clock과는 무관한 하므로, **Combinational logic(조합논리)**에 해당한다.
또한, [그림 19-22]에서 보여준 Simulation은 단순히 HDL Code에 대한 simulation이고, 사용할 device의 특성과는 무관하므로 **functional simulation**이라고 부른다.

이제, [그림 19-22]의 결과를 Active-HDL wave file인 Ch15Ex1.awf file로 저장하도록 하자.

지금까지 여러분과 VHDL에 대한 기본적인 내용을 간략히 살펴보았다. 이제, **testbench**의 개념, simulation에서 사용하는 **Library**의 개념, Modelsim을 이용한 Timing Simulation, 그리고 사용할 device에 개발한 VHDL Code를 구현(implementation)하는 방법에 대해서 살펴볼 것이다.

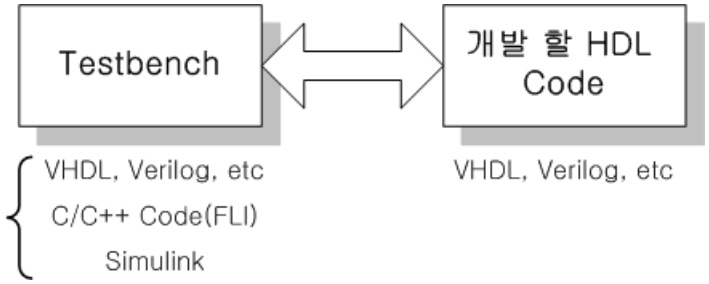
사실, 필자가 VHDL에 대해서 강의를 할 때, 대부분의 수강생들은 너무 많은 tool들을 사용하므로, 혼란스럽고, 어렵다고 하는 것이 사실이다. 그리고, 단순히, UltraEditor와 vendor에서 제공하는 tool들만으로도 충분하다고 얘기하는 분들도 있다.
그러나, 앞으로 보다 고성능의 FPGA가 나타나게 될 것이고, 또한, 보다 복잡한 알고리즘을 구현해야 되는 경우가 많을 것이다. 빠른 시간에 이 모든 요구 조건을 충족하면서 자신의 기술을 인정받기 위해서는 연구·개발 시간을 단축시켜줄 수 있는 최신의 tool들을 사용하는 습관을 갖도록 노력해야 한다고 생각한다.



이번에는 앞에서 개발한 Ch15Ex1VHD.vhd file을 modelsim에서 timing simulation하는 방법에 대해서 알아 볼 것이다. 그러기 위해서는 우선, testbench file을 만들어 주어야 한다.

여러분이 만일, 어떠한 알고리즘을 수행하는 Chip을 구현하였다면, 그 chip을 board에 땜질하여 붙일 것이다. 그러나, 어떠한 Chip도 단독으로 동작하는 경우는 없고, 일정한 interface를 통하여 다른 Chip들과 데이터를 주고받게 될 것이다. 이때, 개발한 Chip과 일정한 interface를 유지하면서 데이터를 주고받도록 새로운 Chip에 대한 HDL Code가 필요하게 된다. 그러나, 이 HDL Code는 Chip에 구현하는 것이 목적이 아니고, 단지, 개발할 HDL Code를 test해 주는 것이 목적이다. 이처럼, 개발할 HDL Code의 test를 목적으로 만든 HDL Code를 **testbench file**이라고 부른다.

보통, testbench file은 HDL로 구현하는 것이 일반적이지만, modelsim **SE/EE** version에서는 C/C++ Code로도 testbench file을 만들 수 있도록 지원해 주는 데, 이와 같은 기능을 **FLI**(Foreign Language Interface)라고 한다.
그리고, 뒤에서 배울 Matlab의 Simulink로도 testbench file을 만들 수 있다.



[그림 19-23] Testbench의 개념.

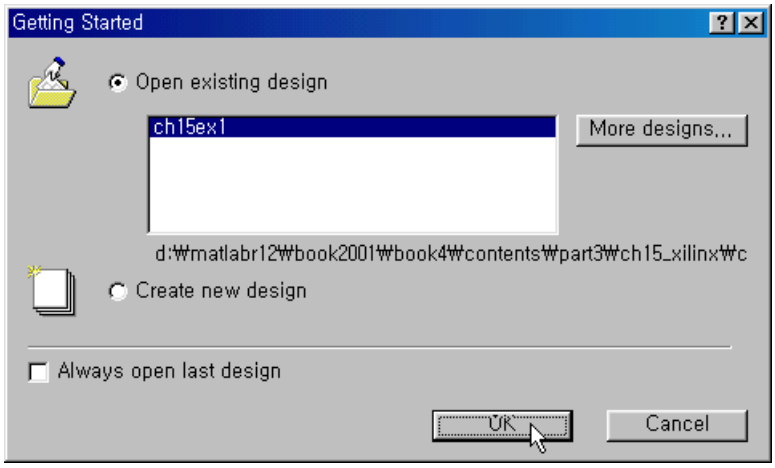
중요한 것은 testbench file은 Chip으로 구현하는 것이 목적이 아니므로 합성을 하지 않는다는 것이다.

VHDL에도 C/C++, matlab, JAVA와 같이 조건문인 **if-else**문이 있다, 그런데, 여러분이 구현할 VHDL Code가 너무 복잡하여서 수천 line이 되었다고 하자. 그러면, 이 VHDL Code에는 헤아릴 수 없이 많은 조건문이 사용 될 것이다. 이때, 모든 **if-else**문에 대한 조건을 점검할 수 있도록 HDL Code를 작성한다는 것은 거의 불가능한 일이다. 그래서, modelsim **SE/EE** version과 Active-HDL에서는 **CodeCoverage**라는 기능을 제공한다. 이 기능은 여러분이 만든 testbench file이 개발한 HDL Code에 얼마나 적용되고 있는지, 각각의 line마다 점검을 해 준다.
사실, 이 기능은 HDL Code뿐만아니라, C/C++, matlab과 같은 언어에서도 상당히 의미가 있는 기능이다.

그러나, 이 책에서는 testbench file의 작성 방법에 대해서만 살펴보고, codecoverage, FLI, performance analyzer 등등과 같은 내용은 다루지 않을 것이다. 이들 내용, 특히나, FLI는 상당한 지면을 요구하므로, 차후에 필자가 출간할 VHDL 서적에서 다룰 계획이다.

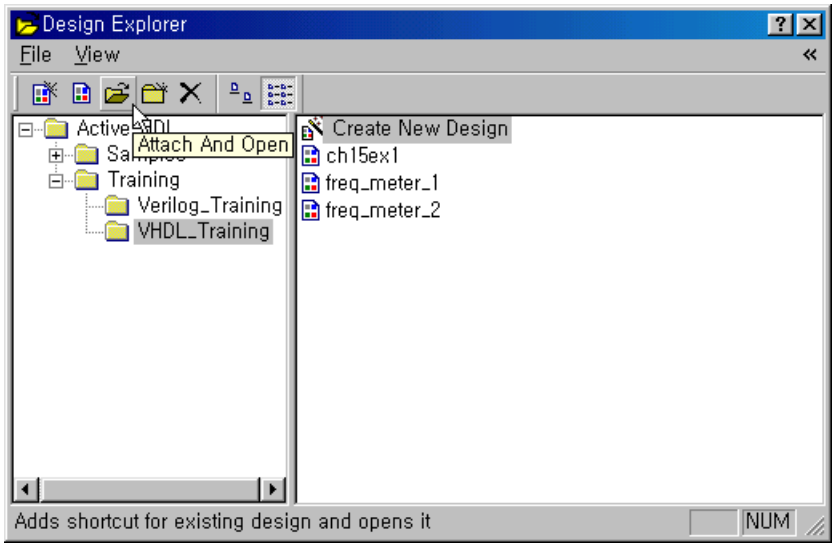
어쨌든, Ch15Ex1VHD.vhd file에 대한 testbench file을 만들기 위해서는 [그림 19-22]에서 저장한 Ch15Ex1.awf file이 필요하다. 왜냐하면, Ch15Ex1.awf file이 표현해 주는 파형(waveform)을 Active-HDL이 자동으로 해당 VHDL Code로 만들어 줄 것이기 때문이다.

만일, 새롭게 Active-HDL을 실행한 뒤에, 이전에 개발한 환경을 그대로 부르기 위해서는 [그림 19-24]와 같이 ch15ex1을 선택한 뒤에 **OK**버튼을 click해 주면 된다.



[그림 19-24] 이전에 개발한 design을 부르는 방법_1.

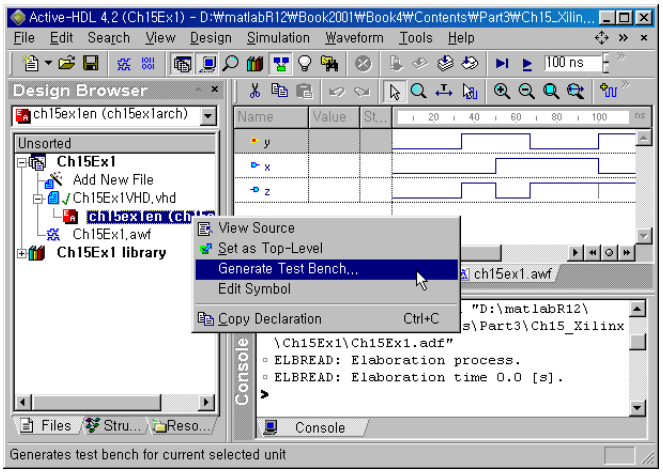
Active-HDL의 main window에서 이전에 개발한 ch15ex1 design을 부르기 위해서는 **File** menu에서 **Open Design...** menu를 선택해 준다. 그러면, [그림 19-25]와 같이 **Design Explorer** window가 나타날 것이다.



[그림 19-25] 이전에 개발한 design을 부르는 방법_2.

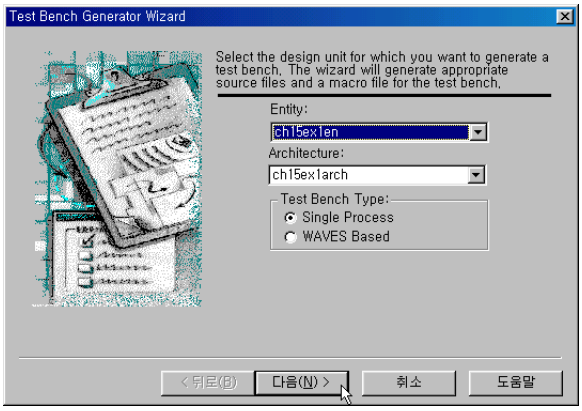
이제, [그림 19-25]와 같이 **Attach And Open** icon을 click해 준 뒤에, ch15ex1 design이 있는 디렉토리에서 **Ch15Ex1.adf** file을 선택해 주면 된다.

그리고, [그림 19-26]과 같이 **ch15ex1en(ch15ex1arch)** item을 mouse로 선택한 뒤에, mouse의 오른쪽 버튼을 click 해 준다. 그리고 나서, **Generate TestBench...** menu를 선택해 준다.



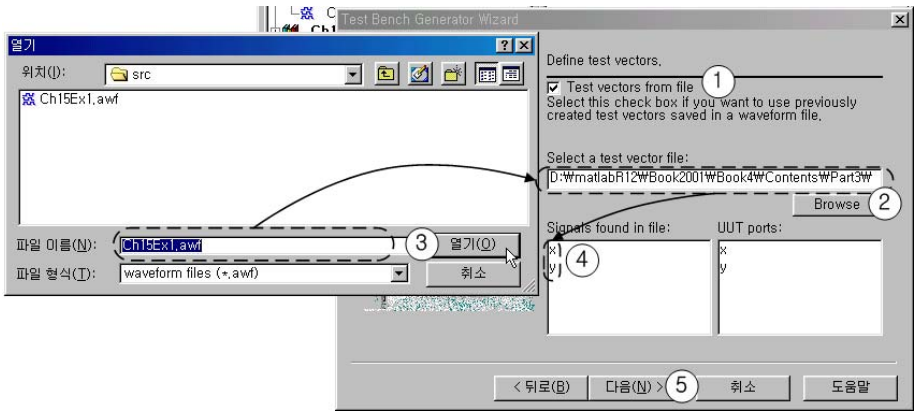
[그림 19-26] testbench file 생성 방법(1).

그러면, [그림 19-27]과 같이 **TestBench Generator Wizard** window가 나타날 것이다.



[그림 19-27] testbench file 생성 방법(2).

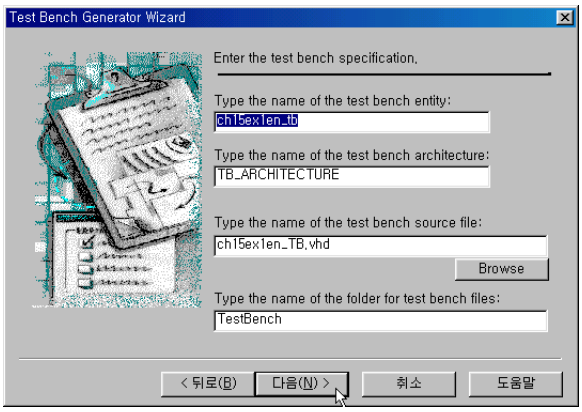
다음 버튼을 click 해 주면, [그림 19-28]과 같이 두 번째 **TestBench Generator Wizard** window가 나타날 것이다.



[그림 19-28] testbench file 생성 방법(3).

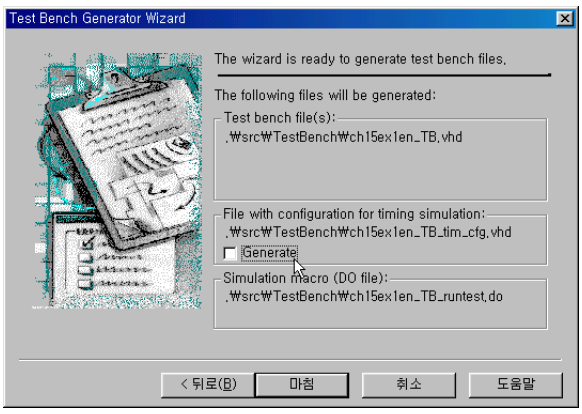
우선, [그림 19-28]에서 보여준 것과 같이, ①번 **Test vectors from file**옆에 있는 checkbox를 check해 주고, ②번 **Browse** 버튼을 click 해 준 뒤에, Ch15Ex1.awf file을 선택하고 **열기** 버튼을 click 해 준다. 그러면, ④번과 같이 입력 port x, y가 나타날 것이다. 이제, ⑤번과 같이 **다음** 버튼을 click 해 준다.

세 번째 **TestBench Generator Wizard** window는 생성될 testbench file에 대한 구성 명세이다. 특별히, 수정할 사항이 없으므로 **다음** 버튼을 click해 준다.



[그림 19-29] testbench file 생성 방법(4).

[그림 19-30]에서 **Generator** 옆에 있는 checkbox를 check해 주면, Active-HDL로 timing simulation하는 데 필요한 configuration file을 자동으로 생성해 준다.
그러나, 우리는 modelsim에서 timing simulation할 것이므로 check하지 않고, **마침** 버튼을 click 해 준다.



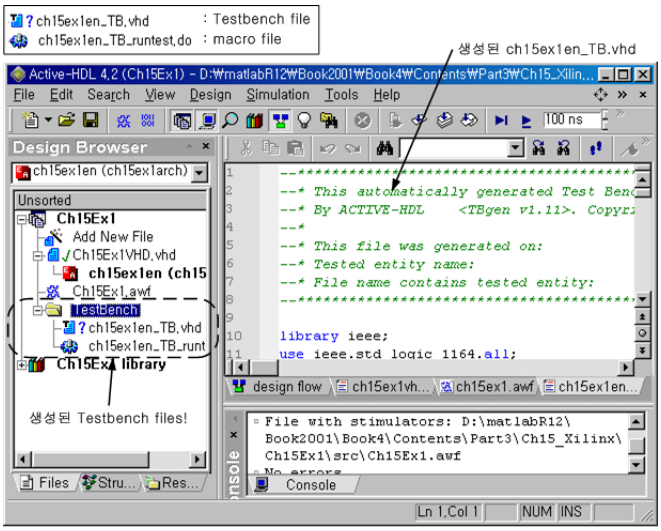
[그림 19-30] testbench file 생성 방법(5).

그러면, [그림 19-31]과 같이 testbench file인 ch15ex1en_TB.vhd file과 macro file인 ch15ex1en_TB_runtest.do file이 만들어진다.

macro file이란, 개발 중인 **Ch15Ex1** design에 대한 library mapping과 compilation, 그리고

simulation 환경을 한번에 수행해 주는 script file이다.

그리고, HDL file icon 옆에 있는 √은 compilation이 성공적으로 이루어졌다는 뜻이고, X은 compilation error가 발생한 것을 의미한다. 또한, ?은 아직 compilation을 하지 않은 경우를 의미한다.



[그림 19-31] testbench file 생성 방법(6).

이제, 자동으로 만들어진 testbench file에 대한 compilation(즉, icon을 click)을 수행해 준다. 그리고 나서, macro file인 ch15ex1en_TB_runtest.do을 mouse로 선택한 뒤에, mouse의 오른쪽 버튼을 누르면, contextmenu가 나타난다. 이때, **Execute** menu를 click 해 주면, [그림 19-22]와 같은 simulation 결과를 얻을 수 있다.

이제, Active-HDL에서 만들어준 testbench file인 ch15ex1en_TB.vhd file을 modelsim에서 timing simulation하기 위해서 사용해 보도록 하자.

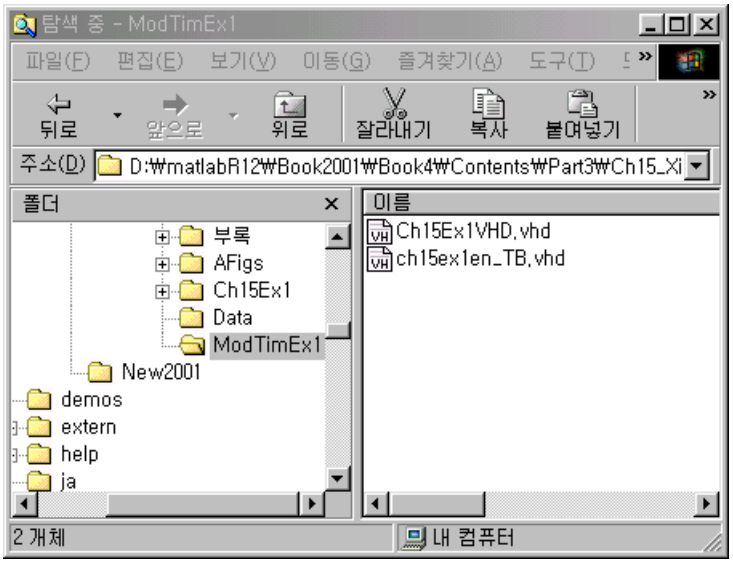
사실, testbench file인 ch15ex1en_TB.vhd file을 설명하기 위해서는 configuration문을 설명해야 하는데, 이 configuration을 설명하기 위해서는 component, port mapping 등등 많은 내용을 설명해 주어야 한다. 그러므로, 여기서는 단순히, configuration이 단어 뜻 그대로 계층구조를 갖는 HDL Code 상에서 entity와 architecture의 연관성을 구성해 주는 선언문이라고만 알아두자.



이제 설명할, modelsim은 사실, FPGA, CPLD뿐만 아니라, ASIC을 개발하는 경우에도 사용되는 세계 최고의 성능과 시장을 갖고 있는 VHDL Code simulator이다. 그 정밀도와 program의 안정성은 대단하다고 생각한다. 물론, verilog code도 simulation을 할 수 있지만,

대부분의 verilog 개발자들은 modelsim을 잘 사용하지 않는 것 같다.
어쨌든, modelsim은 2가지 버전으로 판매가 이루어지고 있는데, 모든 기능을 발휘하는 것은 **SE/EE** version이다. 그런데, modelsim SE/EE version의 값이 수천만원을 하므로 솔직히, 쉽게 접할 수 없는 tool이기도 하다. 물론, 각 vendor가 제공하는 OEM 방식의 modelsim도 있고, modelsim 홈페이지를 통하여 download 받아서 30일간 사용할 수도 있다.

modelsim을 사용하여 Ch15Ex1VHD.vhd file에 대한 timing simulation을 하기 위해서는 Active-HDL이 만들어준 testbench file인 ch15ex1en_TB.vhd file이 있어야 한다.
이제, Ch15Ex1VHD.vhd file과 ch15ex1en_TB.vhd file을 복사(copy)하여 임의의 디렉토리에 paste해 주도록 하자.
예를 들면, [그림 19-32]와 같이 ModTimEx1 directory에 paste해 주자.



[그림 19-32] Modelsim을 위한 directory.

그리고, modelsim을 사용하여 timing simulation을 하기 위해서는 Ch15Ex1VHD.vhd file을 합성(synthesis)하고, 구현할 device의 vendor에서 제공하는 **P&R** tool을 사용하여 **SDF**(standard delay format) file과 timing simulation 용 VHDL file을 만들어 주어야 한다.

즉, Ch15Ex1VHD.vhd file이 실제 device에 구현되기 위해서는 수많은 delay를 고려해 주어야 한다. 즉, device 내부의 각 block간 line에서 발생하는 delay(routing delay), 그리고, 입·출력 단자(pad)에서 발생하는 delay, 또한, block 내부에서 발생하는 delay 등등 수많은 delay가 고려되어야 되는데, 이중에서, 사용하는 device에 관계없이 표준이 되는 delay의 정보를 갖고 있는 file이 SDF file이고, 또한, 각 vendor에서 제공하는 특정 device가 갖고 있는 delay가 있는데, xilinx의 경우에는 **simprim**이라는 Library에 모든 Xilinx device들에 대한 delay 정보들이 있다.

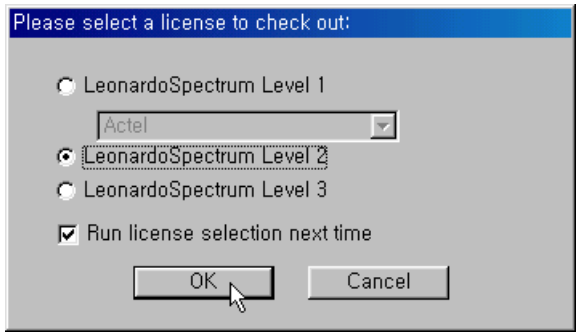
그리고, timing simulation 용 VHDL file이란, Ch15Ex1VHD.vhd file에, 앞에서 설명한 각종 delay를 첨가시켜준 VHDL file이다. 이 file을 열어보면, 대부분 **generic**이라는 VHDL 지정어를 사용하는데, 이 지정어는 해당 변수에 대한 값을 외부에서 할당할 수 있도록 해 줄 때

사용한다. 즉, 실질적인 delay 수치 정보는 SDF file과 Xilinx의 경우, simprim에 있고, timing simulation 용 VHDL file에서는 그 수치 정보를 받아들일 수 있는 **generic**으로 선언된 변수(variable)가 있는 형태이다.

자! 이제, 무슨 뜻인지 실제로 합성을 해보면서 차근차근 살펴보도록 하겠다.
합성기 tool로는 Exemplar회사의 **LeonardoSpectrum**을 사용할 것이다. 과거에 **galileo**로 알려졌던, 이 합성기 tool은 현재, 미국의 거대 EDA(Electronic Design Automation) tool 업체인 mentor가 소유권을 갖고 있다. 물론, Visual HDL tool인 Renoir와 simulator tool인 modelsim도 모두 mentor가 소유하고 있으며, 이들 3개의 tool을 하나의 package로 묶어서 **FPGA Advantage**라는 이름으로 판매하고 있다.

어쨌든, 합성기 tool로는 **LeonardoSpectrum**과 **synplify**, 그리고 **FPGA-Express**가 현재 널리 사용되고 있다.

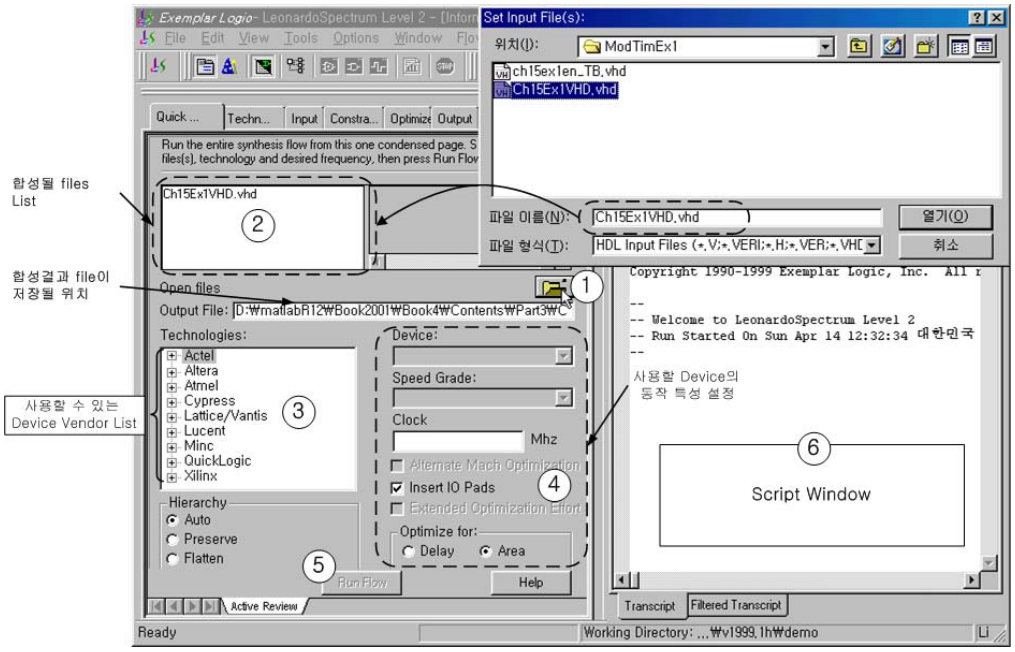
LeonardoSpectrum을 실행하면, [그림 19-33]과 같이 level을 선택할 수 있는 window가 나타난다.



[그림 19-33] level 선택 window.

① **Level 1**: 각각의 device vendor에서 제공하는 OEM version 기능만 수행할 수 있다. 즉, Xilinx LeonardoSpectrum을 가지고 Altera device에 대한 합성은 할 수 없다는 뜻이다. 결국, 상당한 기능적 제약을 갖는다.
㉡ **Level 2** : FPGA, CPLD device의 vendor에 상관없이 합성을 해 주며, 그밖에 선택한 device의 합성결과를 수정할 수 있는 mode와 관찰할 수 있는 mode 및, **critical path delay time**에 대한 모든 정보를 보여둔다. critical path delay time이란, 개발한 HDL Code가 선택한 device에 합성되었을 때, 가장 많은 delay time을 요구하는 path에 대한 시간 정보를 말하며, 이것이 바로 device의 동작 속도를 결정한다고 보아도 된다.
㉢ **Level 3 : ASIC**에 대한 합성 기능을 포함한 경우를 말한다. 일반적으로, FPGA로 원하는 속도와 동작이 나오고, 그 상업성이 보장된다면, 각 반도체 foundry를 갖고 있는 회사의 design house에 개발한 HDL Code를 맡긴다. 그러면, 그곳에서 개발한 HDL Code를 각 반도체 foundry에서 ASIC으로 만들 수 있도록 HDL Code를 수정해준다. 보통, ASIC으로 만들려면, 상당히, 많은 돈이 필요하고, 또한, 돈이 있어도, Memory 산업이 호황이라서 삼성, 현대, 아남 등등의 회사가 바쁘다면, 1년 이상 기다려야 한다고 한다. 그리고, ASIC을 개발하려는 경우, 대부분 LeonardoSpectrum보다는 Synopsys의 design Compiler를 주로 사용한다고 한다.

어쨌든, 각 Level에 따라서 LeonardoSpectrum의 값이 다르므로, 현재, 여러분이 사용하는 license에 맡게 level을 선택해 주어야 한다.
이제, **OK**버튼을 click해 주면, [그림 19-34]와 같이 LeonardoSpectrum의 main window가 나타난다.



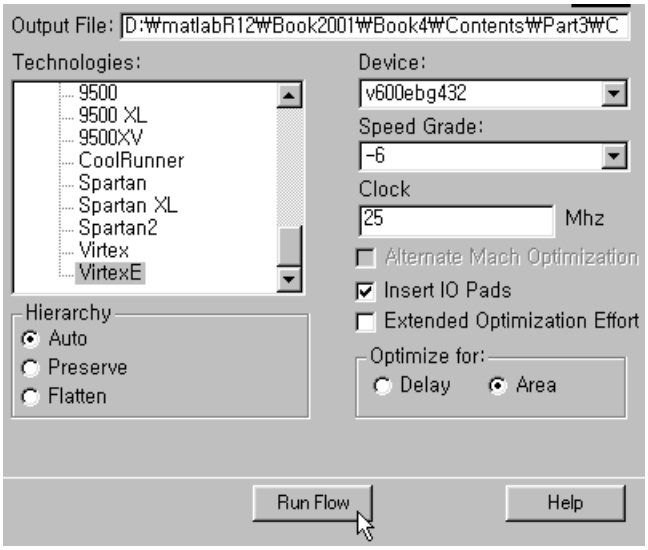
[그림 19-34] LeonardoSpectrum의 main window.

LeonardoSpectrum의 main window가 나타나면, ①번 icon을 click하여 합성할 file, 여기서는 Ch15Ex1VHD.vhd file을 선택하여 준다. 그러면, ②번과 같이 선택한 files list가 나타난다. 여기서, 주의 할 것은 testbench file인 ch15ex1en_TB.vhd file은 합성 대상이 아니라는 것이다. 그리고, 한가지 주의할 것이 있다. 즉, 현재는 합성될 file이 Ch15Ex1VHD.vhd file 하나이므로 상관없지만, 여러 개의 VHDL Codes로 구성된 design의 경우에는 반드시, top-level의 VHDL Code가 맨 아래로 mouse를 사용하여 drag해 주어야 한다.

어쨌든, 합성될 files list를 올바로 선택하였으면, ③번에 있는 device list에서 사용할 device를 선택한다. 그러면, ④번에 비활성화 되어 있던 **device** :와 **Speed Grade** :가 활성화된다.

이제, [그림 19-35]와 같이 원하는 device를 지정해 주고, Speed Grade를 설정해 준 뒤에, 사용할 device의 동작 속도를 설정해 주면 된다.

일반적으로 Xilinx device의 경우에는 Speed grade가 6, 7, 8이고, Altera의 경우에는 3, 2, 1의 수치로 매겨지는데, Xilinx의 경우에는 수치가 높을수록 빠른 device이고, Altera의 경우에는 수치가 낮을수록 빠른 device이다. 물론, device의 속도가 빠를수록 고가일 수밖에 없다. 또한, Altera의 경우, 3x, 2x, 1x와 같이 뒤에 **x**가 붙으면, 해당 device의 내부에 PLL 기능이 있다는 뜻이고, **x**가 없으면 PLL 기능이 없다는 뜻이다. 이에 반해서, Xilinx는 PLL 대신에 DLL을 사용하며 기능은 주파수 채배라는 측면에서 동일하다.
주파수를 채배한다는 것은 입력 주파수보다 높은 주파수를 device 내부에서 발생할 수 있다는 뜻이다.



[그림 19-35] 사용할 device 사양(Spec.) 지정.

어쨌든, 현재, [그림 19-35]로 보아서, 필자는 Xilinx의 VirtexE Series 중에서 v600ebg432 device를 선택하였고, 동작 속도는 25[MHz]를 할당하였다. 사실, 우리가 합성할 Ch15Ex1VHD.vhd file은 clock과 관계가 없는 조합회로이므로 동작 속도 25[MHz]는 아무런 의미가 없지만, 지정은 해 주어야 합성을 할 수 있다. 그리고, 지정한 v600ebg432 device에서 끝에 있는 432는 device의 pin 개수를 의미하며, **bg**라는 접미어는 선택한 device가 BGA type이라는 뜻이다. 그러나, 어짜피, BGA도 사람이 땜질 할 수 없는 device package이므로, 가격이 더 낮은 Fine BGA를 구입하여 사용하는 것이 더 좋을 것이다.

그리고, [그림 19-35]에서 **Optimize for** :를 Area로 설정하였으므로, LeonardoSpectrum은 사용할 device의 내부 gate를 최대한 적게 소모하도록 합성을 할 것이다.
또한, **Extended Optimization Effort**를 check해 주면, LeonardoSpectrum 내부에 있는 여러 가지 최적의 합성관련 알고리즘을 수행하여 그 중에서 제일 좋은 합성결과를 사용하도록 해준다. 그러나, 여러 가지 합성관련 알고리즘을 수행하므로, 합성 시간이 상당히 오래 걸리는 데 주의하자.

이제, **Run Flow** 버튼을 click해 주면, Ch15Ex1VHD.vhd file의 합성 결과인 Ch15Ex1VHD.**edf**화일이 [그림 19-35]에서 지정한 **Output File** :옆에 있는 경로에 저장된다. 올바로 저장되어 있는지 file 탐색기를 사용하여 확인해 보기 바란다.

[그림 19-34]에서 ⑥번에 보여준 script window는 합성에 대한 여러 가지 결과를 보여주고, 또한, LeonardoSpectrum의 options를 지정할 수 있는 window이다.
시간을 좀더 내서 script window가 보여주는 합성결과 list를 꼼꼼히 살펴보기 바란다.

각 device vendor에서 제공하는 OEM LeonardoSpectrum은 지원하지 않지만, 혹시, 자신이 사용하는 LeonardoSpectrum이  기능을 지원한다면, 각각의 icon들을 click해 보기 바란다.  icon들은 각각 LeonardoSpectrum이 어떻게 주어진 HDL Code를 선택한 device의 내부에 합성을 하였는지 그 **내부 모습**과 **delay**에 대한 상세한 정보 및 여러 가지

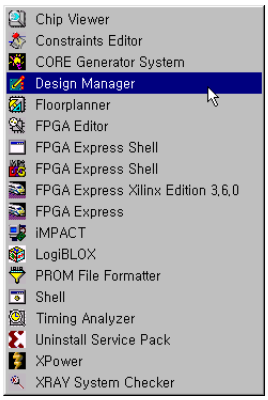
유용한 정보들을 제공해준다.

한가지 재미있는 것은  icon 각각 수백만원 한다는 것이다. 질은 기억나지 않지만, 3개의 icon들을 모두 사용하는 데, 1000만원이상을 요구한 것 같다.



어쨌든, Ch15Ex1VHD.**edf** 파일에 대해서 P&R을 수행하기 위해 Xilinx의 **design manager**를 불러들이도록 하자.

[그림 19-36]은 Xilinx Foundation Series 4.1i의 **Accessories**에 포함되어 있는 icon들이다.



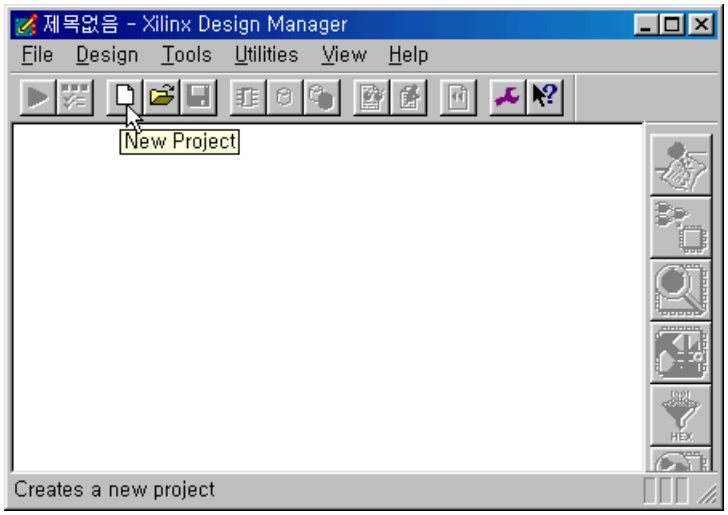
[그림 19-36] Xilinx Foundation Series 4.1i의
Accessories.

[그림 19-36]에서 특별히, 주의할 것은 Xilinx Foundation Series 3.X version까지만 해도, FPGA Express를 단독으로 호출하여 사용할 수 있었지만, 4.1i부터는 project manager(즉, 

)에서만 호출하여 사용할 수 있도록 바뀌었다,

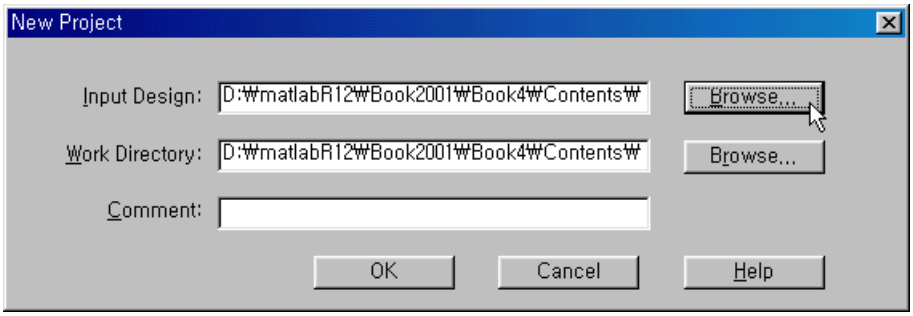
이제부터 설명할 Xilinx Tool에 대한 내용은 Xilinx ISE에서도 동일하게 사용할 수 있다.

이제, [그림 19-36]과 같이 Design manager를 click해 주면, [그림 19-37]과 같이 Xilinx Design manager가 나타난다.



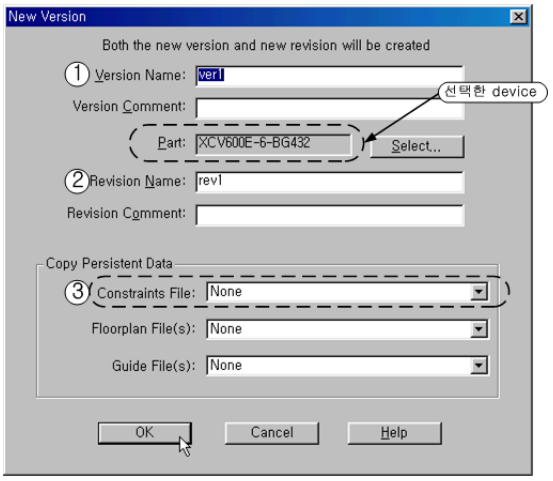
[그림 19-37] Xilinx Design manager Window.

[그림 19-37]과 같이 **New Project** icon을 click해 주면, [그림 19-38]과 같이 New Project window가 나타날 것이다. 이제, 그림과 같이 **Browser** 버튼을 click하여 LeonardoSpectrum으로부터 만든 Ch15Ex1VHD.edf 파일을 선택해 준다.



[그림 19-38] Ch15Ex1VHD.edf 파일 선택.

그리고, **OK** 버튼을 click 해 주면, [그림 19-39]와 같이 **New Version** Window가 나타날 것이다.

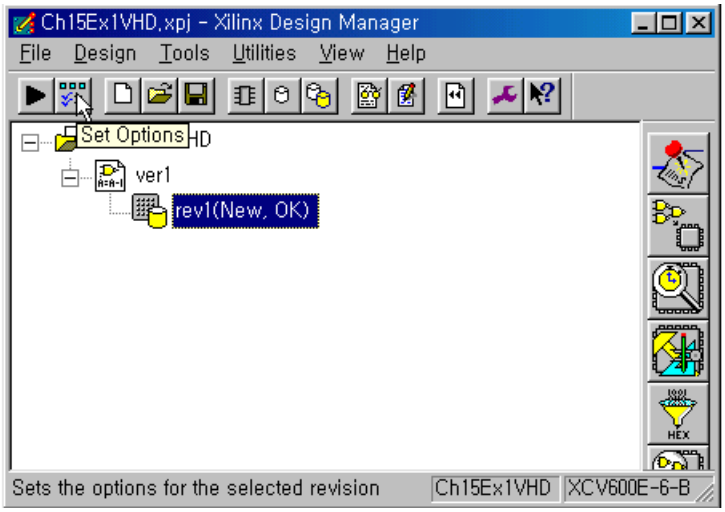


[그림 19-39] version 관리와
constraint 지정.

New Version Window에서 ①번과 ②번은 현재, design manager 상에서 사용하고 있는 version number를 의미하고, ③번의 Constraint File은 사용자가 임의로 선택한 device의 I/O pin 할당, clock 할당 등등을 할 때, 지정한 pin 할당 정보를 갖고 있는 ***.ucf file**의 위치를 지정해 주는 곳이다.

물론, 가장 좋은 pin 할당 방법은 design manager가 자동으로 할당해 주는 pin 번호를 사용하는 것이 좋지만, 대부분의 경우, Chip이 놓일 Board가 먼저 만들어지므로, Chip의 pin 할당을 사용자가 직접 할당하는 경우가 많다. 이때, 사용하는 program이 constraint editor로서 [그림 19-36]에서 실행할 수 있다. constraint editor는 pin 할당 기능 외에도 device 내부에서 발생하는 delay를 조정해 주는 기능도 있다.

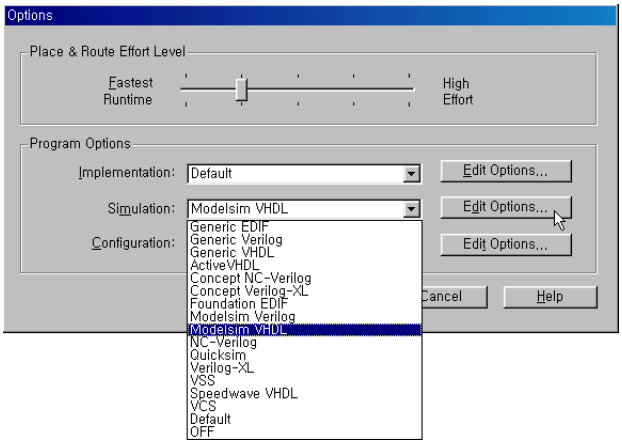
그러나, 여기서는 design manager가 자동으로 할당한 pin을 사용하도록 하자.
이제, [그림 19-39]에서 **OK**버튼을 click해 주면, [그림 19-37]은 [그림 19-40]과 같이 바뀌게 된다.



[그림 19-40] Design manager가 Ch15Ex1VHD.edf 파일을 받아들인 경우.

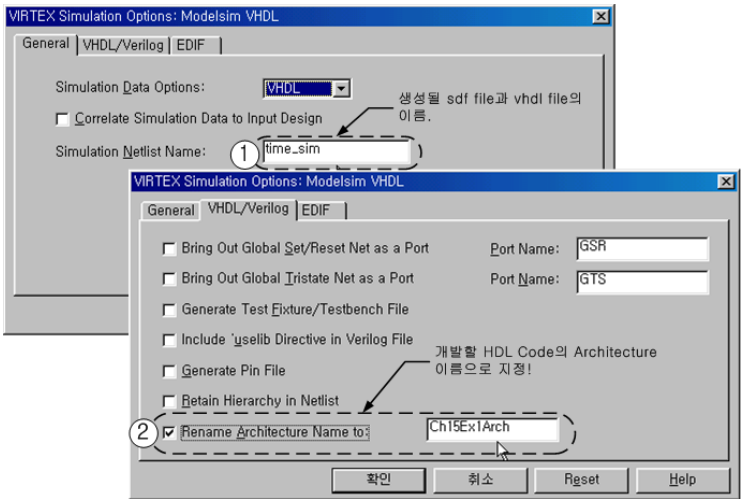
Design manager가 Ch15Ex1VHD.edf 파일을 반아들였으면, [그림 19-40]과 같이 **Set Options** icon을 click 해 준다.

[그림 19-40]과 같이 **Set Options** icon을 click 해 주면, [그림 19-41]과 같이 **Options** Window가 나타난다. 여기서, 그림과 같이 **Simulation** : 옆에 Modelsim VHDL item을 지정해 주고, **Edit Options...** 버튼을 click 해 준다.



[그림 19-41] Simulator option 선택 방법.

[그림 19-41]에서 **Edit Options...** 버튼을 click 해 주면, [그림 19-42]와 같은 Options Window가 나타날 것이다.

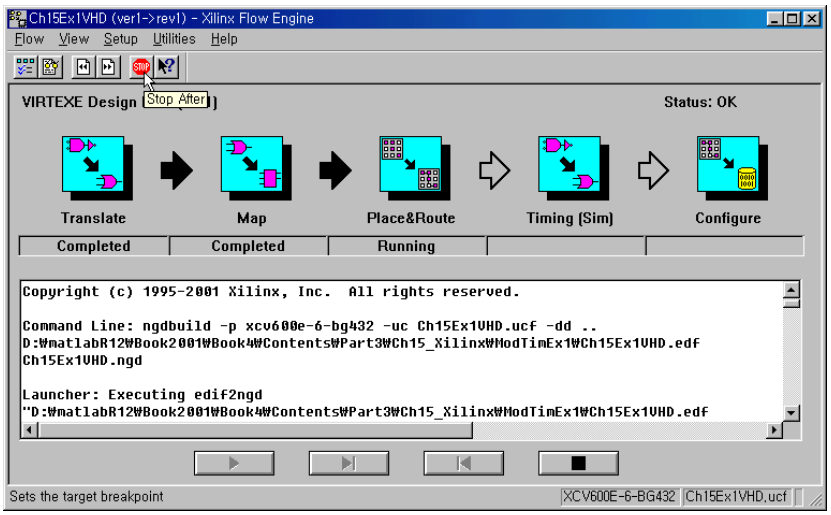


[그림 19-42] Options Window.

①번에서 지정한 **time_sim**은 P&R 이후에 만들어진 sdf file인 time_sim.sdf file과 timing simulation용 VHDL file인 time_sim.vhd file의 이름이다.
그리고, ②번은 기본적으로 **STRUCTURE**로 지정되어 있는데, 이것을 [그림 19-42]와 같이 개발할 HDL Code인 Ch15Ex1VHD.vhd file의 architecture 이름으로 바꾸어 주어야 한다.

이제, 올바로 options를 지정하였으면, **확인** 버튼을 click 해 준 뒤에, design manager의 toolbar에 있는 ▶ icon을 click 해 준다. 그러면, [그림 19-43]과 같이 translation 과정,

mapping 과정, P&R 과정, timing simulation을 위한 데이터 추출 과정, device에 실질적으로 구현할 bit stream 생성과정을 수행한다.



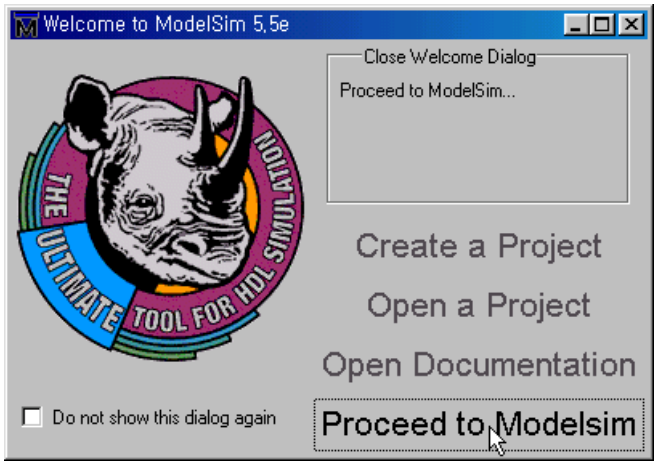
[그림 19-43] vendor tool의 수행과정.

만일, [그림 19-43]에서 보여준 vendor tool의 수행을 중간에 멈추려면, 그림에서 보여준 STOP icon을 click해 주면 된다.

어쨌든, vendor tool을 실행함에 의해서 수많은 file들이 생성되는 데, 이들 각각의 file들에 대한 내용과 vendor tool이 수행하는 각각의 과정에 대한 설명은 너무 많은 지면을 요구하므로 여기서는 생략하겠다.
특별히, timing simulation을 수행하는 데 필요한 time_sim.sdf file과 time_sim.vhd file에 주목하고, device에 적용하기 위한 bit stream file인 ch15ex1vhd.bit에 주목하자.

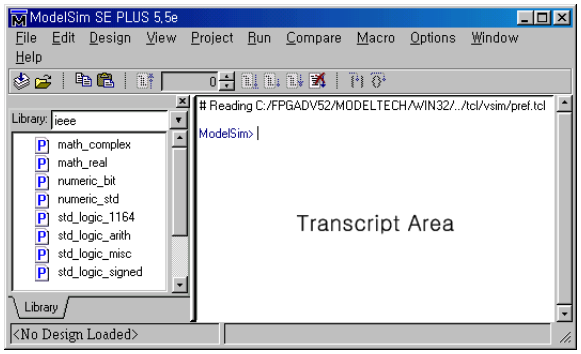


자! 이제, modelsim을 이용한 timing simulation을 수행할 모든 준비가 되었다.
일단, modelsim을 실행해보도록 하자.



[그림 19-44] welcome window.

[그림 19-44]는 modelsim의 welcome window 모습이다.
Proceed to Modelsim을 mouse로 click해 주면, [그림 19-45]와 같이 modelsim main window가 나타난다.
사실, modelsim은 version에 따라서 수시로 GUI 환경이 바뀌므로 주의해야 한다. 그러나, 실질적인 기능상의 차이는 그렇게 크지 않다고 본다.



[그림 19-45] modelsim main window.

modelsim의 GUI 환경을 가만히 살펴보면, 지금까지 우리가 접해온 Windows의 GUI 환경과 모습이 조금 다르다는 것을 알 수 있을 것이다.
사실, modelsim은 matlab과 같이 OS에 상관없이 동일한 GUI 환경과 명령어(command)들을 가진다. 이처럼, OS에 상관없이 어떠한 application의 실행만을 담당하는 언어로서 **Tcl**(Tool Command Language)라는 언어가 있다. 바로, modelsim의 모든 GUI 환경 및 command의 수행은 Tcl에 의해서 이루어지며, 확장자로는 ***.tcl**을 사용한다.
보다 구체적으로 말하면, Tcl은 script 언어와 이를 해석하여 실행시켜주는 interpreter로 구성되며, **Tk**(Tool Kit)는 Tcl을 이용하여 GUI를 손쉽게 구성할 수 있는 Tool Kit을 제공한다.

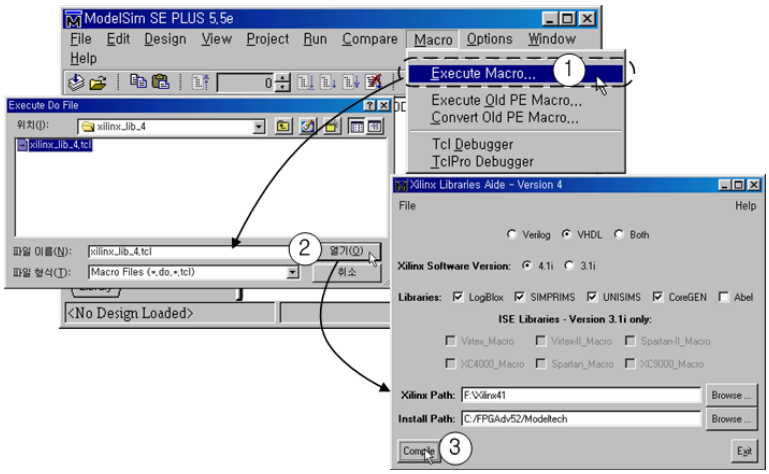
Modelsim에는 ***.do** file이라는 macro file이 사용되는 데, 여기서는 간단히, Tcl/Tk를 보다 정교한 기능을 수행할 수 있는 진보된 macro file로 알아두자.

Modelsim을 사용하여 Xilinx Device에 대한 Timing Simulation을 하기 위해서는 앞에서도 언급하였듯이, 모든 Xilinx Device에 대한 delay 정보를 갖고 있는 **simprim**이라는 library가 미리 compile되어 있어야 한다.

사실, modelsim뿐만 아니라, Active-HDL에서 언급하는 **library**라는 단어는 일종의 directory를 의미한다. 단, file 탐색기에서 만든 directory가 아니라, modelsim의 **vlib** 명령어를 사용하여 만든 directory를 의미하며, 특별히, **logical library**라고 부른다.
그리고, modelsim으로 compilation된 결과 file들이 위치할 directory를 **work library**라고 부르는 데, modelsim의 **vmap** command를 사용하여 vlib로 만든 logical library를 work library에 mapping을 해주어야 compilation된 결과 파일들이 생성한 logical library에 저장된다.
만일, 새로 생성할 logical library 이름을 **work**로 지정한다면, vmap command로 mapping을 해주지 않아도 modelsim이 자동으로 compilation된 결과 file들을 work library에 저장한다.

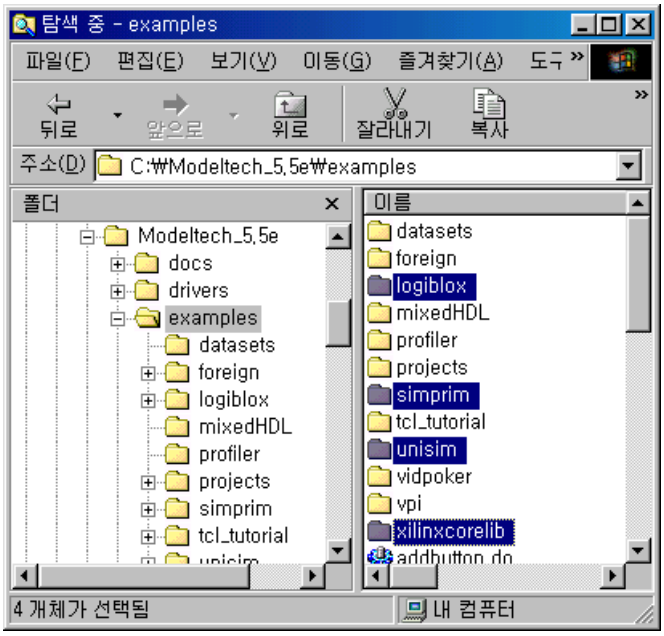
우선, Xilinx device의 모든 delay 정보를 갖고 있는 simprim library를 만들어 보자.
그러나, 초보자가 일일이 vlib, vmap command를 사용하여 새로운 library를 만들어서 modelsim에 등록한다는 것은 쉽지 않으므로, 첨부한 CD에서 제공하는 **xilinx_lib_4.tcl** file을 사용하도록 하자.

xilinx_lib_4.tcl file을 실행하기 위해서는 [그림 19-46]과 같이 modelsim의 main window에서 ①번이 가리키는 **Execute Macro...** menu를 click 해 준다. 그리고 나서, 나타나는 **Execute Do File** window에서 xilinx_lib_4.tcl file을 선택해 주면, ③번과 같은 window가 나타날 것이다.



[그림 19-46] Xilinx Libraries compilation 방법.

이제, [그림 19-46]에서 보여준 것과 같이 **Xilinx Libraries Aide** window를 setting 해 주고, **compile** 버튼 click 해 주면, 선택한 **LogiBlox, simprims, unisims, CoreGen** libraries를 compilation할 것이다.
물론, System에 따라서 다르기는 하지만, 대략 20~30분 가량 소요된다. compiling이 끝나면, modelsim이 install되어 있는 directory에 [그림 19-47]과 같이 각각의 library가 새롭게 만들어진다.



[그림 19-47] Xilinx Libraries.

주의

[그림 19-47]과 같이 4개의 libraries를 새롭게 만들기 위해서는 대략 85[Mbyte]의 공간이 필요하다. 또한, modelsim은 좀 귀찮은 사항이 있는데, 바로, 5.4 version에서 5.5 version으로 소수점 다음 수치가 바뀌면 모든 file을 다시 재 compilation 해야 한다는 것이다.

사실, 이것은 무척 귀찮은 일이지만, compiling 속도를 줄이기 위한 기술적인 upgrade이므로 어쩔 수 없는 사항이다. 그러나, 5.5a version에서 5.5e version 간에는 재 compiling 할 필요는 없다.

실질적으로 simprim library가 각각의 xilinx device에 대한 delay 정보를 갖고 있으므로 timing simulation만 한다면, simprim library만 있어도 된다.

그러나, 입력 주파수를 분주하거나, 채배하려는 경우, Xilinx device 내부에 있는 DLL을 사용해야 한다. 여기서 주파수를 분주 한다는 것은 입력 주파수를 $\times\frac{1}{2}$, $\times\frac{1}{3}$, $\times\frac{1}{4}$, ...과 같이 작게하는 것을 의미하고, 채배한다는 것은 $\times 2$, $\times 3$, $\times 4$, ...과 같이 높여주는 것을 말한다.

대부분, 분주 기능은 쉽게 VHDL Code로 작성할 수 있지만, 채배기능은 구현할 수가 없다. Xilinx device series 중에서 Virtex에는 이와 같은 DLL이 여러 개 있고, 보다 upgrade된 virtex_II에는 DLL 기능과 몇 가지 기능을 더 첨가한 DCM이 여러 개 있는데, 이와 같이 Xilinx device에서 특별히, 지원하는 기능 block들을 구현하고자 할 때, 사용하는 library가 바로 **unisim** library이다. 그러나, ASIC을 목표로 VHDL Code를 개발한다면, 이와 같은 Xilinx 의존적인 library를 사용하면 안되고, 외부에서 clock을 공급받는 형태로 만들어야 할 것이다.

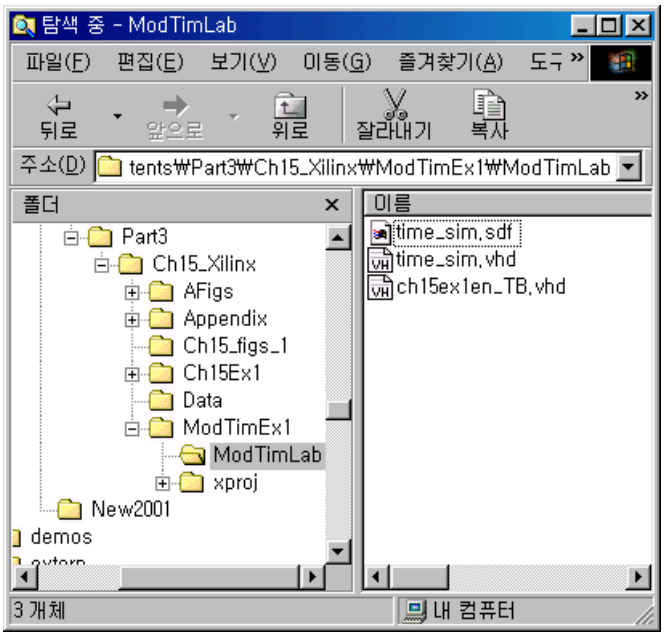
uisim뿐만 아니라, Xilinx 내부에 RAM, DPRAM(Dual port RAM), ROM, FIFO 등등과 같

은 구현하기 힘든 VHDL Code를 제공해주는 library가 **Xilinxcorelib** library이고, logiblox library도 비슷한 기능을 제공한다. 어쨌든, 이들은 모두 Xilinx device에서만 사용할 수 있는 resource library들이다. 만일, Altera device를 사용한다면, 대응하는 library를 vendor engineer에게 문의해야 할 것이다.

자! 이제, Xilinx device에서 timing simulation을 하기 위해서 필요한 모든 files가 준비되었다. 다음은 우리가 준비한 file들의 list이다.

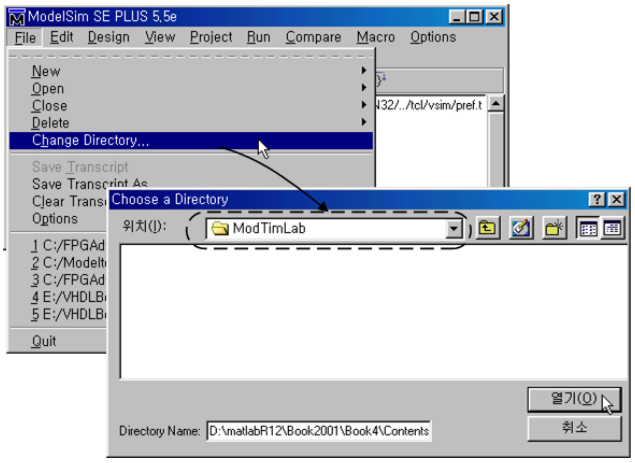
① [그림 19-43]으로부터 얻은 time_sim.vhd, time_sim.sdf file
② simprim library
③ testbench file인 ch15ex1en_TB.vhd file이 필요하다.

우선, [그림 19-48]과 같이 modelsim으로 timing simulation을 수행할 때 발생하는 file들을 저장할 directory를 새롭게 만든다. 그리고, 그곳에 필요한 file들을 위치시킨다.



[그림 19-48] timing simulation용 directory.

그리고, [그림 19-49]와 같이 새롭게 만든 directory로 modelsim의 경로를 바꾸어 준다.

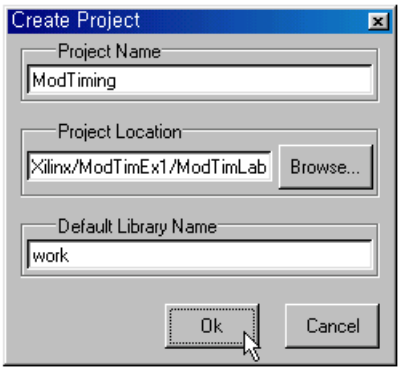


[그림 19-49] Modelsim의 경로 변경 방법.

그리고, modelsim의 현재 위치를 알고 싶으면, 다음과 같이 **pwd** command를 사용하면 된다.

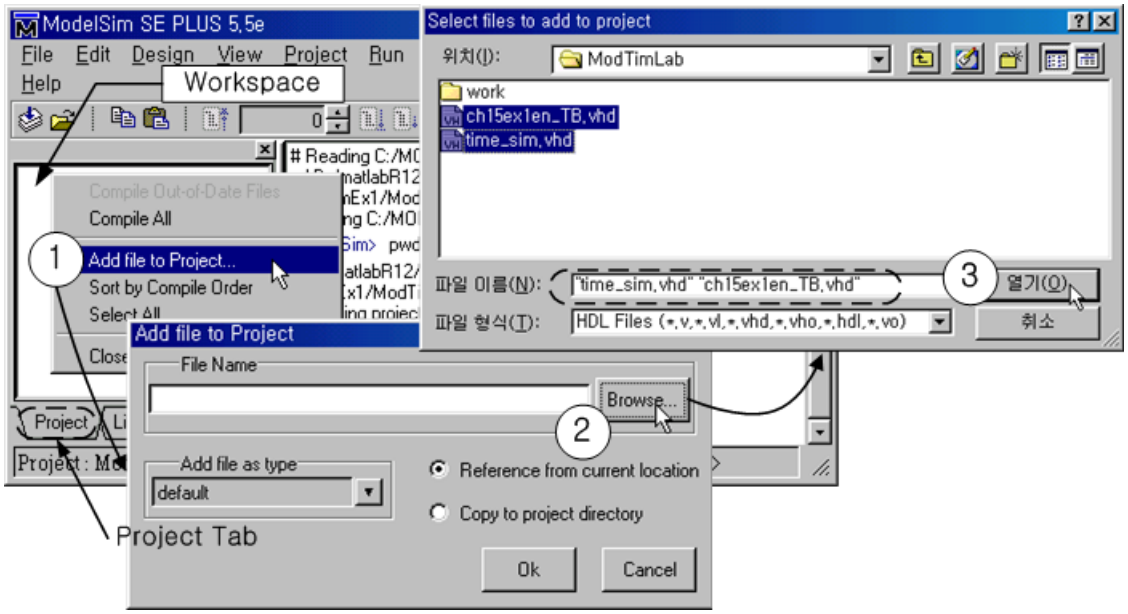
```
ModelSim>pwd
# D:/matlabR12/Book2001/Book4/Contents/Part3/Ch15_Xilinx/ModTimEx1/ModTimLab
```

이제, 새로운 project를 만들기 위해서 Modelsim의 main window에서 **File/New/Project** menu를 click해 준다. 그러면, [그림 19-50]과 같이 **Create Project** window가 나타날 것이다.
Project name을 그림과 같이 지정하고, **OK**버튼을 click해 준다.



[그림 19-50] Create Project window.

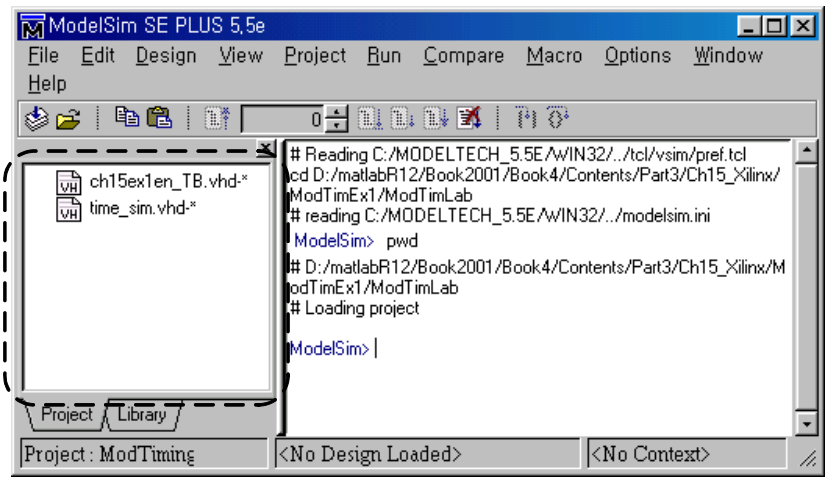
OK 버튼을 click 해 주면, [그림 19-51]과 같이 기존에 있었던, Library tab 왼쪽에 **Project** tab이 새롭게 생겨난다.



[그림 19-51] 새로운 project 생성 방법.

Project tab이 새롭게 만들어지면, **Workspace** 영역에서 mouse의 오른쪽 버튼을 click 해준다. 그러면, ①번과 같이 contextmenu가 나타날 것이다. 여기서, **Add file to Project...** menu를 click 해 주면, ②번과 같이 **Add file to Project** window가 나타난다. 여기서, **Browse...** 버튼을 click 해주면, ③번과 같이 **Select files to add to project** window가 나타난다. 여기서, project에 첨가할 file들을 선택하면 된다. **Ctrl** 버튼을 누른 상태에서 mouse를 사용하여 여러 개의 file을 동시에 선택한다. ③번에서는 time_sim.vhd, ch15ex1en_TB.vhd file을 선택하였다.

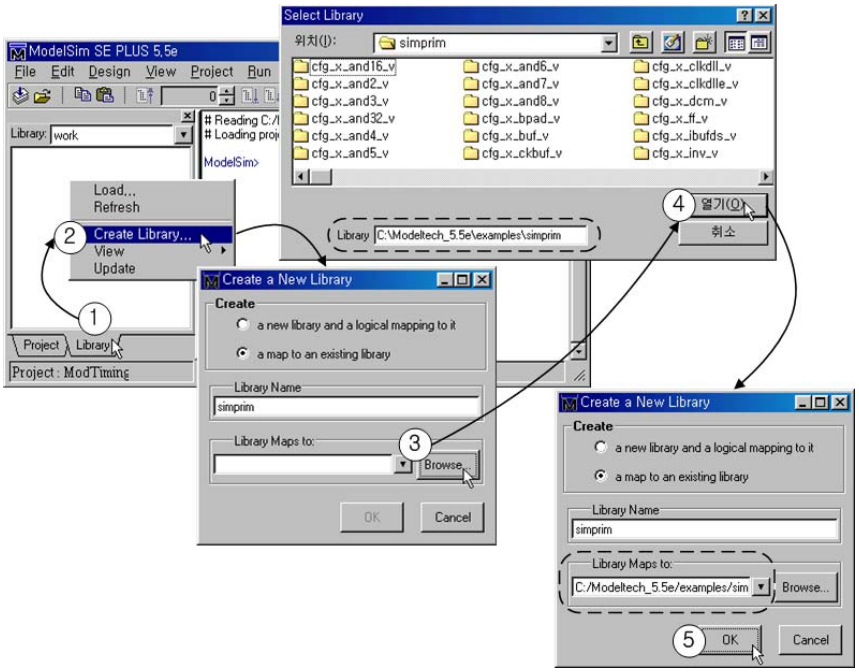
이제, ③번 window에서 **열기** 버튼을 click 해 주고, ②번 window에서는 **OK** 버튼을 click 해주면, [그림 19-52]와 같이 현재 Project의 workspace에 선택한 2개의 file들이 첨가된 것을 볼 수 있다.



[그림 19-52] Project의 workspace에 2개의 files가 첨가된 모습.

그런데, time_sim.vhd file을 열어보면, 주로 component에 대한 port mapping과 generic 문으로 이루어진 것을 알 수 있다.
component에 대한 port mapping이란, C/C++언어에서 보면, 마치, 함수의 원형(prototype)만 선언되어 있고, 실질적인 함수의 내용(body)은 다른 파일에 정의되어 있는 것과 같은 뜻이다. 바로 함수의 내용은 simprim library에 있고, 여기서 generic문은 C/C++ Code에서 보면, 외부 변수를 선언 한 것과 같은 뜻인데, 이 외부 변수의 실질적인 값은 time_sim.sdf file에 있다. 그래서, time_sim.sdf file을 열어보면 주로 수치가 기입되어 있는 것을 볼 수 있을 것이다.
결국, time_sim.vdh file을 compiling하기 위해서는 simprim library와 time_sim.sdf file이 필요하다는 것을 알 수 있을 것이다. 우선, simprim library를 현재 project에서 사용할 수 있도록 library mapping을 수행해 보도록 하자.

일반적으로 library mapping은 modelsim의 **vmap** command를 사용하지만, [그림 19-53]과 같이 GUI interface로 간단히 수행할 수도 있다.



[그림 19-53] simprim **library mapping 방법**.

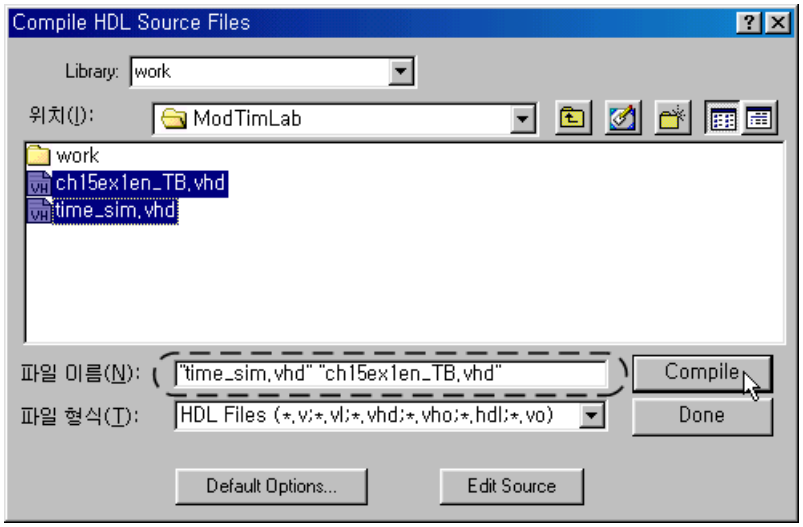
즉, **Library** tab을 click한 뒤에, workspace 영역에서 mouse의 오른쪽 버튼을 click하여 나타나는 contextmenu에서 **Create Library...** menu를 선택한다. 그러면, ③번과 같이 **Create a New Library** window가 나타날 것이다. 이제, **Library Name**에 그림과 같이 **simprim**이라고 지정한 뒤에, **Browse** 버튼을 click 해 주면, ④번과 같이 **Select Library** window가 나타날 것이다. 여기서, simprim이 compiling 된 위치를 지정해 주고, **열기** 버튼을 click 해 주면, ⑤번과 같이 다시, **Create a New Library** window가 나타나며, **Library Maps to**에 선택한 simprim의 위치가 나타난다. 이제, **OK** 버튼을 click 해 주면, simprim library가 현재 project에 mapping되게 된다.

Modelsim 뿐만 아니라, Visual C++에서 외부 library를 link할 때도 방법만 다르지 동일한

개념으로 link를 해 주어야 한다.

어쨌든, 이제, 현재 project에서 simprim library에 있는 component body를 사용할 수 있게 되었으므로, time_sim.vhd file을 compiling 할 수 있게 되었다.

Modelsim에서 임의의 file을 compiling 하기 위해서는 Modelsim main window의 toolbar 중에서 icon을 click 해 주면 된다. 그러면, [그림 19-54]와 같이 **Compile HDL Source Files** window가 나타난다.
여기서, **Ctrl** key를 누른 상태에서 mouse로 compiling 할 files를 선택하면 된다.
단, [그림 19-54]에서 보여준 것과 같이 **low level VHDL file**이 먼저 나와야 한다.



[그림 19-54] Modelsim에서 HDL files의 Compiling 방법.

일반적으로 testbench file이 항상, 개발하려는 HDL files보다 level이 높다. 이 사실은 가만히 생각해 보면 당연한 얘기가 될 것이다.
그래서, [그림 19-54]와 같이 time_sim.vdh file이 ch15ex1en_TB.vhd file보다 먼저 나온 것이다. 이 순서가 바뀌면 compiling error가 발생한다.
이것은 이전에 설명한 LeonardoSpectrum에서 HDL files의 합성시, Top-Level이 맨 아래에 놓여야 한다는 것과 같은 맥락이다.

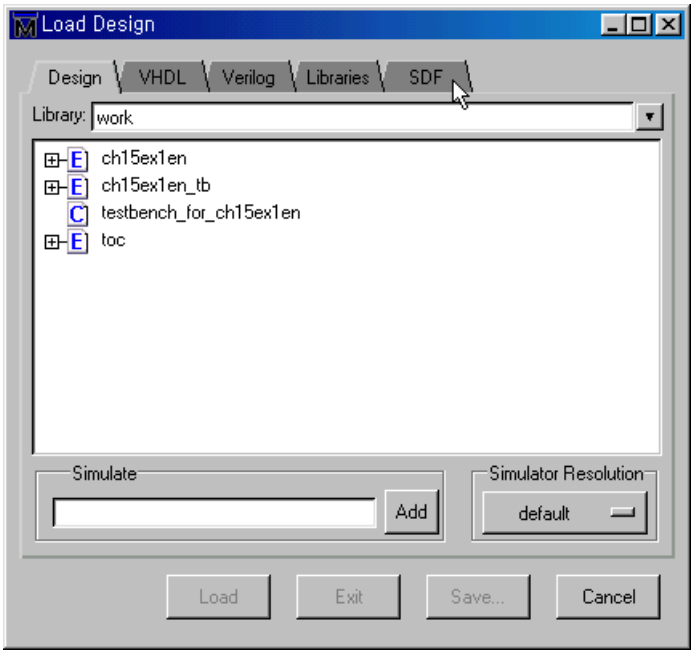
이제, **Compile** 버튼을 click 해 주고, **Done** 버튼을 click 해주면, Modelsim **transcript area**([그림 19-45] 참조)에 다음과 같은 message가 나타날 것이다.

… 중간 생략 …

```
# -- Loading package vital_primitives
# -- Loading package vpackage
# -- Loading entity ch15ex1en
# -- Compiling configuration testbench_for_ch15ex1en
# -- Loading entity ch15ex1en_tb
# -- Loading architecture tb_architecture of ch15ex1en_tb
ModelSim>
```

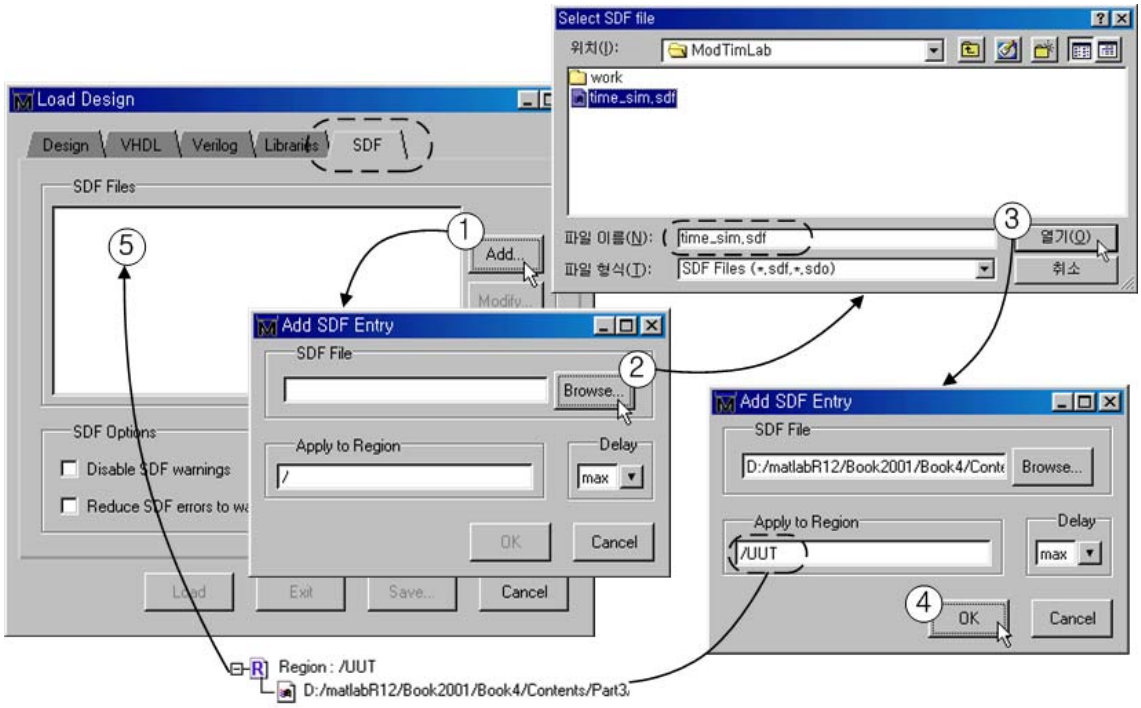
위와 같이 붉은 색의 error message가 나타나지 않았으면 compiling은 성공적으로 이루어진 것이다.

이제, 본격적으로 timing simulation을 하기 위해서 modelsim main window의 toolbar 중에서  icon을 click 해 준다. 그러면, [그림 19-55]와 같이 **Load Design** window가 나타날 것이다.

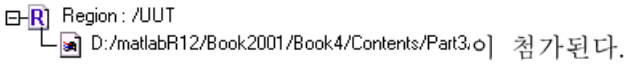


[그림 19-55] **Load Design** window.

Load Design window가 나타나면, [그림 19-55]에서 보여준 것과 같이 **SDF** tab을 click 해 준다. 이제, [그림 19-56]과 같은 과정은 거치면 현재 project에서 time_sim.sdf file을 사용할 수 있게 된다.



[그림 19-56] SDF file 사용 방법.

즉, ①번과 같이 **Add...** 버튼을 click 해 주면, **Add SDF Entry** window가 나타나고, 여기서, ②번과 같이 **Browse...** 버튼을 click 해 주면, ③번과 같이 **Select SDF File** window가 나타난다. 여기서, 원하는 sdf file인 time_sim.sdf file을 선택한 뒤에, **열기** 버튼을 click 해 주면, 다시, 선택한 SDF file의 경로가 첨가된 **Add SDF Entry** window가 나타난다. 이제, **Apply to Region**에 그림과 같이 **UUT**문자열을 첨가 해 준 뒤에, **OK** 버튼을 click해 준다. 그러면, ⑤번 영역에 

이 첨가된다.

④번 과정에서 문자열 UUT를 첨가한 것은 testbench file과 연관성이 있다.
즉, testbench file인 ch15ex1en_TB.vhd을 열어보면, 중간에 다음과 같은 code가 있을 것이다.

… 중간 생략 …

```
--Signal is used to stop clock signal generators
        signal END_SIM: BOOLEAN:=FALSE;

        -- Add your code here ...

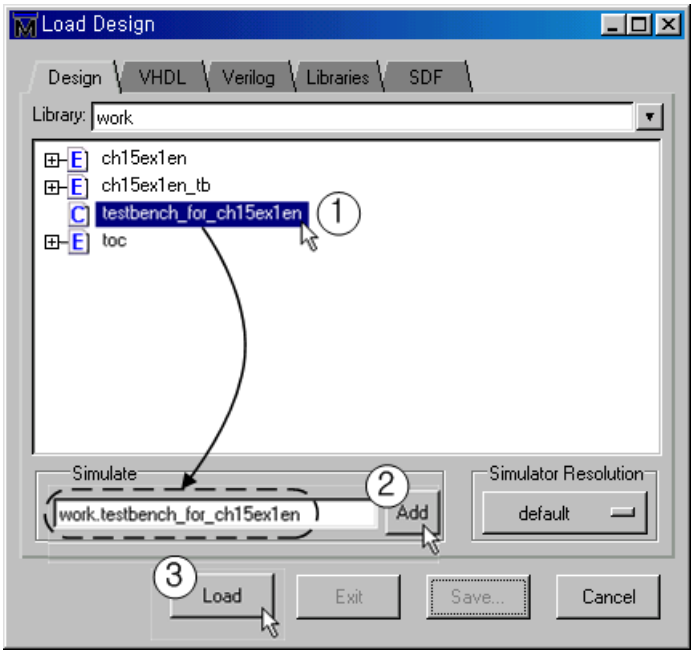
begin

        -- Unit Under Test port map
        UUT : ch15ex1en
        port map
                (x => x,
                y => y,
```

```
        z => z );
                    … 이하 생략 …
```

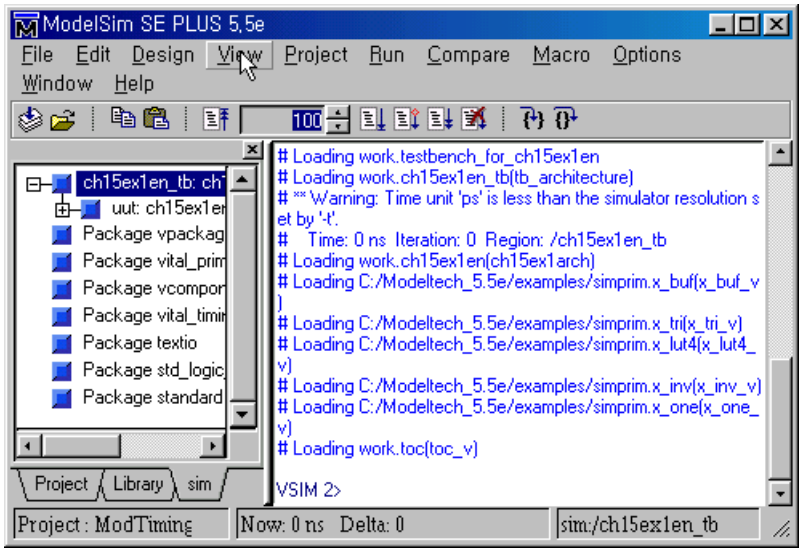
지금 현재, 우리가 개발하고 있는 file인 Ch15Ex1VHD.vhd의 entity name이 ch15ex1en이며, UUT는 ch15ex1en의 label name으로 사용되고 있다. 바로, 이 label name을 사용한 것이다.

[그림 19-56]의 과정을 거치면, 현재 project에 time_sim.sdf file이 등록된다. 이제, [그림 19-56]에서 다시, **Design** tab을 click 해준 뒤에, [그림 19-57]과 같은 과정을 거치면 timing simulation의 모든 준비가 완료된 셈이다.



[그림 19-57] Timing Simulation Setting 방법.

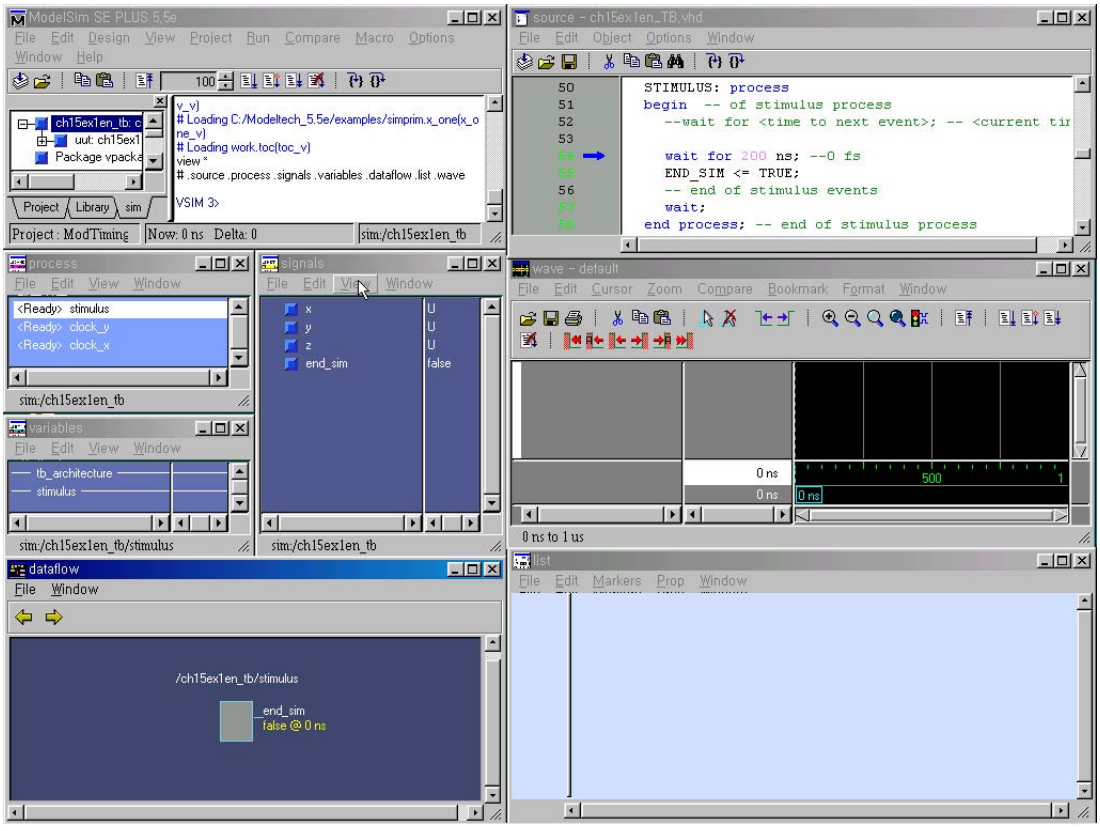
즉, ①번과 같이 **testbench_for_ch15ex1en**을 선택한 뒤에, ②번의 **Add** 버튼을 click 해 준다. 그리고 나서, **Load** 버튼을 click 해 주면, [그림 19-58]과 같이 modelsim main window의 prompt가 바뀌면서 simulation mode로 된다.



[그림 19-58] Modelsim Simulation Mode.

자! 이제, 본격적으로 timing simulation을 해보도록 하자.

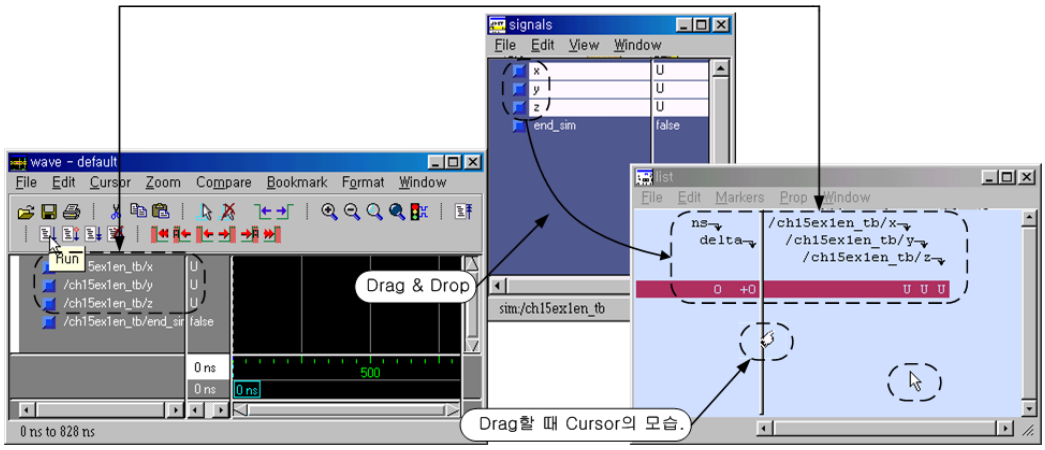
[그림 19-58]과 같이 View menu를 click해 준 뒤에, **All** menu를 click해 주자. 그러면, [그림 19-59]와 같이 8개의 window가 더 생겨난다.



[그림 19-59] Modelsim의 9개 windows 모습.

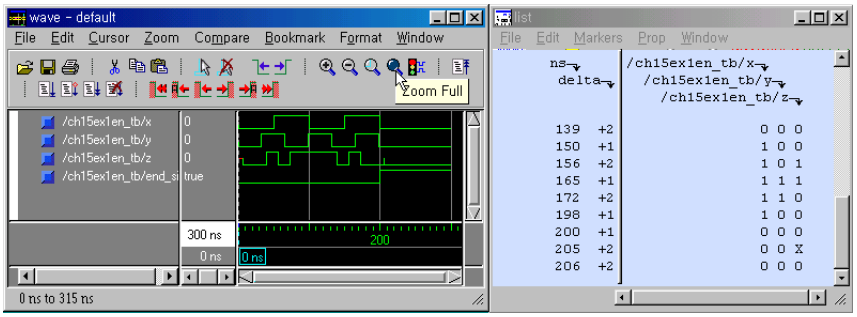
갑자기 여러 개의 windows가 나타나서 혼란스러울 지도 모른다. 그러나, 각각의 window는 저마 중요한 특성을 가지고 있으므로 실질적으로, VHDL로 simulation을 재대로 하려면 이들 모든 windows의 특성들을 이해해야 한다.
여기서 우리가 주로 다룰 windows는 **signals** window, **wave** window, **list** window이다.

이제, [그림 19-59]와 같이 signals window의 **View** menu에서 **Wave/Signals in Region** menu를 선택해 주면, [그림 19-60(a)]와 같이 wave window에 signals window의 item들인 **x, y, z, end_sim**이 등록된다.
이처럼, signals window의 menu를 사용하여 wave window나 list window에 item들을 등록할 수도 있지만, signals window에서 wave 또는 list window에 등록하고 싶은 item들을 **Ctrl** key와 mouse를 사용하여 선택한 뒤에 [그림 19-60(b)]와 같이 wave 또는 list window 영역 안에 drag & drop하여 등록할 수도 있다.



[그림 19-60] signals window의 items를 wave/list window에 등록하는 방법.

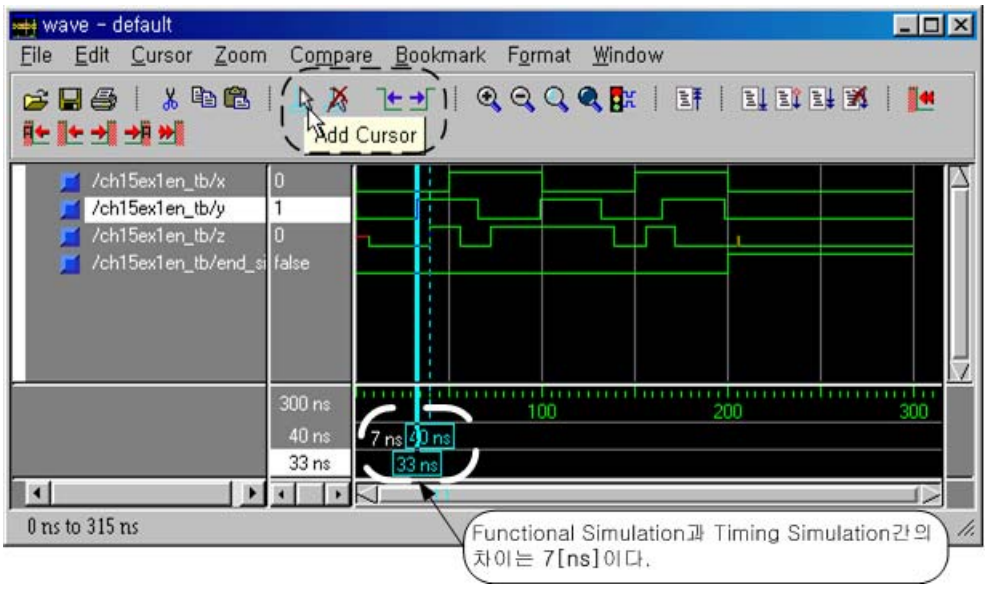
어쨌든, wave window와 list window에 timing을 보기 위한 items를 signals window로부터 모두 등록하였으면, [그림 19-60(a)]에서 보여준 것과 같이 mouse를 사용하여 **Run** icon(즉,)을 click 해 보도록 하자.
그러면, wave window에는 Active-HDL에서 보았던 timing 그래프가 나타날 것이고, List window는 수치적으로 timing도를 나타날 것이다.



[그림 19-61] Timing Simulation 결과.

[그림 19-61]는 Zoom Full icon(즉,)을 click 해 준 것이다.

이제, wave window의 결과를 좀 더 자세히 살펴보도록 하자.
[그림 19-62]에는 Cursor라고 불리는 2개의 수직선들이 있는데, 이와 같은 수직선의 형태를 취하는 cursor를 첨가하기 위해서는 그림에서 보여준 것과 같이 **Add Cursor** icon을 click 해 주면 된다.



[그림 19-62] Timing Simulation 결과. (Wave window)

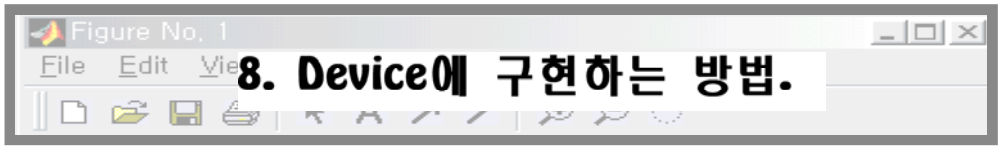
그리고, 만일, 그림과 같이 **y** port를 mouse로 선택한 뒤에, icon을 각각 click 해 보면, signal level이 바뀌는 곳을 순서대로 cursor가 찾아서 위치하게 된다.

[그림 19-62]는 Xilinx의 Virtex 600E(약어로 **XCV600E**)라는 device의 내부에 실질적으로 개발하려는 VHDL Code인 Ch15Ex1VHD.vhd를 합성 및 P&R하여 구현한 경우인 timing simulation이다. 그러므로, Virtex 600E 내부에서 발생하는 갖가지 delay 성분들을 충분히 고려한 결과를 보여주고 있는 것이다.

실제로, XCV600E에 Ch15Ex1VHD.vhd의 bit-stream code를 구운(burning) 뒤에, 보드에 실장하고 입력 **x, y**를 인가하면, [그림 19-62]와 같이 출력 **z**를 발생시켜 준다.

[그림 19-62]와 Active-HDL로 수행한 Functional Simulation의 결과인 [그림 19-22]와 비교해 보기 바란다.

지금까지 Modelsim을 이용한 Timing Simulation에 대해서 간단히 살펴보았다. 이제, 마지막으로 [그림 19-4]와 같이 개발한 VHDL Code인 Ch15Ex1VHD.vhd를 XCV600E에 구워보자.

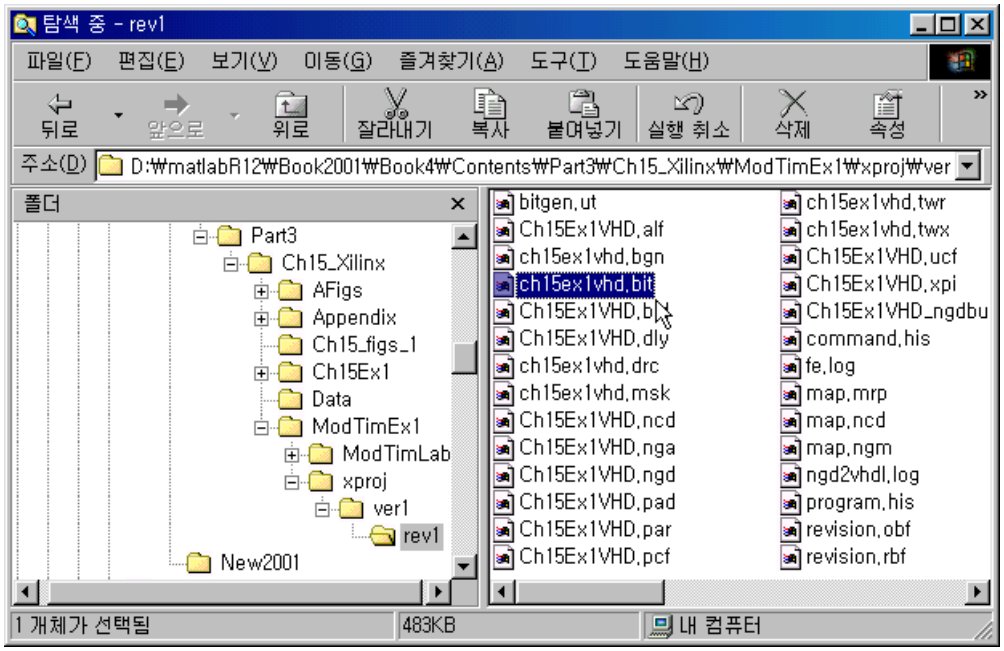


개발한 VHDL Code인 Ch15Ex1VHD.vhd에 대한 timing simulation의 결과가 원하는 대로 나왔으면, 이제, target device인 XCV600E에 구워보도록 하자.

일반적으로, timing simulation이 원하는 대로 나왔으면, 그 개발한 VHDL Code를 Chip에 구운 뒤, 보드에 실장하여 동작을 시키면, 대략 40[MHz]까지는 timing simulation과 거의 동일하게 작동한다. 그러나, 고속의 Chip을 구현하려는 경우, RLC에 의한 작은 시상수의 변화로도 쉽게 timing이 어긋날 수가 있으므로 주의해야 한다.

또한, Xilinx의 Virtex E Series, 즉, XCV600E, XCV1000E 등등의 device들은 전원에 민감하므로 전원부 설계에 주의를 해야 하며, 저가의 FPGA Chip인 SpartanII series의 경우에는 PROM의 사용에 주의를 해야 한다. 그리고, Altera의 Apex20K series FPGA를 사용할 때는 40[MHz] 이상의 고속 RAM을 붙일 때, 즉, I/O로 사용할 때, 특별히 주의를 해야 한다.
어쨌든, 나름대로 조금 더 주의를 하면 문제없이 사용할 수 있다.

앞에서 우리는 Xilinx의 **Design Manager**를 사용하여 P&R을 수행하였다. 이와 같이 P&R을 모두 수행하면 **xproj**이라는 directory가 새롭게 생겨난다.
xproj directory의 최하위까지 내려가 보면, [그림 19-63]과 같이 수많은 files가 생겨난 것을 볼 수 있을 것이다.

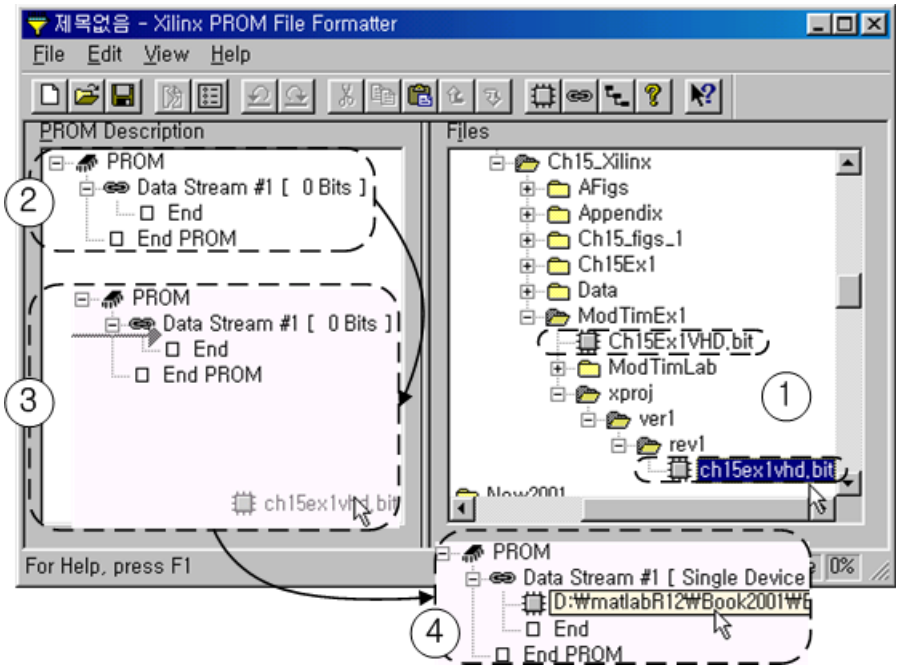


[그림 19-63] **xproj** directory의 file list.

그런데, 가만히 살펴보니, bit-stream code인 **ch15ex1vhd.bit** file이 존재하는 것을 볼 수 있을 것이다.

ch15ex1vhd.bit file이 만들어 졌으면, [그림 19-36]에서 

을 선택한다. 그러면, [그림 19-64]와 같이 Xilinx의 **PROM File Formatter** window가 나타난다.

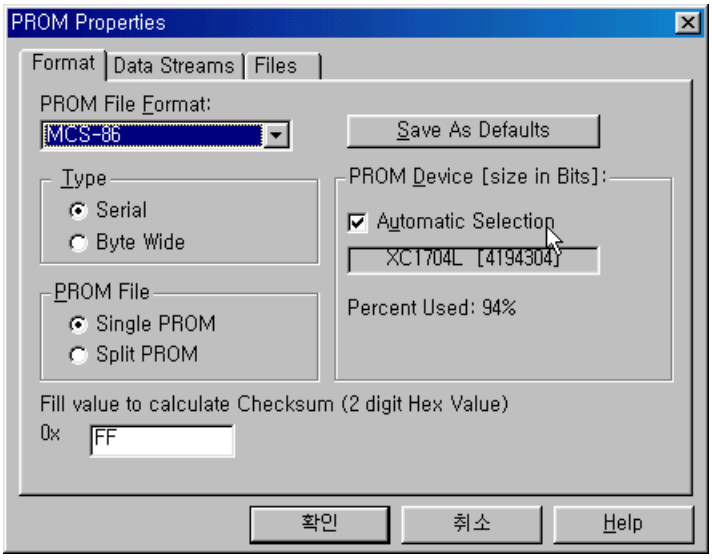


[그림 19-64] Xilinx의 **PROM File Formatter** window.

앞에서도 설명하였듯이 FPGA는 전원이 끊기면, 저장되어 있던 모든 내용들이 지워진다. 그래서, PROM에 FPGA에 저장할 내용들을 기록해 두고, 전원이 들어오면 PROM에 저장되어 있던 내용들이 자동으로 FPGA에 기록되도록 하는 방법을 사용한다.

이처럼, PROM에 bit stream file인 **ch15ex1vhd.bit** file을 저장하기 위해서는 일반적으로 **ch15ex1vhd.mcs** file로 변환해 주어야 하는 데, 이와 같은 기능을 수행하는 tool이 바로 **PROM File Formatter**인 것이다.

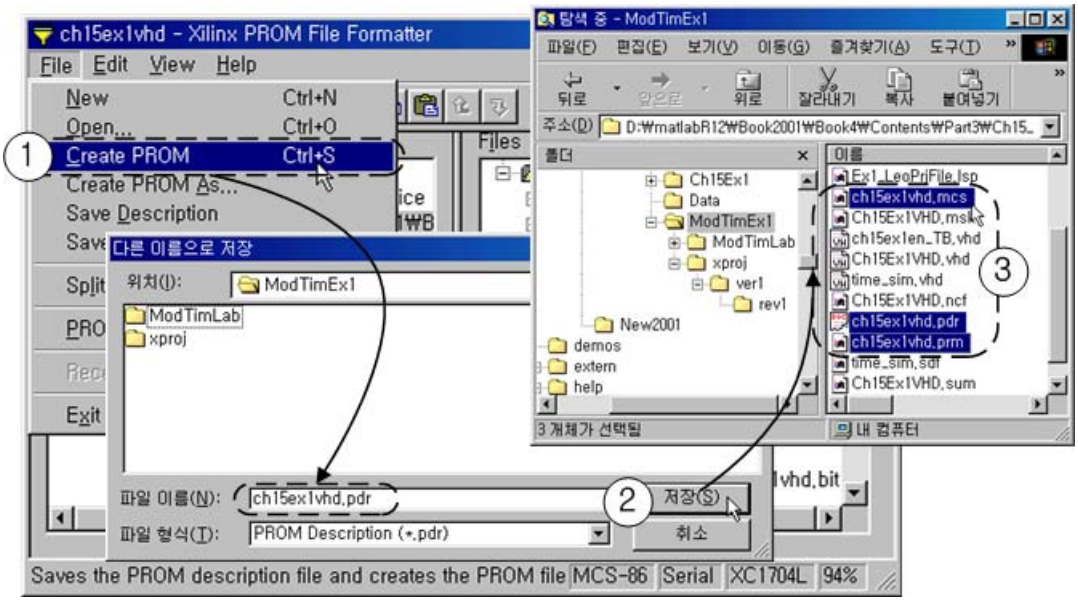
ch15ex1vhd.bit file을 **ch15ex1vhd.mcs** file로 바꾸기 위해서는 [그림 19-64]의 ①번과 같이 **ch15ex1vhd.bit** file의 위치를 찾아본다. 그리고, ③번과 같이 **ch15ex1vhd.bit** file을 mouse로 drag 하여 ②번 영역에 drop해 준다. 그러면, ④번과 같이 bit file이 등록되는 것을 볼 수 있을 것이다. 이제, toolbar에 있는 icon을 click해 주면, [그림 19-65]와 같이 **PROM Properties** window가 나타날 것이다.



[그림 19-65] **PROM Properties** window.

PROM Properties window를 보니, 자동으로 PROM device인 **XC1704L** device가 선택되어 있다. 만일, 원하는 PROM file이 있으면, **Automatic Selection** checkbox를 선택해 주고 새로운 PROM device를 지정해 주면 된다. 이제, 원하는 PROM device를 선택하였으면, **확인** 버튼을 click 해 준다.

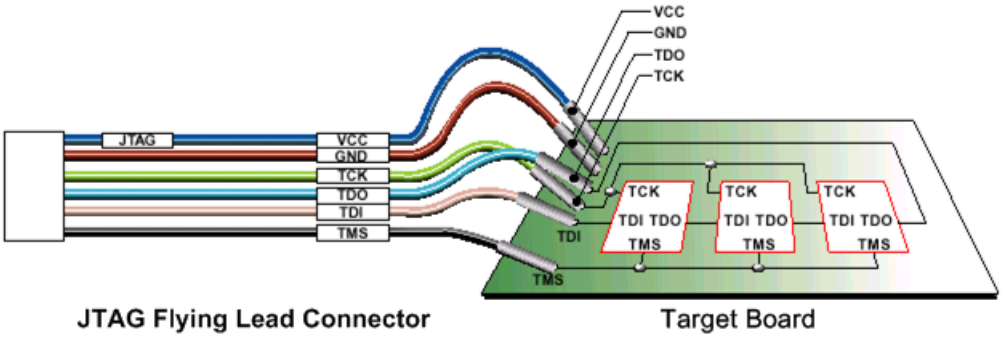
그리고, [그림 19-66]의 ①번과 같이 **File/Create PROM** Menu를 선택해 주면, ②번과 같이 **ch15ex1vhd.pdr** file을 저장해 주는 window가 나타난다.



[그림 19-66] PROM Properties window로부터 ***.mcs** file 생성 방법.

PROM File Formatter는 **ch15ex1vhd.bit** file을 사용하여 ③번과 같이 **ch15ex1vhd.mcs** file과 PROM Description File인 **ch15ex1vhd.pdr** file과 PROM에 대한 data의 memory map을 갖고 있는 **ch15ex1vhd.prm** file, 이렇게 3개의 files를 만들어 준다.

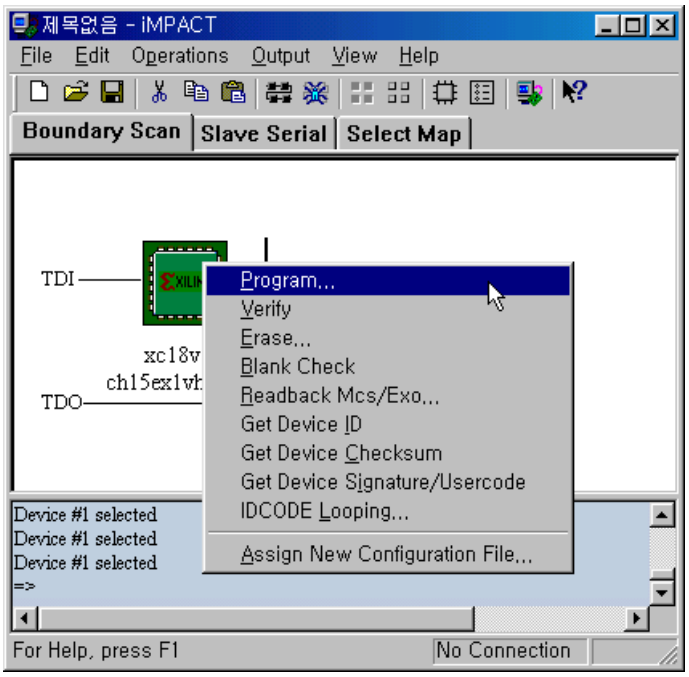
이제, ch15ex1vhd.mcs file을 저장할 PROM이 [그림 19-67]과 같이 보드에 실장되어 있으면, JTAG Cable을 **iMPACT.pdf** manual의 **Chapter 2. Cables**에 있는 내용을 참고하여 PC의 병렬 port에 연결하여 사용하면 된다.



[그림 19-67] **ch15ex1vhd.mcs** file을 PROM에 저장하는 방법.

만일, PROM이 올바로 실장이 되어 있고, 전원이 들어와 있으면, [그림 19-36]에서 

을 click 해 주면, [그림 19-68]과 같이 **iMPACT** window가 나타난다.



[그림 19-68] **iMPACT** window의 모습.

이곳에서 ch15ex1vhd.mcs file을 선택해 준 뒤에, 그림과 같이 mouse의 오른쪽 버튼을 click 해 주고, **Program...** menu를 click 해 주면 ch15ex1vhd.mcs file을 지정한 PROM에 구워준다.

또한, [그림 19-68]에서 **Slave Serial**과 **Select Map** tab은 Xilinx device에 직접 bit stream code를 구현할 때 사용한다. 일반적으로, PC에서 Xilinx device에 직접 bit stream code를 구현해 줄 때는 PC를 통하여 clock이 Xilinx device에 가해 주므로, PC가 master가 되고, Xilinx device가 slave가 된다. 그러나, PROM을 통하여 Xilinx device를 구현해 줄 때

는 Xilinx device가 PROM에 clock을 공급해 주므로 Xilinx device가 master가 된다.
그리고, Select Map mode보다는 Slave Serial mode를 많이 사용하며, 관련된 회로도는 Xilinx engineer에게 요청하거나 manual을 보고 설계하면 된다.

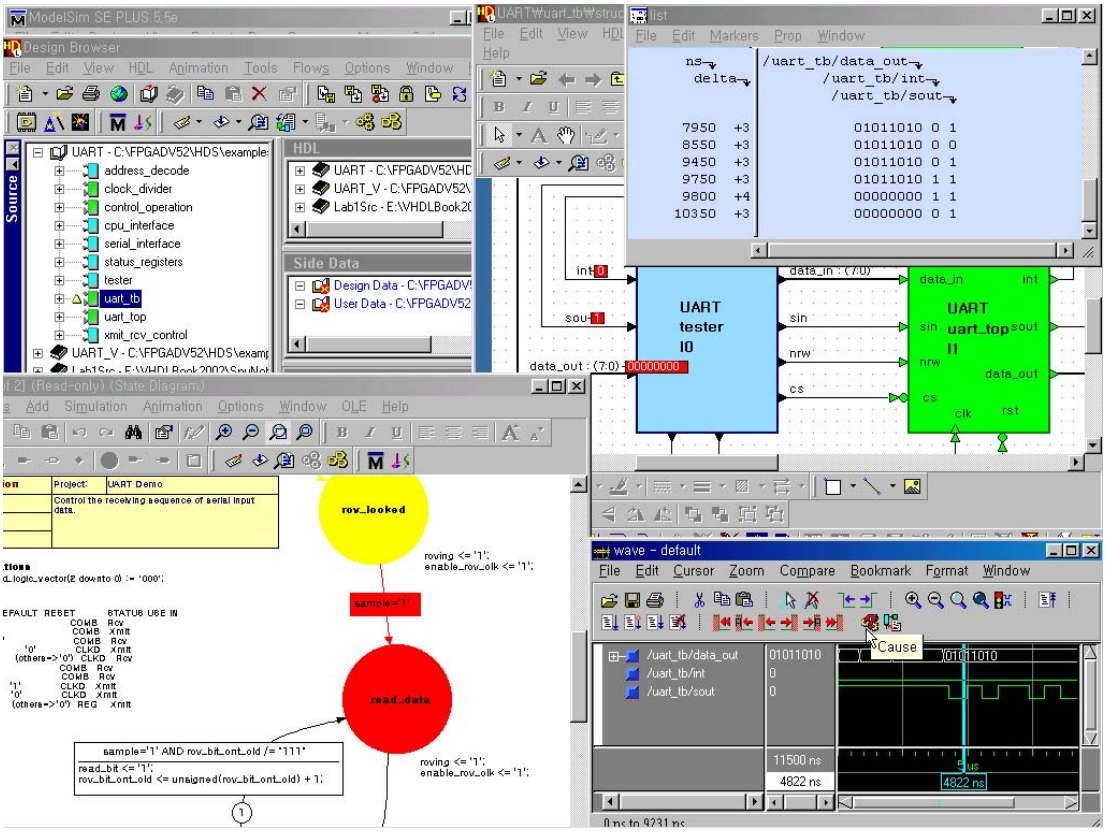
지금까지 VHDL로 Code를 개발하는 전 과정을 간단하게나마 설명을 하였다.

사실, HDL Code를 사용하여 상용으로 판매할 수 있는 Chip을 개발한다는 것은 결코 쉬운 일이 아니다. 우리가 보통, VHDL 또는 Verilog와 같은 HDL 책을 구입하여 열심히 배울 수 있는 것은 극히 한정적이다. 실제로 해당 tool들과 사용할 device의 특성, 그리고 실장 될 board 설계 방법과 device driver에 대한 software의 개념이 없으면 안 된다.

또한, 개발한 VHDL Code가 수백 또는 수천 lines에 달하는 경우, 아마도, 6개월 뒤에, 그 VHDL Code를 보면 전혀 이해하기가 힘들 것이다. 그리고, 수많은 lines를 modelsim이나 Active-HDL의 debugging 기능으로 bug를 잡아내는 것도 만만치 않은 일이다.

이와 같은 **유지 보수 문제**와 **debugging 기능**에 대한 보다 효율적인 방법을 제시하는 것이 바로 mentor 회사의 **Renoir**라고 생각한다. 단, 지금은 **HDL Designer**로 명칭을 바꾸었다. 이 HDL Designer는 **state diagram, truth table, block diagram, flow chart**등을 사용하여 개발자가 보다 쉽게 Visual 한 방법으로 HDL Code를 개발할 수 있도록 도와 준다. 무엇보다도 HDL Designer가 만들어준 HDL Code는 일반적으로 timing simulation을 보장해 준다. 대부분의 HDL Code 개발자가 어려움을 겪는 것은 바로 timing simulation에서 발생하는 bug 수정 작업이다. 이것은 충분한 경험이 있지 않은 분들에게는 무척 어려운 문제가 된다. 또한, HDL Designer는 animation 방법으로 각각의 Visual Item들을 단계 별로 실행해 주므로 bug을 찾는 것도 쉽고, 수천 lines에 달하는 HDL Code보다는 수십장의 Visual items로 구성되어 있는 것이 유지 보수하기도 쉽다.

[그림 19-69]는 HDL Designer pro가 sample로 제공하는 UART에 대한 HDL Code를 animation하는 모습과 modelsim의 wave window와 list window를 보여주고 있다.



[그림 19-69] HDL Designer pro의 모습.

이제, 지금까지 배운 VHDL에 대한 이해와 Chapter 10, Chapter 15의 내용을 바탕으로 Simulink를 사용하여 VHDL Code를 개발해주는 System Generator에 대해서 살펴보도록 하겠다.



이번 Chapter를 학습하기 위해서는 우선, Chapter 10의 **2진 연산**과 Chapter 15. **Simulink**에 대한 지식이 반드시 있어야만 한다.

자! 이제, 2진 연산과 Simulink에 대한 이해, 그리고 앞에서 서술한 VHDL에 대한 이해가 갖추어졌다고 생각하고 Xilinx의 System Generator에 대해서 설명해 보도록 하겠다.

Xilinx 회사와 Mathworks 회사는 공동으로 Simulink 환경에서 개발한 Block diagram을 자동으로 HDL Code로 변환해 주는 tool인 **System Generator**를 개발하였다.
여기서 주의 할 것은 Mentor의 HDL Designer pro와 같이 사용자가 각각의 block diagram을 정의하여 자동으로 HDL Code를 완벽하게 생성해주는 것과는 달리, System Generator는 Xilinx Toolbox에서 제공하는 block들의 조합으로 HDL Code를 생성해 주지만, 실질적인

각각의 block에 대한 소스 code를 제공해 주는 것은 아니다.
즉, matlab의 *.dll file과 같이 사용만 할 수 있고, 내부 소스 code는 제공하지 않는다는 것이다.

현재, System generator의 version은 **2.1**이며, 오로지 Xilinx FPGAs에서**만** 사용할 수 있다. 즉, Xilinx의 device series인 **Virtex, Virtex-E, Virtex-II** 그리고, **Spartan-II**에 최적화된 Xilinx의 LogicCORE modules를 사용하여 HDL Code를 생성해 준다. 또한, System generator가 제공하는 대부분의 block들은 DSP functions, Communications, 기본적인 logic, Arithmetic, Digital에 대한 **bit-true**와 **cycle-true** Simulink library를 제공한다.

HDL Code가 자동으로 만들어지기 때문에 사용자는 생성된 HDL Code가 원하는 결과를 생성할 수 있는지 simulation만 수행해 주면 된다.

실질적으로 System Generator는 합성할 수 있는 VHDL Code뿐만 아니라, 그 VHDL Code에 대한 Testbench file, Timing Constraint files, Leonardo Spectrum으로 합성하는 데 필요한 TCL script와 Modelsim에서 simulation하는 데 필요한 TCL Script file등 다양한 파일들을 만들어 준다.

System Generator**2.1**을 사용하기 위해서는 다음과 같은 software들이 필요하다.

① Windows98/2000에서 사용할 수 있고, Windows NT4.0을 사용하시는 분들은 Service pack 6. 또는 그 이후 버전을 installation 해 주어야 한다.

② Xilinx Foundation Series **4.1i**와 service pack 3. 또는 Xilinx ISE 4.1i와 Service pack1.이 필요하다. 현재, 필자는 Xilinx Foundation Series 4.1i버전에 Service pack 3.을 installation 하여 사용하고 있다.
물론, System Generator V2.0은 Xilinx Foundation Series **3.1i**에서 사용할 수 있다.

③ Matlab v**6.x**와 Simulink v**4.x**가 필요하고, Signal processing Toolbox와 DSP Blockset, Communications Toolbox, Communications Blockset이 있으면 좋겠다.

④ 생성된 HDL Code를 Simulation 해줄 Simulator인 Modelsim이 필요하며, 버전은 가능하면, **5.5**이상이면 좋겠다. 만일, Modelsim이 없다면, Xilinx에서 OEM방식으로 공급하는 **MXE**(Modelsim Xilinx Edition)를 사용해도 된다.

⑤ 생성된 HDL Code를 합성할 software가 필요하다. 보통, Xilinx Foundation Series는 예전부터 FPGA-Express를 OEM 방식으로 공급하여 왔다. 그래서, 그런지 FPGA-Express가 다른 합성 tool 보다 궁합이 잘 맞는 경향이 있다.

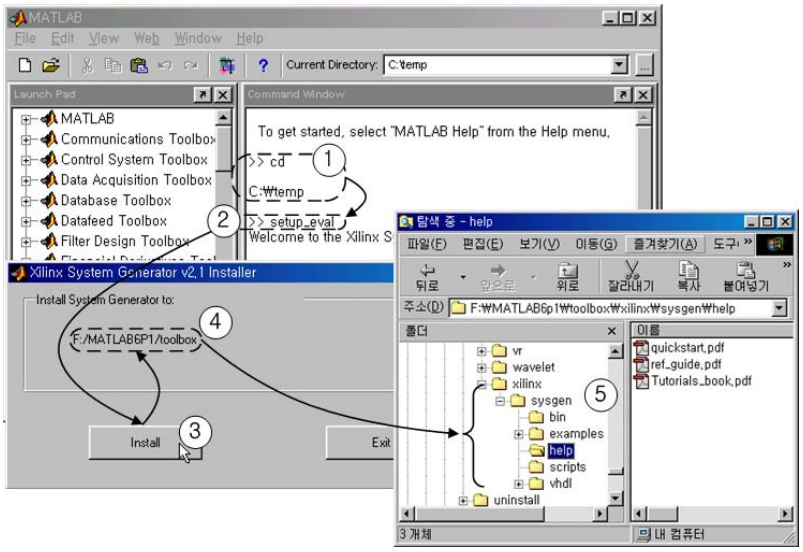
모든 Software들이 준비되었으면 System Generator의 기능에 대해서 살펴보도록 하자.

1) System Generator Installation 방법.

첨부한 CD안에 있는 System Generator**2.1**인 **SysgenEval_v2_1.exe**을 double click 해 준다. 그러면, 임시 directory에 readme.txt file과 matlab에서 사용할 수 있는 **setup_eval.dll** file을 생성해 준다. 단, System generator가 정품인 경우에는 setup_eval.dll file이 아니라, **setup.dll** file이 생성된다.

이제, setup_eval.dll file을 matlab의 command window에서 실행하기 위해서 [그림 19-70]의 ①번과 같이 setup_eval.dll file이 있는 directory로 이동한 뒤에, ②번과 같이 실행을 해본다.

그러면, Xilinx System Generator v2.1 Installer Window가 나타난다. **Install** 버튼을 click 해 주면, ④번과 같이 Matlab이 installation 되어 있는 directory의 toolbox 위치에 **Xilinx**라는 새로운 toolbox가 ⑤번과 같이 생겨난다.

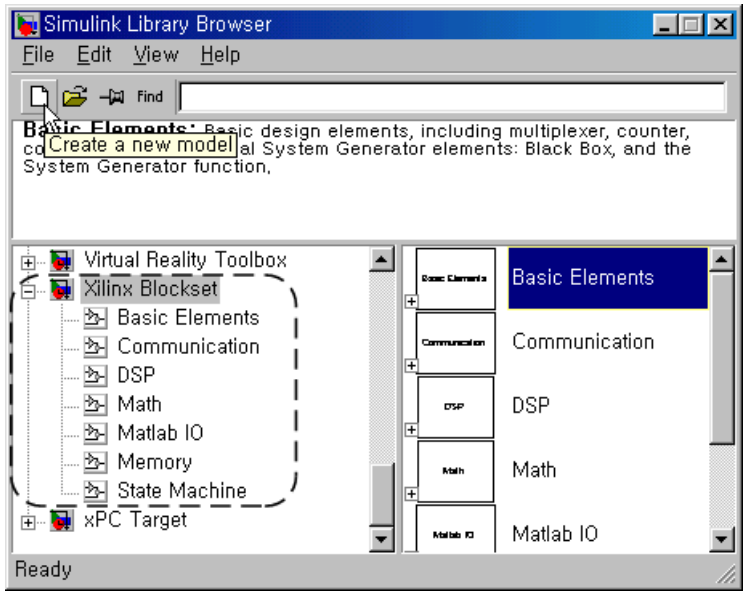


[그림 19-70] System Generator Installation 방법.

새로 생긴 Xilinx directory에는 그림과 같이 help directory가 있는데, System generator에 대한 보다 자세한 사항들을 알고 싶은 분들은 여기에 있는 문서들을 참조하면 된다.

2) Simulink와 System Generator의 interface 개념.

Matlab Desktop에서 Simulink를 실행해 보면, [그림 19-71]과 같이 Xilinx Blockset이 추가된 것을 볼 수 있을 것이다.



[그림 19-71] Xilinx Blockset.

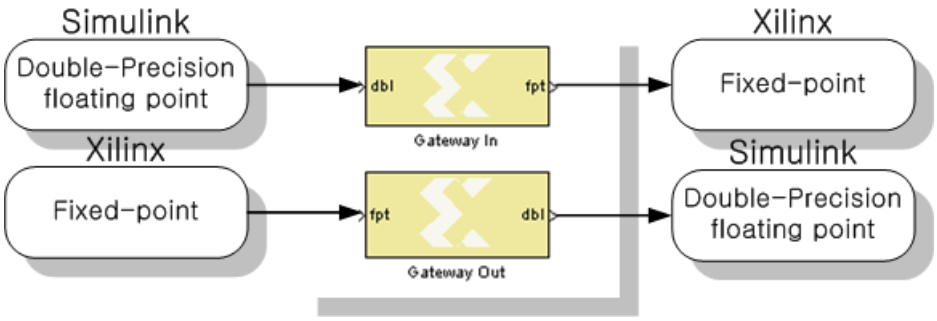
Basic Elements library에 있는 **System Generator block**은 Simulink로 개발한 model을 VHDL Code로 변환해 주는 block이다.

특별히, 주의 할 것은 Xilinx Blockset에 있는 각각의 block들은 FPGA에서 logic으로 구현되어야 하므로 **bit-true**와 **cycle-true**의 데이터 type을 사용한다는 것이다.
결국, Xilinx Blockset에 있는 각각의 block들은 일반적인 Simulink의 block들과 다른 데이터 type으로 정의되어 있다는 뜻이 된다.

Matlab에서 수행되는 모든 계산들은 64-bit double-precision floating-point를 사용하고, System Generator를 사용하는 Simulink 상의 Xilinx 부분은 fixed-point data format을 사용한다.
floating-point와 fixed point에 대한 자세한 사항들은 **Chapter 10. 2진 연산** 단원을 참고하기 바란다.

이처럼 Simulink 상의 block들과 Xilinx Block들간의 서로 상이한 data type을 conversion 해 주는 block이 바로 **Xilinx Gateway block**들이다.

[그림 19-72]에서는 Xilinx Gateway block을 보여주고 있다.

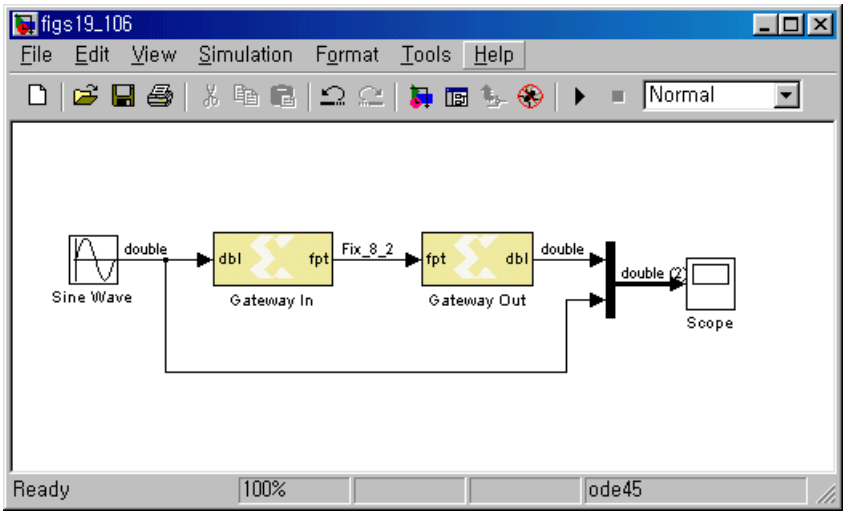


[그림 19-72] Xilinx Gateway block.

실질적으로, matlab으로 작성한 M-file 또는 Simulink로 작성한 *.mdl file을 C/C++로 작성한 program과 데이터 교환을 하려면 [그림 19-72]와 같이 gateway function이 필요하다.

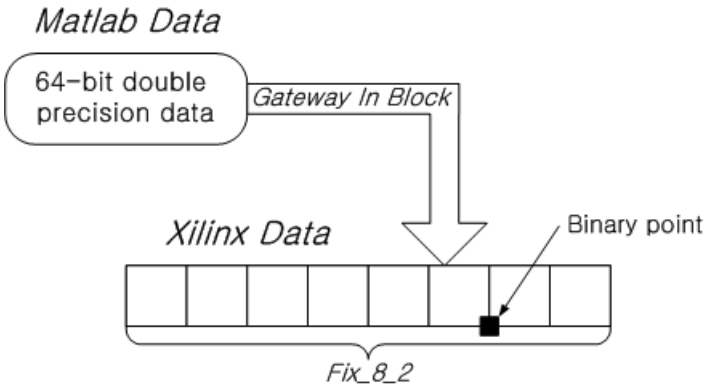
이처럼, 저마다 사용하는 데이터 type이 다른 program간에 데이터를 교환하기 위해서는 적절한 gateway function을 만들어 주어야 한다.
결국, gateway function은 통역관과 같은 역할을 한다고 생각하면 된다.
C/C++로 작성한 program을 Matlab에서 사용하는 방법은 “Matlab 완벽 가이드II(응용편)”에 자세히 설명되어 있으니 참고하기 바란다.

3) System Generator에 대한 간단한 예제(1) :
이제, [그림 19-73]과 같이 Simulink의 sine wave를 Xilinx Gateway block에 통과시켜준 경우와, 통과시켜주지 않은 경우, 어떠한 차이가 발생하는 지 살펴보도록 하자.



[그림 19-73] System Generator에 대한 간단한 예제.

[그림 19-73]에서 **Fix_8_2**라는 net name(signal name)은 Simulink의 64-bit double precision data type이 Gateway In block을 통과한 경우, [그림 19-74]와 같이 전체 2진 8bit 데이터 중에서 binary point의 위치가 2인 data format으로 바뀌었다는 뜻이다.



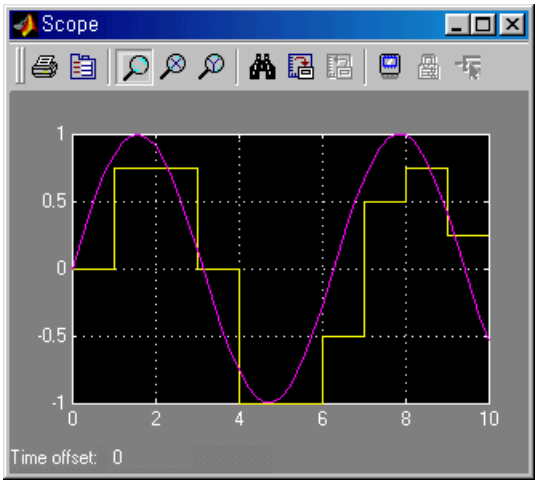
[그림 19-74] Matlab과 Xilinx간의 데이터 변환.

결국, 64-bit의 데이터 type이 8-bit의 data type으로 바뀌었으므로 데이터의 손실이 있을 수밖에 없다.
물론, Gateway In block의 Block parameter window에서 bit의 수를 64로 설정해 주면 데이터의 손실이 발생하지 않는다.

어쨌든, [그림 19-73]과 같이 Simulink Model을 설정하고, fig19_106으로 저장한 뒤에, 실행하면, [그림 19-75]와 같이 Scope block에서 결과를 볼 수 있다.

일반적으로 analog signal이 digital 회로에 입력되면, A/D Converter, 즉 analog 신호를 digital 신호로 바꾸어 주는 기능을 수행하는 device가 필요하다.
analog 신호를 digital 신호로 바꾸어 주는 것을 **sampling**이라고 하는데, [그림 19-75]는 Simulink의 Sine wave block에서 발생한 analog signal을 digital 신호로 sampling을 한 것

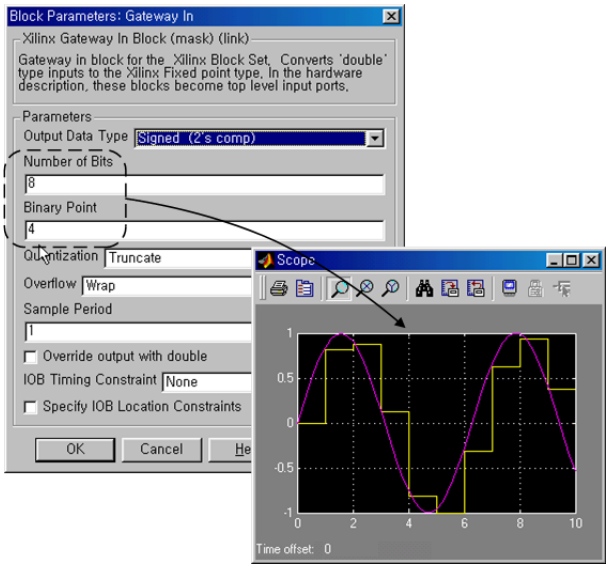
을 보여주고 있다.



[그림 19-75]

즉, y축(진폭)의 값이 연속인 analog 신호를 fix_8_2로 표현할 수 있는 불연속적인 값으로 sampling한 digital 신호를 보여주고 있다.
이때, y축(진폭)을 불연속적인 값으로 표현하는 것을 양자화(quantization)이라고 하며, 실질적인 analog 신호와 양자화 된 digital 신호의 차이를 양자화 error라고 한다. 보다 자세한 내용은 【참고문헌 1번】의 Page 119를 참고하기 바란다.

그렇다면, [그림 19-75]보다 양자화 error를 더 작게 하려면 어떻게 해야 될까?
[그림 19-76]은 Gateway In block의 block parameter window를 보여주고 있다.



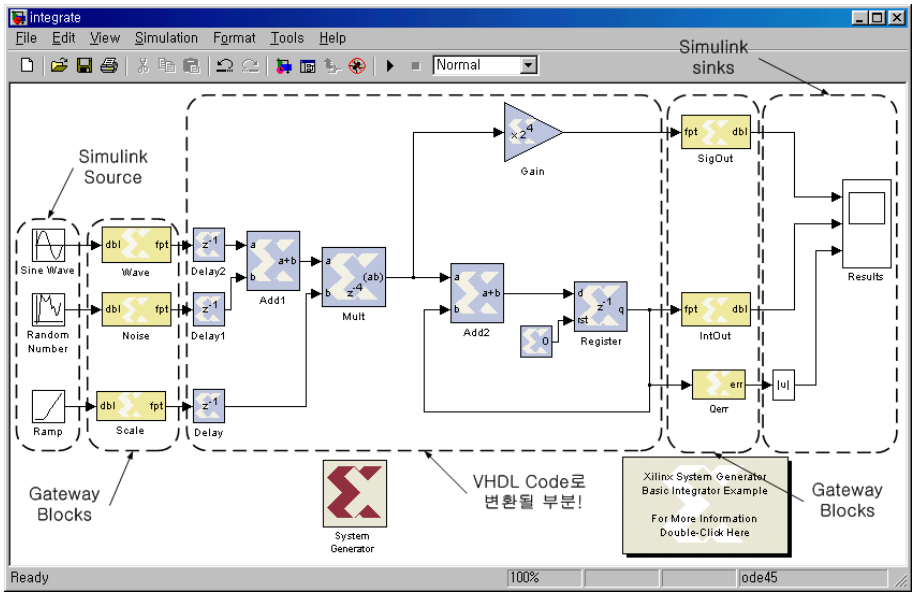
[그림 19-76]

실질적으로 sine wave의 최대·최소값은 각각 1과 -1이므로, Gateway In block의 **Binary Point** 값에 의존한다.

이제! Simulink와 Xilinx block간에 데이터 교환하는 방법을 알았으니 실질적으로 Simulink에서 system을 design한 뒤에 해당 VHDL Code를 생성해 주는 방법에 대해서 알아보자.

4) System Generator에 대한 간단한 예제(2) :

이번 예제에서 다룰 design은 [그림 19-77]과 같다.



[그림 19-77]

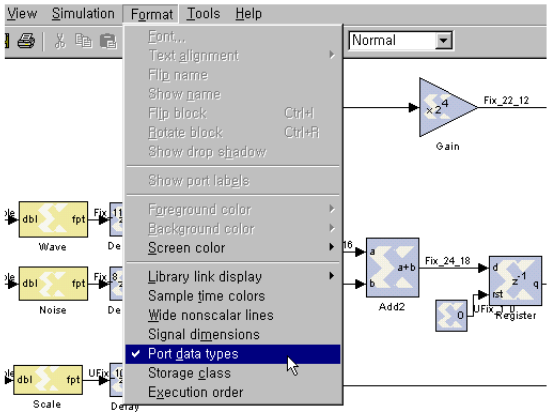
[그림 19-77]에 대한 Simulink file의 이름은 **integrate.mdl**이고, System generator에서 제공하는 demo file이다.

여러분이 **Chapter 15. Simulink의 소개**를 충분히 학습하였다면 쉽게 modeling할 수 있을 것이다.

[그림 19-77]에서는 입력으로 sine wave와 통신에서 자주 사용하는 **AWGN**(Additive White Gaussian Noise)를 사용하였고, 진폭에 대한 특성변화를 위해서 linear ramp signal을 첨가하였다.

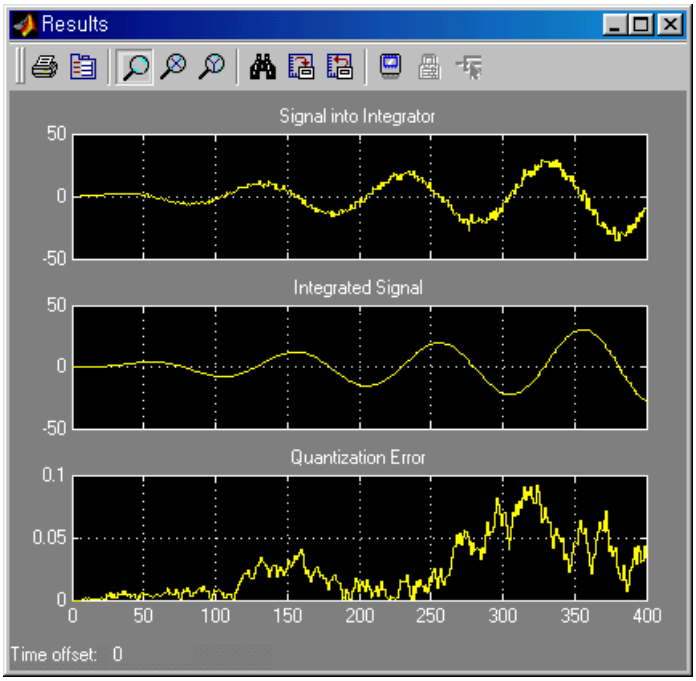
integrator는 AWGN을 약화시키는 단순한 low-pass filter의 작용을 한다.

[그림 19-77]에서 여러분은 어느 부분이 VHDL Code로 변환되는지, 그리고, 각각의 block들에 지정된 parameter 값들은 무엇인지를 파악해 보아야 한다. 그리고, matlab의 double precision data type이 VHDL Code로 변환 될 때, 몇 bits의 fixed-point로 바뀌는 지도 살펴보아야 한다. 각 block의 입·출력 port precision은 [그림 19-78]과 같이 **Format/Port data types** menu를 click해 주면 나타난다.



[그림 19-78] 각 block의 입·출력 port data type을 display하는 방법.

이제, [그림 19-77]에 있는 Results block, 즉, Scope block을 mouse로 double click 해 준 뒤에, Simulation button(즉, ▶)을 click 해 주면, [그림 19-79]와 같은 결과를 얻을 수 있다.

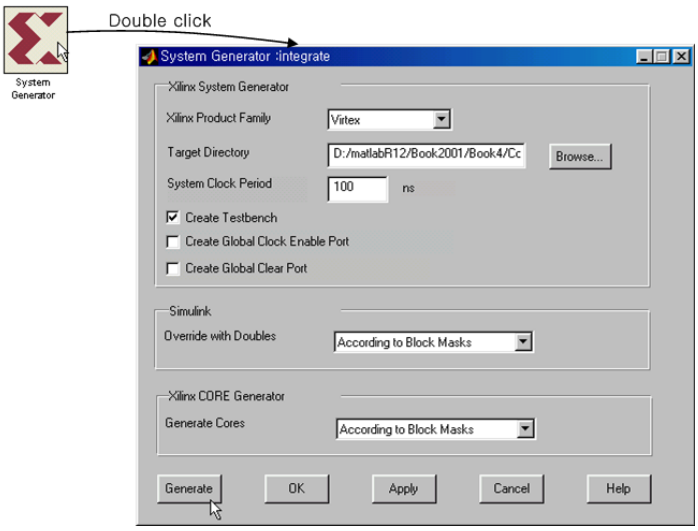


[그림 19-79] [그림 19-77]의 Simulation 결과.

[그림 19-79]에서 맨 윗 그림은 sine wave에 AWGN이 첨가된 모습이고, 중간 그림은 low-pass filter를 통과한 signal의 파형을 보여주고 있다. 그리고, 맨 아랫 그림은 quantization error를 보여주고 있다.

[그림 19-79]로부터 원하는 결과를 얻었으면, [그림 19-77]에서 보여준 Simulink model을 VHDL Code로 변환해 보도록 하자.

Simulink Mode을 VHDL Code로 변환 해 주려면, [그림 19-80]과 같이 System Generator icon을 이용해야 주어야 한다.



[그림 19-80] System Generator Window.

System Generator icon을 double click해 주면, **System Generator window**가 나타나는데, 여기서, 사용할 Xilinx device를 선택해 주고, 생성될 VHDL 관련 files가 위치할 target directory를 지정해 준다. 그리고, device가 작동할 동작 주파수를 **System Clock Period**에 지정해 주고, **Create Testbench** checkbox를 click해 준다. 이제, **generator** button을 click 해 주면, VHDL 관련 files가 생성될 것이다.

이제, [그림 19-81]과 같이 **generation completed** message가 나타나면 모든 VHDL 관련 files가 재대로 만들어 진 것이다.



[그림 19-81]

파일 탐색기를 사용하여 target directory에 만들어진 files를 보도록 하자.
아마도, 수십개의 files가 만들어져서 조금은 당황 스러울지도 모른다.

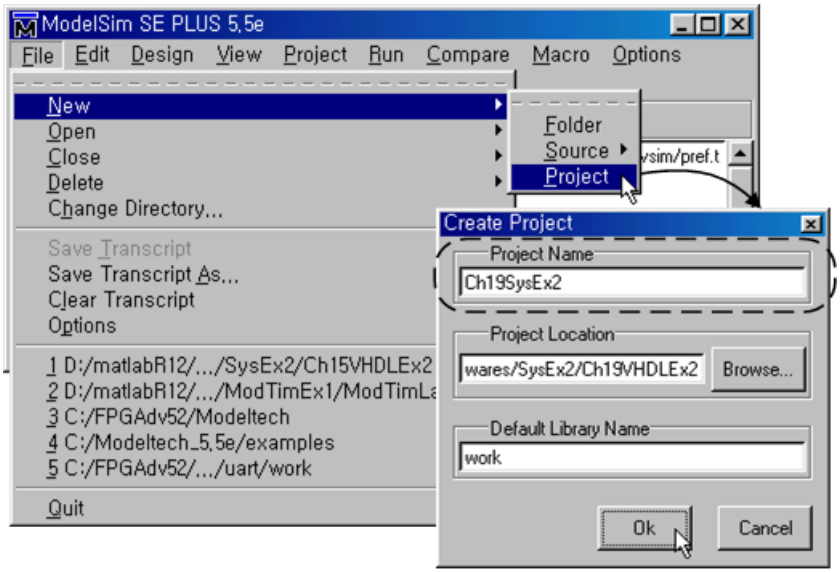
여기서, ***.v** file은 [그림 19-77]에 대한 verilog files이고, ***.edn** file은 Xilinx Core generator로부터 만들어진 관련된 file들이다. 그러므로, 생성된 VHDL(또는 Verilog) files를 합성하기 위해서는 반드시, 합성할 VHDL files와 동일한 directory에 *.edn files가 있어야 한다. 그리고, ***.ucf file**은 Xilinx의 Constraint file이고, ***.tcl** files와 ***.do** files는 leonardo spectrum과 modelsim을 사용하여 합성 및 simulation을 할 때 사용할 macro files이다.
integrate_synplicity.prj file은 Synplify를 사용하여 합성할 때 사용할 project file이고, integrate_xst.prj는 Xilinx Foundation Series에 있는 합성 tool인 **XST**(Xilinx Synthesis

Technology)를 사용하여 합성할 때 사용할 project file이다.

아마도, Xilinx Foundation Series는 4.1i version으로 단종되고, Xilinx ISE Series만 Xilinx software로 사용할 수 있을지도 모른다. **integrate.npl** file은 Xilinx ISE 4.1i에서 사용할 project file이다.

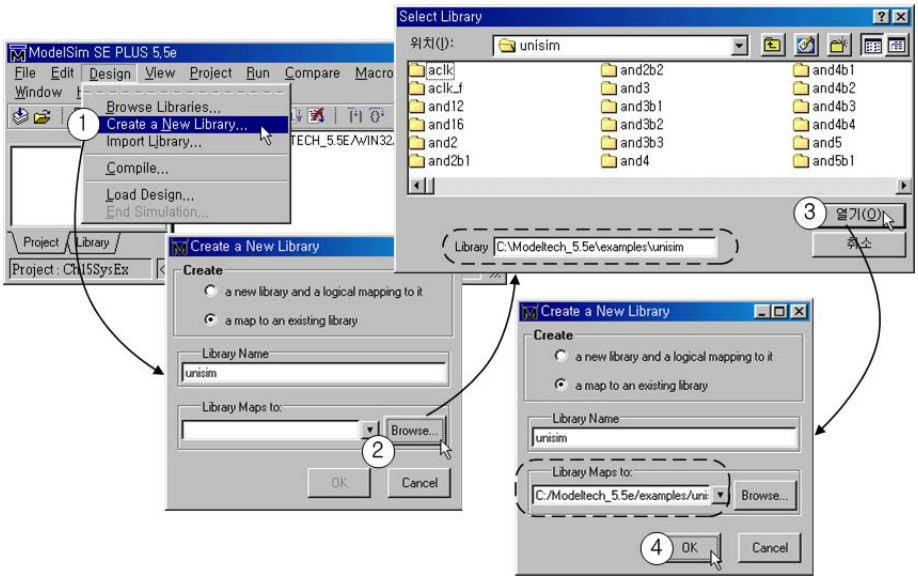
어쨌든, Modelsim을 사용하여 생성된 VHDL files에 대한 functional simulation을 수행해 보도록 하자.
System Generator를 사용하여 만들어진 VHDL files를 modelsim에서 simulation을 하려면, [그림 19-46]과 [그림 19-47]에서 보여준 것과 같이 **unisim, simprim, xilinxcorelib**가 사전에 사용할 modelsim에 의해서 compilation되어 있어야 한다.

일단, Modelsim을 실행한 뒤에, target directory로 modelsim의 경로를 바꾸어 준 뒤다. 그리고, [그림 19-82]와 같이 **Ch19SysEx2** project를 새로 만든다.



[그림 19-82] **Ch19SysEx2** project 생성 방법.

project를 새롭게 만들었으면, compiling하는 데 필요한 unisim과 xilinxcorelib library를 [그림 19-83]과 같이 mapping해 준다.

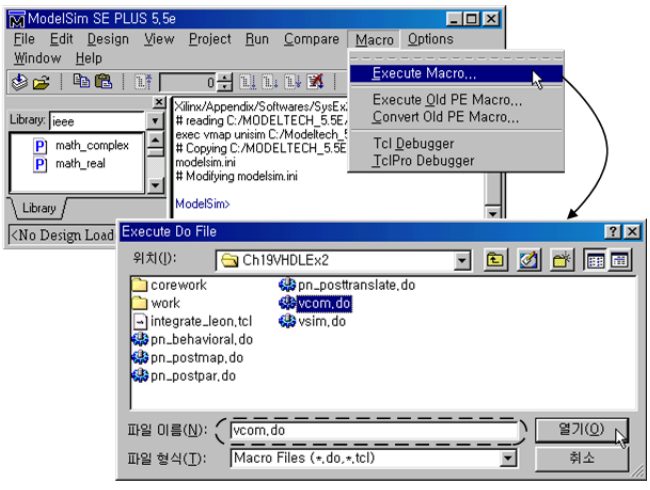


[그림 19-83] unisim library mapping 방법.

[그림 19-83]은 unisim library의 mapping 방법을 보여주고 있다. 같은 방식으로 xilinxcorelib library를 mapping 해 준다.

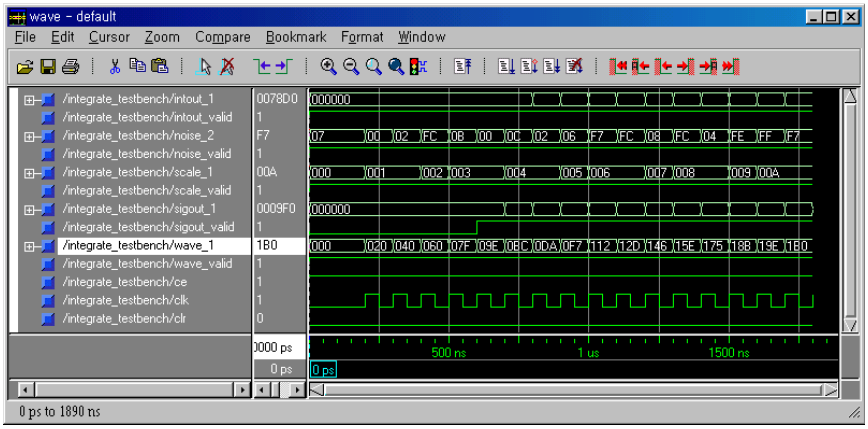
이제, functional simulation을 수행할 VHDL files를 System Generator가 만들어준 **vcom.do** file을 사용하여 compiling 해 보자.

[그림 19-84]는 vcom.do file을 사용하여 compiling 해 주는 방법을 보여주고 있다.



[그림 19-84] vcom.do file을 사용하는 방법.

compiling을 모두 수행하는 데 약간의 시간이 걸릴 것이다.
이제, error없이 모든 compiling이 끝났으면, [그림 19-84]와 같은 방법으로 **vsim.do** file을 사용하여 simulation을 수행해 보도록 하자.

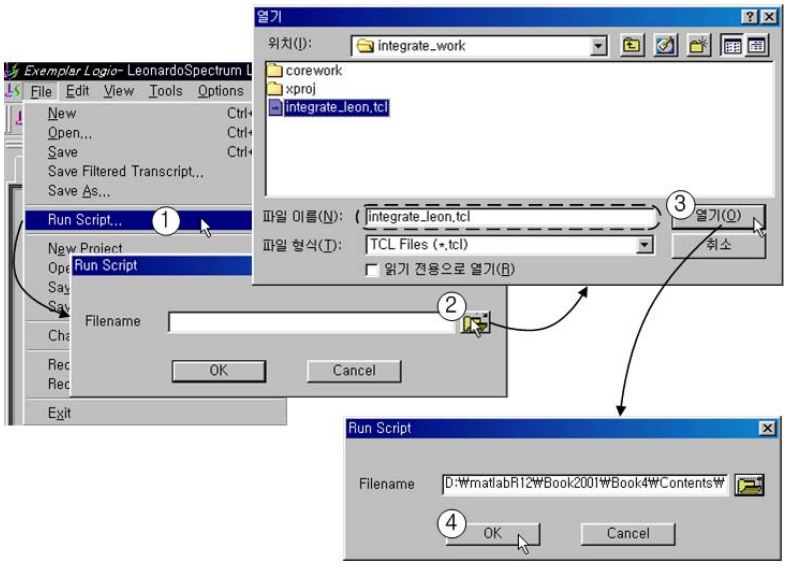


[그림 19-85] [그림 19-77]에 대한 functional simulation 결과.

[그림 19-85]에서는 모든 bit stream을 16진수 표현하였다. 그러나, 이들 16진수는 [그림 19-79]와 같이 분명하게 simulation 결과가 눈에 들어오지 않는다.
사실, modelsim만을 사용하여 simulation을 하는 경우에는 모든 bit stream이 수치로만 표현되므로, [그림 19-79]와 같이 눈에 분명한 그래프 표현을 얻을 수는 없다.

이와 같은 경우에는 modelsim의 list window로부터 simulation data file을 얻어서 matlab과 같은 그래픽 tool로 표현하여 그 결과 값을 관찰하는 방법을 즐겨 사용한다.

어쨌든, Functional Simulation 결과가 원하는 데로 나왔으면, integrate_leon.tcl macro file을 Leonardo Spectrum에서 [그림 19-86]과 같은 방법으로 사용하여 합성을 해 주면 된다.



[그림 19-86] Leonardo Spectrum을 사용한 합성 방법.

모든 합성이 완료되었으면, target device에 구현하기 위해서 P&R을 수행해야 한다.
Xilinx의 design manager를 사용하여 P&R을 수행하기 위해서는 합성 결과로 만들어진 integrate.edf file과 System generator가 만들어준 ***.edn** file이 동일한 directory에 있어야

한다.
design manager를 사용하여 bit stream code를 만들고, 이 bit stream code를 PROM file Formatter로부터 PROM에 저장할 수 있는 *.mcs file을 만드는 방법, 그리고 iMPACT tool을 사용하여 target device에 구현하는 방법은 앞에서 설명한 방법과 동일하다.

지금까지 여러분과 함께 VHDL에 대한 기본적인 내용과 관련된 software들을 사용하는 방법을 알아보았다.
비록, VHDL에 대한 syntax와 이론적인 내용에 대해서 심도 있게 다루지는 않았지만, VHDL이 무엇이고, 어떻게 simulation하는지, 그리고, target device에 구현하는 방법에 대해서는 충분히 설명하였다고 생각한다.
물론, Xilinx device 외에도 Altera, Lattice와 같은 많은 device들이 있지만, 임의의 device에 상관없이 modelsim, Active-HDL, Leonardo Spectrum의 사용 방법은 동일하다. 그러므로, Xilinx의 design manager에 상응하는 Altera. Lattice program만 학습한다면, 어려움 없이 개발한 VHDL Code를 해당 device에 구현할 수 있을 것이다.

보다, 깊이 있는 VHDL에 대한 syntax와 이론적인 내용을 학습하고 싶은 분들은 【참고 문헌】 2번을 보고 바란다. 그리고, 필자의 동생이 운영하는 http://matlabschool.com/도 여러분의 VHDL 학습에 도움이 될 것이라고 생각한다.

【VHDL 용어 정리】

*** Architecture :**

simulation될 모든 entities는 architecture를 가진다. architecture는 entity의 작동양식(behavior)을 서술하는 부분이다.
한 개의 entity는 여러 개의 architecture를 가질 수 있다.
즉, 한 개의 architecture는 entity의 움직임을 서술하고, 또 다른 architecture는 구조(structure)를 서술할 수도 있다.

*** ASIC :**

Application Specific Integrated Circuit.

*** Attribute :**

attribute는 VHDL object에 도달하는 data, 또는 VHDL object에 대해서 미리 정의된 data를 말한다. 예를 들면, buffer의 현재 drive 능력, 또는 device의 최고 작동 온도를 말한다.

*** Bus :**

bus는 신호 group 또는 hardware 상의 design상에서 사용되는 특별한 통신의 방법을 말한다. 즉, 여러 개의 node들을 표현하는 Graphic editor file에서 사용되는 thick line을 말한다. bus는 design(회로) 상의 여러 성분들 사이에서 다중 신호를 운반하는 데 사용된다.

*** Component :**

VHDL에서 primitive 또는 macrofunction의 **port**들을 말한다.
그러므로, component는 primitive 또는 macrofunction의 이름과 이들 logic function의 입·출력들에 대한 list로 구성된다. 구체적으로 Component는 실질적인 회로소자의 내부에서 사용할 회로 IC를 말한다. 그리고, Components는 Component Declaration에서 지정된다.

*** Component instantiation :**

선언된 component의 port signal에 실제 signal을 연결(mapping)시키는 것을 말한다.

*** Configuration :**

임의의 Configuration문은 component instance를 entity-architecture pair에 묶기 위해서 사용된다. 즉, VHDL components의 instances를 entities에 mapping하기 위해서 사용된다.
결국, 회로(design)에 대한 part(부품) list로 간주된다.
그리고, 어떻게 여러 개의 design entities가 하나의 완성된 design(회로)을 형성하기 위해서 연결되어 있는가를 서술한다.
그러므로, Configuration Declaration은 각각의 entity에서 사용할 architecture를 서술한다.

*** CPLD(Complex Programmable Logic Device) :**

내부에 여러 개의 LAB(Logic Array Block)과 LAB의 연결선인 PIA(Programmable Interconnection Array)로 구성되어 있다. 내부에 RAM, ROM 및 FIFO등의 특수 형태가 존재 할 수 있다.

*** Design unit**(설계단위) :

VHDL 문장에서 **독립적으로 분석 될 수 있**는 단위의 문장들을 말하며, 다음과 같은 5가지 설계 단위가 있다.
즉, entity 선언, architecture body, configuration 선언, package 선언 및 package body이

있다.

*** Driver** :

신호의 source를 말한다. 만일, 신호가 2개의 3상태(tristate) inverter에 의해서 구동된다면, 그 2개의 inverter가 active 일 때, 그 신호는 2개의 drivers를 가진다고 한다.

*** Entity** :

모든 design들은 entities항으로 표현됩니다. Entity는 회로에서 block을 만드는데 가장 기본이 된다. 회로의 가장 윗쪽 level은 **top-level entity**라고 부른다.

만일, 회로가 계층구조(hierarchical structure)를 갖는다면, top-level은 그것에 포함되는 lower-level을 가질 것이다. device 소자마다 하나씩 entity가 존재한다.

*** FPGA(Field Programmable Gate Array)** : 연결 상태에 따라서 라우팅 delay가 다르게 되어 timing 예측이 어렵고 논리 소자의 기능 또한 CPLD의 구조에 비해 떨어진다. 그러나, 사용자가 사용할 수 있는 **Gate 용량이 대규모**라는 장점이 있다.

*** Generic** :

information을 entity에 전달하는 parameter를 말한다.

예를 들면, 만일, entity가 rise 그리고 fall delay를 가진 gate level model이라면, rise 그리고 fall delay에 대한 값들은 generic을 통하여 entity에 전달된다.

임의의 parameterized design entity의 instance는 parameter 이름들과 그 instance의 function 값을 지정하는 Generic Map을 가져야 한다. Generic은 VHDL 전문 용어 이다.

*** Instance** :

design file에서 primitive 또는 macrofunction의 이용(use)을 말합니다.

즉, Graphic editor에서는 symbol ID number를 의미하고, Waveform editor에서는 node의 이름을 의미합니다. instance는 Component Instantiation문에서 선언됩니다.

*** Library** :

이전에 설계한 설계 단위(design unit) 즉, entity, architecture, package 등을 저장하여 필요시에 사용할 수 있도록 하는 일종의 저장장소이다.

*** LRM** :

VHDL Language Reference Manual. 이 manual은 VHDL 개발자를 위한 것이지 VHDL를 배우려는 초보자를 위한 것은 결코 아니다.

*** Package** :

design 상에서 자주 사용되는 subprogram들과 data type들을 모아놓은 matlab의 toolbox와 같은 것을 말한다. 부프로그램으로는 **function**과 **procedure**가 있다.

*** PLD(Programmable Logic Device)** :

내부 논리 기능이 고정되어 있는 것이 아니라, 사용자의 목적에 따라 가변적으로 논리 기능을 구현할 수 있다.

*** Primary unit** :

외부와의 interface를 위하여 선언하는 entity선언 또는 procedure 선언과 같은 것을 말한다.

*** Process** :

VHDL에서 실행(execution)의 기본 단위를 말한다.

VHDL의 simulation에서 수행되는 모든 operations는 한 개 또는 여러 개의 process들로 나뉜다. process는 Process문으로 정의된다. process문은 concurrent문이다. 그러나, process문안에 있는 문(statement)들은 순서대로 실행되는 sequential 형태이다.

*** Second unit :**

primary unit에 비해서 몸체에 해당하는 architecture와 package body를 말한다.

*** VHDL :**

Very High Speed Integrated circuit Hardware Description Language.

【참고 문헌】

1. Alan V. Oppenheim · Ronald W Schafer의 “Discrete-Time Signal Processing” Prentice Hall, 1989

2. Charles H. Roth, Jr.의 "Digital Systems Design Using VHDL" PWS, 1998